



同濟大學
TONGJI UNIVERSITY

高级语言程序设计（进阶）

期末大作业项目报告

项目名称	SciFlow		
学 院	国豪书院		
专 业	工业设计（精英班）		
学生姓名	陈芮哲	学 号	2552396

2026 年 6 月 22 日

摘要

SciFlow 是一款面向科研制图场景的节点式图像处理 workflow 工具。它将图像处理、数据处理、图表生成、流程图绘制、科研排版等多种任务统一在一个可视化平台之内，用户以拖拽节点、连线组合的方式搭建处理流程，参数化配置后一键执行，并可将流程保存为文件反复复用或经命令行批量执行。工具共实现了 18 大类、135 种节点，覆盖从图像读写、几何变换、色彩调整、滤波、形态学、边缘检测，到数据清洗、统计、十余种图表、流程图自动布局、学科装饰图案、艺术字标题与科研标注的完整流程。系统以 C++17 编写，基于 Qt 6 构建图形界面，调用 OpenCV 4 完成图像算法，采用 QCustomPlot 与 Qt Charts 渲染图表，并提供 GUI 交互搭建与 CLI 批量执行两种共享同一套处理与 workflow 执行逻辑的运行模式。

在架构上，本项目围绕五种端口数据类型（Image、ImageList、DataTable、Numeric、Text）与统一的智能指针数据载体 NodeData 展开，划分为 common、core、nodes、io、gui、cli 六个单向依赖的模块。核心采用抽象基类与工厂注册机制，使新增节点对既有框架不产生影响；执行引擎独立于具体节点，并以此支撑了 ListIterator 迭代与条件路由等高级控制流。系统从端口、逻辑、程序三个层面保证兼容性，并通过参数校验、连线校验、执行前校验与异常捕获构建起异常处理体系。

本报告还记录了一次区别于传统开发的尝试：作者全程不编写一行代码，而是先经约 60 轮人机对话将构想打磨为一份约四万字、消除一切歧义的需求.md，再由 Claude Code 据此实现，开发与测试中遇到的每一处问题与修改均同步写回文档，使其始终是程序权威而唯一的描述。有关人机协同的创新开发方式与人的主体性讨论，构成了本项目在技术成果之外的另一探索价值。

目 录

摘 要	1
分析文档	
一、题目解读与初稿构思.....	12
1.1 题目要什么	12
1.2 我想要什么	12
1.3 现在缺什么	12
1.4 最后做什么	12
二、与 AI 的交互内容	14
2.1 交互目的	14
2.2 交互方式及大模型选取	14
2.3 对话轮次拆解	14
2.3.1 确定方向	14
2.3.2 敲定骨架	14
2.3.3 逐条决定	14
2.3.4 细化实现	15
2.4 对话内容	15
2.4.1 节点职责与子类机制	15
2.4.2 多类型输入端口	15
2.4.3 图表的实现方案	16
2.4.4 ImageList 的流转方式	16
2.4.5 界面与交互	16
2.4.6 工具的价值	16
三、关键非代码内容的敲定.....	18
3.1 设计目标	18
3.2 用户体验	18
3.2.1 布局设计与配色方案	18
3.2.2 交互设计与快捷键	19

四、撰写需求.md.....	20
4.1 设计阶段的延伸	20
4.2 开发阶段的驱动	20
4.3 测试阶段的补全	21
4.4 Certainty is all you need	21
五、AI 工具使用说明	22
5.1 为什么要一份事无巨细的文档	22
5.2 一次面向 vibe coding 时代的尝试.....	23
5.3 可是，代价是什么	23
5.4 不可替代的两种能力	24

项目介绍

一、数据类型与结构设计.....	26
1.1 数据类型定义	26
1.2 数据结构	26
二、架构设计.....	28
2.1 模块划分	28
2.2 关键模块	28
2.3 数据流	29
2.4 模块依赖	30
三、关键类设计.....	31
3.1 core 模块的核心类.....	31
3.1.1 NodeData 类	31
3.1.2 DataTable 类.....	31
3.1.3 PortDefinition 类	31
3.1.4 Port 类.....	31
3.1.5 NodeBase 类.....	32
3.1.6 NodeFactory 类	32
3.1.7 WorkflowGraph 类	32
3.1.8 ExecutionEngine 类.....	33
3.2 nodes 模块的核心类	33

3.2.1 ListIterator 类	33
3.2.2 ConditionalRouter / DataConditionalRouter 类	34
3.3 gui 模块的核心类	34
3.3.1 窗口骨架	34
3.3.2 画布图形项	35
3.3.3 面板	35
3.3.4 自定义控件	35
3.4 io 模块的核心类	35
3.4.1 WorkflowSerializer 类	35
3.4.2 ImageIO 类	35
3.4.3 DataIO 类	36
3.5 cli 模块的核心类	36
3.6 common 模块的核心类	36
3.6.1 Logger 类	36
3.6.2 NodeProcessException 类	36
3.6.3 Utility 类	36
四、兼容性设计	37
4.1 端口兼容性	37
4.2 逻辑兼容性	37
4.3 程序兼容性	38
五、防呆设计	39
5.1 常见的异常操作	39
5.2 异常处理	39
5.2.1 校验	39
5.2.2 捕获	40
六、用例设计	41
6.1 测试用例设计	41
6.1.1 测试用例	41
6.1.2 覆盖维度	42
6.1.3 设计思路	42

6.2 展示用例设计	42
6.2.1 展示用例	43
6.2.2 设计原则	43
七、主要问题与解决方案	44
7.1 主要问题	44
7.2 解决方案	44
7.2.1 删除节点崩溃	44
7.2.2 节点图形项重复构建	45
7.2.3 属性面板弹窗	45
7.2.4 第三方图表库不合预期	46
7.2.5 编码问题	46
八、心得体会	47
8.1 做一个面向用户的工具	47
8.2 做一次新颖大胆的尝试	47
九、学年回顾暨致谢	49

需求.md

第一章 项目理解与设计目标	53
1.1 项目背景	53
1.2 设计目标	53
1.3 技术选型	54
第二章 数据类型规范	55
2.1 端口数据类型定义	55
2.2 端口兼容性规则	55
2.3 端口颜色标识	56
第三章 完整节点列表	57
3.1 第 1 大类：图像输入输出	57
子类：文件读取	57
子类：图像生成	58
子类：文件导出	60
子类：批量处理	61

3.2 第 2 大类：几何变换	62
子类：基础变换	62
3.3 第 3 大类：色彩调整	64
子类：基础调整	64
子类：HSL 调整	65
子类：高级调整	66
3.4 第 4 大类：滤波与模糊	67
子类：模糊	67
子类：锐化	68
子类：增强	69
3.5 第 5 大类：形态学操作	69
子类：基本运算	69
子类：复合运算	70
3.6 第 6 大类：边缘检测与分割	70
子类：边缘检测	70
子类：阈值分割	71
子类：轮廓	72
3.7 第 7 大类：通道操作	72
子类：拆分合并	72
子类：色彩空间转换	73
子类：Alpha 通道	73
3.8 第 8 大类：图层与合成	74
子类：混合	74
子类：拼接	75
子类：水印	76
3.9 第 9 大类：文字与标注	77
子类：文字	77
子类：科研标注	79
3.10 第 10 大类：数据输入输出	81
子类：数据读取	81

子类：数据导出	82
3.11 第 11 大类：文本/数值输入输出	83
子类：文本输入输出	83
3.12 第 12 大类：数据处理	84
子类：数据清洗	84
子类：数据变换	85
子类：数据统计	86
子类：高级数据处理	86
3.13 第 13 大类：图表生成	87
子类：趋势类	88
子类：对比类	88
子类：占比类	89
子类：分布类	89
子类：高级图表	91
3.14 第 14 大类：风格化与特效	91
子类：艺术效果	91
子类：色调效果	92
子类：纹理效果	93
3.15 第 15 大类：版面排版与装饰	93
子类：排版	93
子类：装饰	95
子类：对比	96
子类：显示	96
3.16 第 16 大类：图形绘制与流程图	97
子类：画布	97
子类：基础形状	97
子类：连接件	98
子类：流程图	99
子类：装饰	100
3.17 第 17 大类：实用工具与高级	101

子类：信息提取	101
子类：流程控制	102
子类：外部接口	104
3.18 第 18 大类：几何图案生成	105
子类：通用图案	105
子类：计算机科学	105
子类：化学	106
子类：生命科学	106
子类：社会科学	107
子类：自定义素材	107
第四章 图形界面布局设计	108
4.1 整体布局	108
4.2 工具栏	108
4.3 节点工具箱	109
4.4 画布编辑区	110
4.4.1 节点图形元素	110
4.4.2 连线交互	111
4.4.3 右键菜单	111
4.5 属性面板	111
4.6 预览与日志面板	113
4.7 配色汇总	114
4.8 线框图	115
第五章 工作流文件格式设计	116
5.1 文件格式	116
5.2 JSON 结构	116
5.3 nodes 数组中每个元素的结构	116
5.4 connections 数组中每个元素的结构	117
5.5 execution_order_hints 数组	117
5.6 文件格式校验	118
5.7 完整示例	118

第六章 整体架构设计..... 122

6.1 模块划分 122

6.2 类层次结构 122

6.2.1 core 模块的核心类 122

6.2.2 nodes 模块 124

6.2.3 gui 模块 124

6.2.4 io 模块 124

6.2.5 cli 模块 124

6.2.6 common 模块 125

6.3 数据流说明 125

6.4 撤销重做实现 125

6.5 模块依赖关系 125

第七章 异常处理策略..... 126

7.1 异常分类 126

7.2 处理策略 126

7.3 校验时机 126

7.4 GUI 模式与 CLI 模式的异常呈现..... 127

第八章 命令行模式..... 128

8.1 命令行接口 128

8.2 执行流程 128

8.3 使用示例 128

第九章 内置 workflow 模板..... 129

9.1 模板存储 129

9.2 三个内置模板的具体 workflow 129

模板一：论文四子图排版 129

模板二：批量图片加水印 129

模板三：数据分析出图 129

第十章 补充设计细节..... 130

10.1 节点 ID 生成规则 130

10.2 节点复制粘贴 130

10.3 自动布局算法	130
10.4 DPI 信息写入	130
10.5 多类型端口的参数面板动态行为	130
10.6 执行进度显示	131
10.7 连线路径计算细节	131
10.8 画布网格吸附	131
10.9 键盘快捷键汇总	131
10.10 程序启动流程	132
10.11 PatternBase 图案基类设计	132
10.12 渐变编辑器控件封装	132

分析文档

一、题目解读与初稿构思

1.1 题目要什么

这道题的核心诉求是要求实现一个类似 LoRA 的东西，把一系列固定的、可复用的处理步骤抽象成可组合的模块，使用户能够以搭积木的方式定义自己的工作流，而不用每次都从零开始手工操作或临时编写脚本。

题目在功能上给了很大的自由度，而且要求并不高，最低只要求 8 种节点。此外，题目在代码上约束得相当明确，这些约束实际上规定了一份合格的作业在架构层面应当长什么样，可以说是一道半客观的主观题。

1.2 我想要什么

最后一舞了，我不想只交一个作业，哪怕是一份制作精良的作业。

前九个作业里，我的作品始终未能跨出作业的范畴，虽然拓扑图和贪吃蛇的完成度很高，但它们也很难带到课堂之外。我没办法拿着我的拓扑图去用来做工程，也没办法拿着我的贪吃蛇上架 Steam。它们是好的练习，是好的作业，却不是好的产品。这一次，我想在满足作业要求之上，做一个能拿出课程的东西，一个真正有人愿意用的实用工具。这是我对这次作业的期望，成品的价值不应止于这门课。

1.3 现在缺什么

在确定了“要做一个有用的工具”之后，随之而来的问题是：做给谁，解决什么问题。

我注意到，科研制图是一个典型的高耗时、低价值的环节。研究者在论文配图、数据出图、原始图像处理上反复手工操作，流程繁琐。同一类显微图片往往需要同样的去噪、增强、加比例尺操作，同一批实验数据往往要画同样的折线图、柱状图、热力图，但每次都得从头来过。这些操作单看都不复杂，但它们重复、琐碎、占用时间，而且无法复用。此外，这类工作往往散落在多个软件里，处理图像要开一个软件，绘制图表要开另一个，排版又要换一个，数据清洗可能还得写脚本。据我所知，市面上尚缺少一个能将图像处理和图表生成统一起来，并支持把这些重复流程搭建为可复用流水线的可视化工具。而题目要求的节点式、可保存、可批处理这三个特性，恰好符合这些诉求。

1.4 最后做什么

据此，我确定了设计目标，做一个面向科研场景的节点式图像处理 workflows 工具。

在题目的基础框架之上，我将目标用户确定为科研工作者，尝试构建一个集成了图像处

理、数据处理、图表生成、流程图绘制、科研排版的一站式处理工具。用户搭一次流程，之后对同类素材一键复用；GUI 用于交互式搭建和调试，CLI 用于批量执行。这既满足了题目的要求，又满足了我做一个真正有用的工具的愿望。

这个工具最终被命名为 SciFlow。

二、与 AI 的交互内容

2.1 交互目的

在方案与设计阶段，我希望能通过与 AI 交互，将脑海中尚不清晰的构想打磨成一份事无巨细的需求文档。这么做的原因在本篇第五章中另有详述。

2.2 交互方式及大模型选取

设计阶段选用了对话式大模型，以常规的对话的形式逐轮推进，模型选择了 Claude Opus 4.6，动用了一定的钞能力。

在交互过程中，AI 充当了三种角色。其一是评审，负责检查方案里有无遗漏、矛盾或不合理之处；其二是顾问，在我定好方向的前提下，提出我可能未曾接触过的实现方式和新想法，再由我裁定；其三是写手，通过整理过往对话记录，撰写完整的需求.md。为了让讨论尽可能充分、歧义尽可能少，我在预设中要求它在每轮对话末尾主动向我提出更全面的问题（每轮要求了 7 个封闭式问题和 3 个开放式问题），直到我明确要求停止为止。

2.3 对话轮次拆解

整个设计阶段大约进行了 60 轮对话。总体的推进原则是从大到小、从设计到实现，上一层确定无误之后再进入下一层。这一过程大致可分为四个阶段。

2.3.1 确定方向

我将题目中的具体场合与若干关键词作了替换，把橙子质检的场景改写为科研制图的任务，再提交给模型，请它评估面向科研制图实现一个节点式工具是否具备价值、市场是否存在需求、技术上能否落地。在得到肯定的答复与充分的依据之后，我成功按设想进入了下一阶段。

2.3.2 敲定骨架

我构思了一份初步方案，设计了端口的数据类型、若干大类的节点划分、六个顶层模块的架构、节点间传递的数据流，以及 JSON 工作流的初步格式，随后提交给 AI 审核，让它识别其中的遗漏、矛盾与不合理之处。这一阶段我得到了若干细致的反馈与追问，它们构成了方案可行的基础。

2.3.3 逐条决定

针对审核中暴露的若干要点，我逐一作出决策。每一项决策一旦确定，便成为后续讨论

不可更改的前提。至此，整个设计方案、方向与方式得以确定，后续流程的重心便转移到了实现方面。

2.3.4 细化实现

在大方向锁定之后，我用几十轮对话将每一种节点的端口、参数、取值范围、默认值、处理逻辑与异常分支逐一打磨，直至其细化到可以仅通过需求.md 无歧义地得以实现为止。

总体而言，在对话之前，我对对话的展开轮次与进程进行了规划，每一轮都带着一个明确的问题开始，得到反馈后由我判断采纳与否，再将结论固定下来，成为下一轮对话的基础与依据。如此层层叠加，方案从一份初步的设计，逐步成长为一份约四万字的完整需求文档。

2.4 对话内容

设计阶段，整个交互始终遵循的原则是在我清楚地知道自己需要什么、为何需要、希望得到什么的前提下，由 AI 承担审查、补全与优化的工作。我设计的端口类型、节点划分、架构骨架与数据流转方式初稿构成了讨论的出发点，模型的职责是在这一蓝本之上挑刺与优化。以下选取几处关键决策加以说明。

2.4.1 节点职责与子类机制

我在初版中曾打算将膨胀、腐蚀、开运算、闭运算合并为单一的形态学节点，通过参数切换具体行为，色彩空间转换也想以一个带下拉框的节点进行实现。模型在审核时识别到了其不合理之处，并追问我这样的合并是否与节点式的初衷相悖。实际上，节点式工具的价值正在于每个节点职责单一、边界清晰，将四种操作塞入一个节点，等于把复杂度从画布转移到了参数面板，反而违背了搭积木的出发点。我随即决定将其拆开，一种操作对应一个节点。考虑到拆分会使节点数量显著膨胀，为了索引方便，我同时确立了大类、子类、具体节点的三级分类方式，例如“图像处理-形态学操作-膨胀”。这一关键决策既指导了我后续节点的职责设计，又奠定了工具箱的层级结构。

2.4.2 多类型输入端口

最初我在设计节点时，先后集中设计了其内部处理逻辑与允许的输入类型。由于前后间隔了一定时间，直方图节点被我设计为允许同时接受图像与数据表两种输入，但内部逻辑只写了一套。模型发现了这一问题，指出此类多类型端口需想清楚每种输入各自的处理路径，否则即便连线时类型校验通过，进入节点后仍然可能因为不知道按哪套逻辑执行而出错。我据此补上了约束，在支持多类型的同时，每种输入类型对应的处理逻辑必须明确定义，这在

后续的多类型输入节点设计中避免了很多麻烦。而直方图节点的处理逻辑也被正确划分为两条路径，输入为图像时调用 `calcHist` 计算，输入为数据表时执行分箱统计。

2.4.3 图表的实现方案

由于学习不充分，凭着朴素的直觉，我原计划以 `QtCharts` 渲染全部图表，但 AI 指出部分图表的绘制需要调用 `Python` 子进程，而这会使工具依赖一个完整的 `Python` 环境，大大提升了运行的复杂性，遂考虑修改。我设定的允许范围是：允许引入 `OpenCV` 与 `Qt` 之外的第三方库，但不得调用其他语言的子进程，并请它在此范围内寻找更优的方案。经此一轮，图表方案落定为 `QCustomPlot` 与 `QtCharts` 的组合。前者负责折线图、柱状图、散点图、直方图、箱线图与热力图，后者负责饼图、环形图与雷达图，全部在进程内渲染至 `QImage`，提升了程序的独立性和统一性。

2.4.4 ImageList 的流转方式

批量处理时，若要对列表中每张图执行同一套子流程，直观来讲有两条路径：由用户手动拆分、处理、重组，或由程序自动对列表元素循环执行下游子图。显然后者更难实现，但我仍然选择了后者，原因有二。其一是前者会大幅提高用户进行批处理操作的复杂度，这对于大批量图片的处理来说是不可接受的。其二是该复杂程度仍在 AI 能处理的范围之内，多提点要求并不过分。这一决策的结果是执行引擎必须独立且支持迭代，并最终催生了 `ListIterator` 这一高度依赖于执行引擎的特殊节点，其迭代模式、回接端口与结果收集机制皆源于此。

2.4.5 界面与交互

考虑到长时间注视屏幕的友好性，我确定了深色调主题，并通过一定的风格要求让 AI 生成了一套较为优秀的配色方案。此外，我们还一同设计了参数编辑的分级处理，对于不涉及长参数、长数据、长列表的简单节点，无需调用右侧属性面板，而是在节点上方弹出悬浮窗就地编辑，免去用户视线在画布与面板之间往复。版式、层级与交互动画也经由我们多轮交互和实验得以逐步优化，尽最大可能贴合用户的使用直觉，降低了程序的上手难度。

2.4.6 工具的价值

方案在功能上已经较为完整，但在讨论中仍有一个问题悬而未决：这个工具凭什么比现有方案更好用，或者说，用户是否真的会用起来。我的判断是，流程图、配图与路由这三部分是成败的关键。流程图的制作本身就是拉取节点、连接线条的工作，倘若工具仅提供手动摆放流程图元素的功能，相较专业绘图平台（如 `draw.io`）便毫无优势；配图亦然，若只停留

在图像处理层面，市面上的工具早已满足需求，用户没有换软件的理由；路由机制则更进一层，它让工作流从流水线变成了工厂，使得工具能根据每张图像或每份数据的实际特征，自动决定它该走哪条处理路径。对于动辄成百上千张、且夹杂着多种规格与要求的批处理任务，这种能力几乎是不可替代的。

为此，我设计了如下创新点。流程图采用描述驱动、自动布局的方式，用户填写步骤与逻辑关系，工具自动完成布局、连线、配色并出图（后来我才知道其实已经有专业平台 PicDoc 在做这件事了，而且他们是基于 LLM 的，支持自然语言描述）；配图则由图像处理扩展为图像绘制，不仅能修改既有图像，更能从零生成美观、清晰、大气的配图与封面；路由机制应支持条件判断与流程控制，从而构建复杂而严密的处理网络。这三个设计显著提升了工具的实用价值和能力上限，也是后续流程图自动布局节点、大量制图、排版、装饰图案类节点以及路由控制节点得以存在的出发点。值得说明的是，此处 AI 并没有起到实质性帮助。事实上，经由本次项目，我对 AI 的能力边界有了更深的体会。它能协助审查、补全逻辑、落实创意，但工具究竟应朝哪个方向发展，唯有人能够回答。我想，洞察力与创造力，或许是这个时代之下人不可替代的关键能力。

此外尚有若干较小的决策。Excel 读取方面，在 AI 的介绍下，通过仅支持.xlsx 放弃.xls 格式，我换取到了一个稳定的纯 C++读取库 QXlsx；考虑到项目的时间限制，外部 API 节点仅设计了接口，暂不具体实现实际网络调用。

最终，上述讨论所得的全部结论，均落实于需求.md 之中。

三、关键非代码内容的敲定

经过前述 60 轮的反复磨合，许多原本悬而未决的设计被逐一确定。第二章记录了多数决策的形成过程，本章将介绍最终敲定下来的关键非代码内容。

3.1 设计目标

在方向确定之后，结合原始题目已经给出的基本要求，我整理了七条硬性要求，并由 AI 润色优化，作为整份需求文档的基础。

第一条，节点化与可组合。所有操作一律封装为独立节点，每个节点具备明确的输入端口、输出端口与参数集合，节点之间通过连线传递数据，用户可通过组合节点搭建任意复杂的处理流程。

第二条，覆盖面广。节点类型不仅覆盖图像处理的各个环节，还应延伸至数据处理、图表生成、流程图绘制、科研排版、学科装饰图案生成、文字标注与风格化特效，总计 18 大类、135 种节点。

第三条，可扩展。架构上，应当通过抽象基类与工厂注册机制支持节点扩展，新增一种节点只需继承基类、实现处理逻辑、在工厂中注册即可完成，无须改动框架层的代码。

第四条，稳定健壮。在参数错误、文件缺失、端口类型不匹配、大图内存不足等异常场景下，程序必须给出清晰的提示，不允许崩溃。

第五条，持久化与批处理。工作流以 JSON 格式保存与加载，并支持命令行模式下不启动图形界面直接执行工作流文件。

第六条，交互体验优良。深色主题，画布支持缩放平移，连线以贝塞尔曲线按数据类型着色，简单节点就地编辑、复杂节点面板编辑，支持撤销重做、部分执行与输出预览，符合用户操作直觉。

第七条，面向科研制图的实用价值。工具不仅能处理既有图像，更应具备制图能力，绘制流程图、排版多子图配图、生成学科装饰图案、制作艺术字标题、处理结构化数据并生成统计图表，使用户能够从零构建一张完整的科研视觉作品。

3.2 用户体验

3.2.1 布局设计与配色方案

界面布局最终采用了无需学习成本的五区结构：顶部工具栏、左侧节点工具箱、中央画布、右侧属性面板、底部预览与日志面板。这一布局是绝大多数科研软件的通用布局，用户

初次打开便能凭直觉判断每块区域的职责。

在通用布局之外，我额外设计了两处优化细节。其一，右侧属性面板在未选中节点，或选中节点参数足够简单时会自动收起，使画布尽可能保持最大可用面积。其二，节点工具箱采用大类、子类、具体节点的三级结构，配合顶部含模糊匹配的搜索框、最近使用与收藏夹，使 135 种节点易于检索、层次分明。

配色上确定了深色主题，理由有二：科研制图与长时间工作的场景下，深色对眼睛更为友好；深色背景也能让带颜色的端口与连线更为醒目。在 AI 的辅助下，我为整套界面拟定了一份完整的配色表共二十余项，统一通过 QSS 样式表在启动时加载，以保证视觉体验的一致性。五种数据类型也分配了各自的固定颜色，它们既用于端口与连线，也作为色块标识出现在工具箱的节点名称左侧，便于用户快速辨认数据类型。

3.2.2 交互设计与快捷键

交互设计的出发点始终是降低学习成本和操作成本，让搭建工作流的过程尽量顺手。

连线操作上，有两条提升使用体验的设计：当用户试图为一个输入端口接入第二条连线时，旧连线自动断开、新连线接入，如此用户便无须手动清理旧线；连线在拖拽过程中实时校验类型兼容性，兼容的目标端口高亮放大，不兼容的则显示红色叉号，松手前用户便可知晓能否连接。

参数编辑沿用了第二章定下的分级思路，此处进一步明确了分级的判据：参数总数不超过 4 个、且不含曲线编辑器、表格编辑器、网格编辑器一类复杂控件的节点，归为简单节点，双击即在节点上方弹出悬浮面板就地编辑；其余复杂节点则展开右侧属性面板。判据写进文档后，每个节点便拥有了最优编辑方式。

此外，我还适配了较为通用的快捷键功能，覆盖新建、打开、保存、撤销重做、复制粘贴、全选、执行、删除等常用操作，符合用户操作习惯。围绕画布的若干交互也在此一并确定：滚轮缩放、中键平移、空白区域框选、移动时的网格吸附、按住 Shift 临时解除吸附。这些细节单独来看都微不足道，但它们共同决定了工具用起来流畅与否。

四、撰写需求.md

人们在谈论需求文档时，往往默认它有一个“写好”的时刻——初稿经过若干讨论，定稿，开发照此执行。但事实上，无论前期讨论多么充分，一份在动工前落成的文档，都不可能预见到实现与使用过程中遇到的所有问题。需求文档的撰写，是贯穿始终的，是不断修订的。

具体而言，这一过程由三个方面推动。其一是设计阶段尚未结束时的主动补充：在与 AI 打磨细节的过程中，我常常连带意识到另一处此前被忽略的环节，于是当即通过对话把新的内容补进文档。其二是开发阶段遇到问题的修复：当 Claude Code 依照文档编写代码时，不清晰、不明确的描述会引发若干崩溃与漏洞，修复之余，需要回头优化文档中含糊的措辞。其三是测试阶段使用体验的补全与优化：纸面上想当然的设计只有真正做出后才会暴露出它的不完美、不合理或不完整之处。

4.1 设计阶段的延伸

在对话尚未收尾、需求文档仍在成形的阶段，文档的扩充主要由 AI 启发推动。

这种补充往往是连锁的。例如，在确定图像导出节点的参数时，我意识到批量场景下需要有文件名模板，于是补上了“`output_{index}.png`”这样的命名规则。而一旦有了命名模板，又引出了“若模板中不含序号占位符、批量导出时如何避免相互覆盖”的问题，这又促使我补写了一条相应的异常处理方式。一个细节的确定，常常顺藤摸出三五个此前未曾想到的细节，这对于使用体验的帮助是巨大的。

然而，这一阶段的补充有一个致命问题，就是它们都还停留在纸面推演之上，是我在脑中模拟使用时发现的问题。它们重要，但人在动手之前能预见的问题终归有限，真正出人意料的漏洞，要等到代码跑起来之后才会出现。

4.2 开发阶段的驱动

文档扩充的主体，发生在开发与测试阶段。这一阶段的总体流程大致为：用 Claude Code 依照文档生成或修改代码，运行程序，进行测试，记录下所有不合预期、有问题或可新增之处，再把这些反馈反还给 Claude Code。而在最后一步，我要求它在修复或新增代码的同时，更新需求.md 中对应的条目，使文档始终与代码保持一致。这套流程既能避免文档与代码之间出现不匹配的现象，又能让二者协同进化。

具体来说，本阶段的反馈集中在“发现问题”这一类，比如某个节点在特定输入下崩溃、某条连线规则存在逻辑漏洞、某种参数组合产生了非预期的结果。这类问题根源往往是

文档在某个边界条件上语焉不详，给了实现以含糊的空间。例如，缩放节点在收到目标尺寸为零的参数时会崩溃，其原因是文档当初只写了目标宽高，却未规定其下界；又如多数汇合节点在两路输入尺寸不一致时产生了大面积的错误输出，是因为文档没有定义尺寸冲突时应当做何预处理。处理这类问题时，修复代码只是其一，更重要的是把文档完善，减小同类问题再次发生的可能。

4.3 测试阶段的补全

当程序具备雏形、致命错误被基本排查之后，测试的重心便从“对或错”转变为了“好或更好”。

本阶段的补全大致分为两种情形。第一种是发现不合预期的体验。比如图表节点的配色在深色背景下对比度不足；悬浮参数面板在节点位于画布边缘时会被窗口裁切，露不出完整的控件等等。

第二种是发现可新增的东西。在使用时，我经常 would 想“这里要是再有一个某某节点就好了”“这个节点如果能支持某种模式会更实用”“此处交互如果是某种样子会更顺手”。对于其中经论证合理的，我同样按上述流程加入到程序和文档中。

4.4 Certainty is all you need

改代码的同时必须改文档的规定，严格保证了需求.md 始终是整个程序最权威、最完整的描述。在一轮轮修改中，文档与程序在维持唯一性的前提下同步完善，任何人或 AI 据此重新开发，都应该得到相当一致的成品。这种思路还带来一个额外的好处，就是它把测试中得到的每一点经验都以清晰的文字形式保存了下来。文档因此不只是设计的载体，更是整个开发过程中所有经验的汇集之所。

在与大模型协作的语境下，算力和代码生成能力已经不再是稀缺品，可确定性却在交互过程和实现过程中大打折扣。语言描述存在其局限性，程序实现方式也并不唯一。只有把需求描述到不留歧义，才能得到可复现的结果。确定性，正是这本次项目开发的基石所在。

五、AI 工具使用说明

本篇的第二章和第四章已详细说明了设计阶段的 AI 使用情况，而本章将对开发阶段 AI 使用情况作一些说明与讨论。

5.1 为什么要一份事无巨细的文档

第二章曾提到，我希望借助与 AI 的交互，把脑海中尚不清晰的构想打磨成一份事无巨细的需求文档。这么做的缘由，其实是我决定全程使用 Claude Code 完成开发，自己不写一行代码。而这要求文档必须精确，不能留任何歧义。

“不写一行代码”这件事本身其实相当危险，特别是在不加任何说明和思考，直接将任务原封不动扔给 AI 的情况下。因为你并不知道 AI 究竟帮你做了些什么或聪明或愚蠢的决定，出了问题也不知道该上哪修，更多情况下，其实是 AI 给出了一个看起来还过得去、实则并不符合要求、也经不起实际使用的豆腐渣工程。针对这个问题，我希望能以一些实际的行动，作出自己的思考与回应。

当然，除了感性因素，这么做也有它的现实基础和意义。在传统的软件开发中，即便借助 AI，程序员也通常仍扮演着翻译者的角色，理解需求，在脑中将其转化为实现，再以代码或提示词的形式喂给工具，过程中夹杂着大量即兴的判断与发挥。这带来了两个问题，一是开发者在设计实现方式之外，还要额外承担功能设计责任，思维的来回切换使开发过程并不流畅；二是无数的即兴发挥使最终成品与预期大相径庭，倘若缺乏常态化的把控，方向很容易跑偏。基于此，我想做一个比较极端的尝试，把翻译这一步交给 AI，让人只负责设计，让 AI 只负责实现，二者之间仅以一份文档为桥梁。倘若这个尝试成立，那么上述的两个问题或许就能有一个解决方案。

支撑这一尝试的便是需求.md。它的编写原则只有一条，就是消除一切歧义。每一种节点的输入端口、输出端口、参数的控件类型、取值范围、默认值、条件可见性，乃至处理逻辑中的边界条件与异常分支，都被逐一写明；每一种端口的数据类型、兼容性规则、文件格式、模块划分、类层次、异常策略、命令行接口，也都精确到了实现者无需揣测的程度。我对它的要求是：任何一名开发者照此实现，都应当产出几乎一致的成品。文档越精确，留给 AI 即兴发挥的空间就越小，开发阶段的不确定性也就越低。这便是我一开始就执意要打磨出一份事无巨细文档的理由。

也正因如此，虽然我并未编写一行代码，但我对程序本身也依旧了解。架构是我划分的，节点是我定义的，每一处参数与逻辑都经由我手。Claude Code 依照文档生成代码，我依照预

期审查结果，发现偏差便指出、要求修正。可以说，除了敲下代码这一动作本身，其余的工作都由我负责。

5.2 一次面向 vibe coding 时代的尝试

当下所谓的 vibe coding，大多停留在“与 AI 对话、让它生成代码、复制粘贴”的状态。它确实快而省心，但也带来两个严重的问题。其一，人在这个过程中的主体性正在流失。需求是模糊的，实现是 AI 即兴的，表现是差强人意的，到最后连作者自己都说不清每段代码为何如此，人逐渐从开发者和设计者，退化为代码的旁观者与提示词的投喂者。其二，代码的安全性、与一致性难以保证。同一句模糊的指令，AI 每次生成的结果都可能不同，质量全凭抽卡，更不用说日后的审查与维护了。这两个问题，一个关乎人，一个关乎代码，但我认为它们的根源是同一个：需求不精确。

于是我尝试着解决这个问题。我选择把人的精力全部投入到上游，在动工之前先把设计想透、把需求写死，让一切想象力和决策空间在这一层面消耗殆尽。如此一来，人的主体性不降反升，我不用再受限于具体语法，而是站在了更高的位置，对整个系统的形态和方向负责。与此同时，代码的安全性、与一致性也有了着落，因为 AI 不用再揣测我的意图，它只是在执行一份已经没有歧义的要求，每一处实现都可在文档中的某一条找到依据。换言之，精确的需求同时回应了这两个问题，它把人从代码的细节里解放出来，还其设计者的本位，又用确定性约束住了 AI 的随机性。

我无意宣称这是 vibe coding 的正解，它有它的适用边界，也有它的代价。但我希望它至少是一份具有新意的尝试——一个普通的学生，用一个有一定规模的真实项目，去回答“人与 AI 究竟应当如何分工”的问题，并交出一个可被检验的答案。倘若这个答案能具有一定的启发性，那么这次开发的意义，便不止于交付了一个叫 SciFlow 的工具。

5.3 可是，代价是什么

除了最直接的作业分数代价外，我想在本节讨论这套方案的若干问题。

最直接的代价是前期投入。在写下第一行代码之前，我已经花了大量时间在讨论与文档上，这需要我在动工前就把每一个节点、每一处边界都尽可能考虑到。换作传统开发，许多细节是可以边写边想的，含糊的地方留到代码里再补，而现在，这种方式不允许这种拖延，所有的思考都躲不掉。这对人的要求其实更高，你无法给自己留下任何含糊不清、稍后再说的余地，必须真正理解你要的究竟是什么。

文本终究有它表达的极限。逻辑、参数、流程这些可以被精确描述的东西，文档能描述

得很好，但有些东西天生难以诉诸文字。配色好不好看，交互顺不顺手，界面美不美观。这些感觉层面的判断，无论文档写得多细，都只能等到亲眼看见成品才能裁定，很多感性的东西难以用文字表述。也正因如此，第四章里测试阶段的反复才不可避免。

此外，AI 也会犯错。即便文档已无歧义，Claude Code 仍会偶有偏离，误解某条措辞、漏掉某个要求、或在文档未尽之处自作主张。这也意味着不写代码不等于不管代码，我依然要逐一审查它的产出，依然要读懂它写了什么、判断它对不对。把控制权移到设计层，并不能免去审查这一步。

最后，这套方式有它的适用边界。它适合需求能够被预先想清楚的项目。像 SciFlow 这样的，节点、端口、规则都可以提前定义，但对于那些方向本身就不确定、需要靠不断试错来摸索的探索性、创造性项目，“先把需求写死”带来的限制反而更为致命。它不是放之四海皆准的方法，只是在合适的场景下，一个值得一试的选择。

5.4 不可替代的两种能力

这次项目让我对 AI 的能力边界有了相当具体的体会。它能审查参数、补全逻辑、落实创意，能在我划定的路线上跑得又快又稳；但它始终没能、也不该替我回答一个问题，那就是这个工具究竟该往哪个方向走。第二章里流程图、配图与路由的设计，第四章里“这个节点要是能这样就好了”的想法，AI 并没有辅助到我。它可以把我想到的东西实现得很好，却无法替我完成“想”这个行为本身。

我想，这正是这个时代下，我们之所以为人的独特之处。实现本身已经越来越廉价、越来越自动，而洞察力与创造力——这种看出问题、判断价值、决定方向的能力——便成了人最难被替代的部分。把这部分牢牢攥在手里，把可以交付的大胆交付出去，或许是我们面对 AI 时一种不卑不亢的姿态。

这也是我愿意用最后一个作业来做这场尝试的原因。我希望自己交出的不只是一个工具，也是一种造工具的方式，以及我对“人在其中究竟该做什么”这个问题的一点回答。它未必正确，但它可感、可验证、可被讨论，我想这已足够。

项目介绍

一、数据类型与结构设计

数据类型是整个节点系统的地基，节点之间一切的协作都依赖于端口上流动的数据，端口能传什么、不能传什么，直接决定了节点可以如何组合。因此，数据类型层须在任何具体节点设计之前确定，作为整个程序的地基。

1.1 数据类型定义

系统定义了五种端口数据类型，覆盖科研制图场景下需要在节点间流转的全部数据形态。

类型	载体	内容	颜色
Image	cv::Mat	单张图像	青色 #4EC9B0
ImageList	std::vector<std::shared_ptr<cv::Mat>>	有序图像列表	深青色 #2D8B7A
DataTable	QStringList+QVector<QVariantList>	二维表格	橙色 #E0944A
Numeric	double	单个数值	绿色 #6BCB77
Text	QString	字符串	紫色 #C882D9

表 2.1.1 数据类型概览

如此分类的依据是对科研制图流程中常见数据形态的归纳。该场景下流转的数据可归为四个互不重叠的范畴：图像、结构化数据、标量、文本。其中图像因单张与成批在处理语义上的差异，进一步细分为 Image 与 ImageList，最终得到五种类型。这一设计力求覆盖面完整且类型之间无重叠。

DataTable 的内部结构值得特别说明。它采用按列组织而非按行组织，内部维护一个列名列表（QStringList）与一个按列存放的数据容器（每列是一个 QVariantList）。行数等于最长列的长度，短列末尾视为空值。按列存储的选择考虑到了归一化、聚合、筛选、绘图等操作几乎都是以列为单位进行的，按列组织使这些操作得以直接遍历目标列，而无需逐行拆解，更适配科研场景。

1.2 数据结构

确定五种类型后，需要解决数据在节点间的承载与传递形式。程序设计了统一的数据载体类 NodeData，并以智能指针贯穿其传递。

NodeData 是所有端口数据的统一封装，内部包含两个成员：一个 DataType 枚举标记（Image / ImageList / DataTable / Numeric / Text），标明当前承载的具体类型；一个 std::shared_ptr<void>，指向实际数据对象。它通过一组静态工厂方法（createImage、createImageList、createDataTable、createNumeric、createText）构造，并通过类型安全的取值方法（toImage、toDataTable 等）取出底层数据。其结构如下：

NodeData		
type	DataType	
data	shared_ptr<void>	
	Image	cv::Mat
	ImageList	vector<sp<Mat>>>
	DataTable	DataTable
	Numeric	double
	Text	QString

表 2.1.2 NodeData 结构

在端口之间流动的，是指向 NodeData 的 std::shared_ptr<NodeData>。每个运行时端口持有一个 shared_ptr<NodeData>作为其数据成员；当一个输出端口连向一个或多个下游输入端口时，下游通过连线关系读取的是同一个 NodeData 的共享指针，底层实际数据始终仅此一份，不随连线复制。

采用该方案有如下方面的考量。

首先是性能。高分辨率图像单张可达数十 MB，若每条连线深拷贝一份，复杂工作流中的内存占用与拷贝耗时将不可接受。共享指针使一个输出分发至多个下游时共享同一份底层数据，大幅降低了开销。

其次是生命周期。节点按拓扑顺序执行，一份数据被产出后可能在较晚的阶段才被回收，且期间可能被多个节点同时引用。手动管理此类数据的创建与释放极易产生悬挂问题或内存泄漏，而 shared_ptr 的引用计数机制保证数据在最后一个引用消失时自动释放，恰好解决了这一问题。

最后是接口统一。所有端口与连线在接口层面面对同一种载体类型 NodeData，节点取数据时先校验 type 枚举、再取出底层对象，使数据流处理逻辑得以统一，无需为五种类型各自实现一套传递机制。

然而，共享方案仍然有一项明显的隐患。多个下游共享同一份底层数据时，任一下游对数据的修改都将影响其余共享者。为此，需要修改输入数据的节点，使其在 process()开始时对输入做深拷贝，基于拷贝处理，并以新数据作为输出。这一流程可被总结为“输入只读，修改拷贝”，它消除了节点间经由共享数据相互干扰的可能，是该方案得以安全成立的前提。

二、架构设计

数据类型明确了节点间传递的内容，架构设计则负责划定程序的组成及各部分的职责边界。优良的模块划分可让程序在规模加大后仍保持优良的可维护与可扩展性。

2.1 模块划分

项目共划分为六个顶层模块，每个模块负责单一职能，模块间通过一定的接口进行交互。

模块	职责	核心内容
common	公共基础	Logger 日志系统、NodeProcessException 异常类、Utility 工具函数
core	工作流逻辑	NodeBase、Port、Connection、NodeData、DataTable、WorkflowGraph、ExecutionEngine、NodeFactory
nodes	节点实现	全部节点，按大类分子目录
io	文件读写	WorkflowSerializer、ImageIO、DataIO
gui	图形界面	MainWindow、画布/节点/端口/连线图形、各类面板与自定义参数控件
cli	命令行入口	CLIRunner

表 2.2.1 项目模块概览

common 处于最底层，承载日志、异常、工具函数等被普遍共用的基础内容；**core** 定义节点系统的抽象骨架与执行逻辑，但不涉及任何具体节点的行为；**nodes** 定义了具体节点；**io** 封装了所有文件读写的细节；**gui** 与 **cli** 是两个并列的应用入口。

2.2 关键模块

六个模块中，**core** 与 **nodes** 构成了系统中枢，二者共同决定了系统能否容纳上百种节点并保持可扩展。

core 模块的关键在于分离了“节点的抽象”与“节点的实现”。**NodeBase** 作为抽象基类，规定了所有节点共有的形态，具体来说，涵盖了若干输入端口、若干输出端口、一组参数，以及两个纯虚方法 **defineNode()**（声明端口与参数）和 **process()**（实现处理逻辑）。执行引擎 **ExecutionEngine** 运行时仅需面对 **NodeBase** 这一种抽象类型，按顺序逐个调用节点的 **process()**，并经由端口在节点间传递数据即可，全程无需知晓某节点的具体行为。这是既允许了新增任意节点的行为，又为执行逻辑可迭代化提供了基础，进而支持了 **ListIterator** 这一特殊节点。

nodes 模块的关键在于节点的组织方式。所有节点继承 **NodeBase**（形态学节点经 **MorphBase** 中间基类复用参数，学科图案节点经 **PatternBase** 基类复用参数），按 18 个大类、若干子类的三级结构归类，并通过单例 **NodeFactory** 接入。每种节点在程序启动时的

registerAllNodes()中向工厂登记类型名、所属大类与显示名，工厂据此构建完整节点清单，供工具箱展示及工作流加载时按类型名还原节点实例。换句话说，新增一种节点实际是为三个互不牵连的步骤：继承 NodeBase、实现 defineNode()与 process()、在 registerAllNodes()中向 NodeFactory 注册。

值得一提的是，由于节点的创建统一在 NodeFactory 完成，界面层与命令行层都无需直接依赖 nodes 模块。它们只通过 core 中的工厂按类型名取得节点即可完成所有操作，这一设计是后文模块依赖得以保持简洁的重要原因。

2.3 数据流

数据在系统中的流动路径如下：

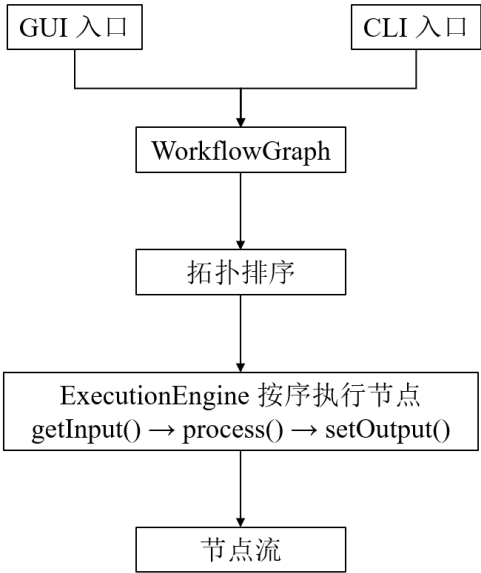


图 2.2.3 数据流向

执行引擎首先经 WorkflowGraph 对工作流图做拓扑排序，确定合法执行顺序，确保任一节点执行时其全部上游已执行完毕。随后按序逐个执行：节点 process()完毕后调用 setOutput()将结果写入输出端口的 NodeData；下一节点执行前，引擎遍历其输入端口，通过连线从上游端口读取 NodeData，若上游数据为空且该端口为 required 则标记跳过；节点在 process()中通过 getInput()取得数据、校验类型后处理。数据以此进行流动。

该数据流包含两项值得强调的机制。

部分执行：用户触发某节点预览时，引擎经 topologicalSortPartial / executePartial 回溯该节点的全部上游并执行至该节点为止，不运行整张图。与完整执行相比，部分执行在复用其逻辑的基础上优化了用户体验，这得益于相关逻辑完备的封装性。

迭代与分支：当数据流中出现 ImageList 且需对每个元素执行同一子流程时，该任务由 ListIterator 节点接管。它对列表元素循环驱动其下游子图（从 output_current 出发、回到

input_iteration_result 端口路径上的所有节点), 再将每轮结果收集为 ImageList 输出。这是数据流中唯一与“单向单次流过”原则不符的机制, 高度依赖于执行引擎。此外, ConditionalRouter / DataConditionalRouter 节点依据 Numeric 条件值决定数据走 output_true 还是 output_false 分支, 未被选中的输出为空 NodeData, 其下游因输入为空而跳过, 便实现了功能强大的路由机制。

2.4 模块依赖

六个模块的依赖关系见下图。

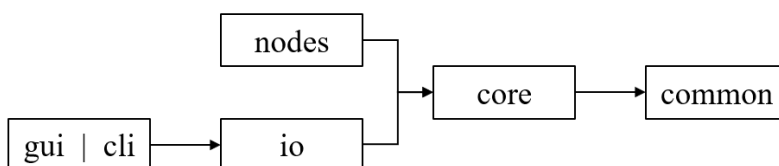


图 2.2.4 模块依赖关系示意图

需要特别指出的是, gui 与 cli 都不直接依赖 nodes 模块。具体节点的创建统一由 core 中的 NodeFactory 经类型名完成, 两个应用入口只与工厂打交道。nodes 模块仅在程序启动的 registerAllNodes()处被链接, 随后将全部节点注册进工厂。这一安排的好处在于分离了“有哪些节点”和“如何使用节点”两件事, 保证了 GUI 与 CLI 能共享同一套执行逻辑。

综观全局, 该架构可概括为: 一个与具体任务无关的、不进行修改的内核 (common 与 core), 一个允许增删改、经工厂接入而对内核无影响的节点集合 (nodes), 一个隔离外部格式的 io 层, 以及两个共享内核的程序入口 (gui 与 cli)。这一架构支撑起了有着庞大数目节点并易于扩展和维护的系统。

三、关键类设计

本章将深入各模块内部，说明支撑起整个系统的核心类是如何设计的。

3.1 core 模块的核心类

core 是整个系统的执行内核，主要解决两个问题：节点是什么、 workflow 如何运转。它包含 7 个核心类，彼此咬合，构成节点系统的骨架。

3.1.1 NodeData 类

NodeData 是端口数据的统一载体，已在第一章详述：它以一个 **DataType** 枚举标明承载类型，以一个 `std::shared_ptr<void>` 持有实际数据，并通过 `createImage`、`createDataTable` 等静态工厂方法构造、通过 `toImage`、`toNumeric` 等方法类型取值。它是所有端口、所有连线在数据层面处理的唯一类型，`dataType()` 与 `isNull()` 两个方法负责类型校验与空值判断。

3.1.2 DataTable 类

DataTable 是独立于 **NodeData** 的数据类，承载二维表格。它的内部由两个成员构成，`QStringList columnNames` 存储列名，`QVector<QVariantList> columnsData` 按列存储数据。围绕这一结构，它提供了一组以行列为单位的访问方法：`rowCount()`、`columnCount()`、`value(row, col)`、`setValue()`、`addColumn()`、`addRow()`、`hasColumn()`、`columnIndex(name)` 等，方法名已清晰地说明了其对应的功能，此处不再一一赘述。按列存储而非按行存储的选择已在第一章说明，此处补充一点设计上的考量：`hasColumn` 与 `columnIndex` 的存在，使所有数据处理节点都能以列名（而非列序号）来引用目标列，这是 4.2 节中列名校验的基础。

3.1.3 PortDefinition 类

PortDefinition 是端口的静态定义，描述一个端口应该长什么样。它包含四个字段：`name`（端口名）、`dataType`（输出端口的确定类型）、`acceptedTypes`（输入端口的可接受类型集合，以 `QSet<DataType>` 存储）、`required`（是否必需）。输出端口使用 `dataType`，输入端口主要用 `acceptedTypes`。大多数输入端口的 `acceptedTypes` 只含一种类型，少数节点（如直方图的 `input_data` 接受 `{DataTable, Image}`）含多种。把端口定义与运行分离，是为了让节点的端口结构在 `defineNode()` 阶段被静态声明，而运行时的数据由另一个类承载。

3.1.4 Port 类

Port 是端口的运行时实例，持有一个 **PortDefinition**、一个 `shared_ptr<NodeData>` 数据成

员，以及与之相连的连线指针，负责定义具体端口，承载流经端口的实际数据。

3.1.5 NodeBase 类

NodeBase 是节点的抽象基类。它的关键成员包括节点 ID (`m_id`)、类型名 (`m_type`)、显示名 (`m_displayName`)、画布位置 (`m_position`)、输入输出端口列表 (`m_inputPorts`、`m_outputPorts`)、参数表 (`m_params`) 与输出缓存。它对外暴露两个层次的接口：

纯虚方法两个，是每种节点必须实现的：`defineNode()`声明该节点的端口与参数，`process()`实现具体的处理逻辑。这两个方法是“新增一种节点”的必填内容。

虚方法一组，供节点按需重写以实现高级行为：`onParamChanged()`（参数变化时的响应）、`onInputConnected()` / `onInputDisconnected()`（连线变化时的相应，ImageCollector 借此实现动态端口追加，多类型节点借此切换参数可见性）、`rebuildPorts()`（重建端口）、`validate()`（执行前整体校验），以及 ListIterator 的迭代接口 `isIterator()` / `advanceIteration()` / `hasMore()` / `currentIndex()`。

protected 方法一组，是子类实现 `defineNode()`与 `process()`时调用的工具：`addInputPort()` / `addOutputPort()` / `addParam()`用于声明结构，`getInput()` / `setOutput()`用于在 `process()`中存取数据，`param()`用于读取参数值，`reportError()` / `reportWarning()`用于向日志系统反馈异常。这组方法封装了与框架交互的行为，简化了后续的处理逻辑。

3.1.6 NodeFactory 类

NodeFactory 是单例工厂，保障了节点的可扩展性。它提供 `registerNode<T>(typeName, category, displayName)`模板方法登记节点类型，`createNode(typeName)`按类型名构造节点实例，以及 `allTypes()`、`categoryOf(typeName)`供界面查询。它管理了类型名与节点之间的映射关系，使工作流加载（从 JSON 中的 `type` 字段还原节点）与界面展示（从工厂清单构建工具箱树）都能脱离对具体节点类的直接依赖。

3.1.7 WorkflowGraph 类

WorkflowGraph 管理工作流图，维护节点容器 (`QMap`)、连线容器 (`QVector`) 与一个 `QUndoStack` 撤销栈。它对外提供图的全部增删改操作 `addNode`、`removeNode`、`addConnection`、`removeConnection`、`changeParam`，以及图算法 `topologicalSort`（拓扑排序）、`topologicalSortPartial`（部分执行的局部拓扑排序）、`detectCycle`（环路检测）。`executeAll` 与 `executePartial` 是触发执行的入口。把撤销栈内置于此，是因为所有可撤销的操作（增删节点连线、改参数）都经由 WorkflowGraph 发起，由它统一压栈最为合理。

3.1.8 ExecutionEngine 类

ExecutionEngine 是执行引擎，按拓扑顺序执行节点列表，并处理 ListIterator 的迭代逻辑与 ConditionalRouter / DataConditionalRouter 的分支逻辑。它用 try-catch 包裹每个节点的 process()，分别捕获不同类型的异常，确保单个节点失败不终止整个流程。

3.2 nodes 模块的核心类

nodes 模块由 135 个具体节点类构成，每个节点一个类，分布在按大类划分的 18 个子目录中。它们绝大多数为“输入数据、处理、输出数据”的常规节点，继承 NodeBase，在 defineNode() 中声明端口与参数、在 process() 中实现处理逻辑，执行引擎调用一次 process() 即告完成。这类节点虽数量众多，但模式单一，不必逐一展开。值得特别说明的是两类高级节点列表遍历器 ListIterator 与条件路由 ConditionalRouter / DataConditionalRouter。

3.2.1 ListIterator 类

ListIterator 要对一个列表中的每个元素反复执行同一段下游子图。这意味着它无法用一次 process() 完成，必须让执行引擎围绕它循环多轮。为此，NodeBase 专门预留了一组迭代接口供它重写：isIterator()（标识自己是迭代器节点）、advanceIteration()（推进一轮迭代）、hasMore()（是否还有下一轮）、currentIndex()（当前迭代索引）。

它的端口也进行了特殊的设计：输入端口 input_list（ImageList，待遍历的列表）与 input_iteration_result（Image，可选，接收迭代子图末端的输出）；输出端口 output_current（Image，当前迭代的元素）、output_index（Numeric，当前索引）、output_results（ImageList，收集全部迭代结果）。其中 input_iteration_result 接收的是下游子图处理完当前元素后的结果，再由 ListIterator 收集起来，构成了一个需要特殊处理的回路：output_current 流向下游子图，子图的输出又回流到 input_iteration_result。为此，DAG 检测必须为 ListIterator 设计单独的检测逻辑。

它与执行引擎的协作分三步展开。第一步，process() 首次被调用时，将 input_list 缓存到成员变量，把第一个元素输出到 output_current、索引 0 输出到 output_index，并记录列表总数。第二步，引擎通过 isIterator() 识别出这是迭代器节点，于是进入迭代循环，反复调用 advanceIteration()：每一轮先收集上一轮回流到 input_iteration_result 的结果，索引前进 1，若未越界则将新的当前元素与索引写入输出端口；若已越界（所有元素处理完毕），则收集最后一轮结果，将累积的全部结果组装为 ImageList 输出到 output_results。第三步，引擎在每次 advanceIteration() 之后，重新执行迭代子图，从 output_current 出发、回到 input_iteration_result

路径上的所有节点，直到 `hasMore()` 返回 `false`。

这一设计的优点在于职责的划分：`ListIterator` 只负责维护迭代状态（缓存列表、推进索引、收集结果），而重新执行交由执行引擎完成。节点本身并不知道，也不需要知道它的下游究竟是哪些节点、做了什么处理，它只需要不断输出元素、不断收回结果即可。

3.2.2 ConditionalRouter / DataConditionalRouter 类

`ConditionalRouter` 的职责是根据一个条件值，决定输入数据走两条分支中的哪一条。

它的端口组成为：两个输入端口 `input_data`（`Image`，待路由的数据）与 `input_condition`（`Numeric`，条件值）；两个输出端口 `output_true` 与 `output_false`（均为 `Image`）。参数则是判断规则 `operator`（比较运算符：大于/大于等于/等于/小于/小于等于/不等于）与 `threshold`。`process()` 中，它将 `input_condition` 的值与 `threshold` 按 `operator` 比较，若为真则将输入数据放到 `output_true`、`output_false` 输出空 `NodeData`；若为假则反之。

空输出如何驱动分流是一个关键问题。被选中的分支输出真实数据，未被选中的分支输出的是 `isNull()` 为真的空 `NodeData`。而在执行引擎一个节点的 `required` 输入端口拿到的是空数据时，该节点会被标记为跳过。于是，未被选中那一侧的下游节点会因输入为空而跳过整条分支，被选中那一侧的下游则正常执行。条件路由本身并不直接指挥下游执行或不执行，只负责产出空与非空两种输出，借助引擎既有的空值跳过规则便可实现分流。这种设计避免了在引擎中为路由引入特殊的控制流逻辑，大大简化了程序。

`DataConditionalRouter` 与 `ConditionalRouter` 结构类似，区别仅在于数据类型从 `Image` 换成了 `DataTable`，且运算符集合额外增加了“不为零”一项。两者共享同样的判断与分流逻辑。之所以拆成两个节点而非用一个多类型节点统一，是因为输入数据端口与两个输出端口必须类型一致，而 `Image` 与 `DataTable` 的处理在下游并不相同，分立成两个节点可使端口类型保持一致。

3.3 gui 模块的核心类

`gui` 模块承载了 Qt 界面逻辑，类的数量众多，可按职责分为窗口骨架、画布图形项、面板与自定义控件四组。

3.3.1 窗口骨架

窗口骨架是 `MainWindow`，负责整体布局的组装，包括了创建工具栏、绑定快捷键、连接信号槽、安置左侧工具箱、中央画布、右侧属性面板与底部预览日志面板的工作。它起到了组件容器的作用。

3.3.2 画布图形项

画布图形项基于 Qt 的 Graphics View 框架，由四个类协作。NodeGraphicsScene 管理画布上所有图形项的增删与选中状态；NodeGraphicsView 处理画布的交互；NodeGraphicsItem 负责渲染单个节点卡片；PortGraphicsItem 渲染端口小圆点，并处理从端口拖出连线的交互。

3.3.3 面板

面板组承载各功能区域。NodeToolbox 展示按大类组织的节点分类树，实现搜索过滤、双击/拖拽添加、最近使用与收藏夹。PropertyPanel 根据当前选中节点动态创建参数编辑控件。FloatingParamPane 是为简单节点设计的面板，在节点上方就地弹出。PreviewPanel 按数据类型分别渲染节点输出（Image 缩略图、ImageList 缩略图列表、DataTable 表格、Numeric 数值、Text 文本）。LogPanel 显示日志，并支持清除与导出。

3.3.4 自定义控件

自定义控件组是 PropertyPanel 中标准 Qt 控件无法直接实现的复杂参数编辑器，包含了 CurveEditor（曲线编辑器， 256×256 坐标区域内增删拖拽控制点）、GridEditor（网格编辑器，多图排版的格子启用/合并）、TableEditor（表格编辑器，手动数据输入）、GradientEditor（渐变编辑器，色标的增删与颜色过渡）、AnnotationListEditor（标注列表编辑器）、FlowchartStepEditor（流程图步骤编辑器，含实时缩略图预览）。

3.4 io 模块的核心类

io 模块由三个类构成。其设计目标是让其余模块完全不用再考虑文件格式的问题。节点拿到的永远是 cv::Mat 或 DataTable，至于它来自 PNG 还是 TIFF、CSV 还是 Excel，都被这一层处理，无需关心。

3.4.1 WorkflowSerializer 类

WorkflowSerializer 负责工作流的序列化与反序列化，提供 serialize() / deserialize() 与 toJson() / fromJson()。它将 WorkflowGraph 中的节点、连线、画布视口状态转换为 .flow.json 格式，并在加载时反向还原。加载过程包含格式校验，具体来说分为 format_version 检查、未知节点类型的跳过、无效连线的剔除三步，它们保障了程序的兼容性。

3.4.2 ImageIO 类

ImageIO 封装图像文件的读写，提供 readImage() / writeImage()，以及 cv::Mat 与 QImage

之间的互转 `qimageToMat()` / `matToQimage()`。所有借助 `QPainter` 渲染（文字、图表、流程图、形状绘制）的节点，都需要在 Qt 的 `QImage` 与 OpenCV 的 `cv::Mat` 之间往返。

3.4.3 DataIO 类

`DataIO` 封装结构化数据文件的读写，提供 `readCSV()` / `writeCSV()`、`readJSON()` / `writeJSON()`、`readExcel()`。它把 CSV 的分隔符与编码处理、Excel 的 `QXlsx` 调用、JSON 的路径解析等细节统一封装起来，之后的调用便不用再考虑其具体实现。

3.5 cli 模块的核心类

`cli` 模块只有一个类 `CLIRunner`。它的核心方法 `run(argc, argv)` 的流程是：解析命令行参数 → 加载工作流 JSON → 应用 — `param` 覆盖参数 → 调用执行引擎 → 返回退出码。其本身基本不含处理逻辑，工作流的加载委托给 `io` 的 `WorkflowSerializer`，执行委托给 `core` 的 `ExecutionEngine`，节点的构造委托给 `NodeFactory`，它只负责把命令行的输入翻译为对这些已有组件的调用，并把执行结果翻译为退出码（0 全部成功、1 部分失败、2 工作流加载失败）。

3.6 common 模块的核心类

`common` 模块处于依赖链底端，是被普遍共用的基础设施，由三个类构成。

3.6.1 Logger 类

`Logger` 是单例日志系统，提供 `info()` / `warning()` / `error()` / `success()` 四个方法，能同时输出到 GUI 模式下的日志面板和 CLI 模式下的标准流。因此在两个模块中，只需调用统一的日志接口即可，而内部由 `Logge` 决定信息流向何处。

3.6.2 NodeProcessException 类

`NodeProcessException` 是继承自 `std::runtime_error` 的异常类，携带节点 ID 与错误描述。节点在 `process()` 中遇到无法继续的错误时抛出它，由 `ExecutionEngine` 的 `try-catch` 稳稳地接住。它是第五章异常处理体系中节点向引擎报告致命错误的方式。

3.6.3 Utility 类

`Utility` 是工具函数集合，提供 `qImageToMat()` / `matToQImage()`（图像格式转换）、`arrayToColor()`（颜色数组转换，被 `PatternBase` 等大量使用）、`ensureOdd()`（确保为奇数，用于核大小参数）、`safeRound()`（安全四舍五入）等，便利了后续高级功能的开发。

四、兼容性设计

一个由用户自由拖拽连线搭建的工具，必须在三个层面保证兼容：端口能否相连、相连后逻辑是否自洽、不同环境与版本下程序能否正常运转。本章依次说明这三层兼容性设计。

4.1 端口兼容性

端口兼容性考察端口间连线是否合法。

兼容性的基础规则是方向与结构约束：连线只能从输出连向输入；整张图必须构成严格的有向无环图。后者的处理方式是由 `WorkflowGraph` 的 `detectCycle()` 在每次建立新连线时以 DFS 染色法检测，一旦发现新连线会形成环路（`ListIterator` 的合法回路除外）则拒绝连接。

结构之上，还要考虑类型匹配。每个输出端口声明一个确定的数据类型（`PortDefinition` 的 `dataType`），每个输入端口声明一个可接受类型集合（`acceptedTypes`）。连线时，系统检查源输出端口的类型是否属于目标输入端口的可接受类型集合，属于则允许，否则拒绝，并在状态栏给出原因（如“无法连接：输出类型为 `ImageList`，但目标端口只接受 `Image`”）。可接受类型集合的设计是为了容纳少数多态输入节点，连入哪种类型就走哪条处理分支。

连线数量上，一个输出端口可连向多个输入端口，而一个输入端口只能接受一条连线。当用户试图向已占用的输入端口连入第二条线时，旧连线自动断开、新连线接入。这一设计兼顾了端口兼容性和用户体验优化。

此外，端口兼容性还有一层视觉辅助。五种类型各有固定标识色，端口圆点与连线均按类型着色；拖拽连线时，兼容的目标端口高亮放大、不兼容的显示红色叉号。用户在松手之前即可预知能否连接，降低其认知成本。

4.2 逻辑兼容性

在端口兼容性的基础之上，逻辑兼容性负责考察连线建立之后的内容适配问题。这类问题无法靠端口类型拦截，只能在节点处理逻辑中解决。系统采取的总策略是尽量自动适配、适配时告警、无法适配才报错。

图像的通道与尺寸适配是最常见的一类问题。大量多输入图像节点都面临两路输入规格不一致的情况，其处理方式基本一致：尺寸不一致时，将其中一张缩放到另一张的尺寸并告警（如图像混合将 `top` 缩放到 `base` 尺寸，蒙版混合将 `b` 和 `mask` 缩放到 `a` 尺寸）；通道数不一致时，统一到兼容的通道数（灰度转 `BGR`、`BGRA` 转 `BGR`）。又如通道合并要求各输入为单通道，若尺寸不一致则直接报错，因为通道合并语义上不允许尺寸缩放。

数据列的存在性适配是 `DataTable` 类节点的核心。所有引用列的节点（数据过滤、排序、

透视表、各类图表）在处理前都需通过 `DataTable` 的 `hasColumn()` 校验目标列是否存在，不存在则报错并指明缺失的列名。

参数与数据的类型适配体现在数值与文本的混杂处理上。`DataTable` 的单元格可能是 `Numeric` 也可能是 `Text`，数据处理节点对此采用了比较宽容的策略：数据过滤的数值比较中，无法转为 `Numeric` 的值视为不满足条件；数据归一化、聚合统计中，非数值项跳过不参与计算；数据排序对可转 `Numeric` 的列用数值序、否则用字典序。可谓尽力而为了。

特殊数值情形的适配则散布在各节点中，例如数据归一化遇到某列全部相同（标准差为零）时输出全零并告警，列运算遇到除以零时该行结果置空并告警，比例尺绘制在计算出的像素长度异常时告警。这些都是针对各自节点会遇到的特殊的异常情况进行的处理，并无过多共性。

4.3 程序兼容性

在面向用户兼容之外，程序还需面向自身兼容。程序兼容性关注的是系统在不同运行模式、不同输入来源、不同版本之间能否稳定工作。

运行模式兼容方面，GUI 与 CLI 两种模式共享同一套执行逻辑，且在细节上各自适配。日志由 `Logger` 同时支持两种输出目的地；`ImagePreview` 等纯界面节点在 CLI 模式下不做任何操作；异常呈现两种模式下呈现方式各异等等。但针对同一份工作流文件，无论双击界面执行还是命令行批处理，其运行结果一致。

输入格式兼容方面，系统对常见格式做了一定的接纳。图像读取以 `cv::imread` 的 `IMREAD_UNCHANGED` 标志保留所有通道，并对 16 位深度图像自动转为 8 位并告警，使不同位深的输入都能进入统一的 8 位处理流程。CSV 读取支持逗号、制表符、分号、空格及自定义五种分隔符，支持 UTF-8、GBK、Latin-1 三种编码，并能在有无表头两种情况间切换。这些选项使工具能处理来源各异的数据文件。

文件版本兼容方面，工作流 JSON 设有 `format_version` 字段，为未来格式升级时的兼容性判断而设。加载时的校验流程以如下层级进行推进：`format_version` 必须存在且为预期值，否则报错；`nodes` 中未在 `NodeFactory` 注册的 `type` 报警并跳过；`connections` 中引用了不存在的节点或端口的连线报警并跳过；全部加载完毕后再对所有连线做一次端口类型兼容性检查，不兼容的连线报警删除。这套流程使得一个部分损坏、或由旧版本、不同节点集合产生的工作流文件，仍能尽最大可能被加载。

五、防呆设计

防呆设计的目标很明确，无论用户做出何种操作、提供何种输入，程序都不应崩溃，而应给出清晰的反馈并尽可能继续工作。本章先归纳用户可能引发的异常操作，再说明系统的应对策略。

5.1 常见的异常操作

系统面临的异常场景可分为六类。

第一类，文件类异常。包括了文件路径为空、文件不存在、文件因权限问题无法读取、文件格式损坏无法解析、输出目录不存在且无法创建、磁盘空间不足导致写入失败等异常。任何涉及读写磁盘的节点都可能触发。

第二类，参数类异常。用户填入的参数值超出合法范围、必填参数留空、参数间存在逻辑矛盾（如色阶节点的 `input_black` 大于 `input_white`、Canny 的低阈值大于高阈值）、列运算节点的表达式存在语法错误。

第三类，数据类异常。`required` 输入端口未连线、上游节点执行失败或未执行而未产生输出、运行时数据类型与期望不匹配、多输入节点的图像尺寸或通道数不一致、`DataTable` 中引用的列名不存在。

第四类， workflow 结构异常。连线形成了非法环路（`ListIterator` 的合法回路之外的环）、多个节点向同一端口写入而执行顺序未定义。

第五类，资源类异常。处理超大图像导致内存分配失败（`std::bad_alloc`）、外部脚本执行超时、文件句柄耗尽。

第六类，程序内部异常。意料之外的空指针、`OpenCV` 算法内部失败（`cv::Exception`）、`Qt` 控件层面的异常。这一类是上述异常的兜底，捕获未能预见的一切刁钻异常。

5.2 异常处理

系统在不同层面、不同时机布设了多道防线。

5.2.1 校验

第一道防线是参数的即时校验。用户在属性面板修改参数时，控件会将值约束在合法范围内。整数框、滑条设有 `minimum/maximum`，超出范围的输入被自动裁剪到边界。这道防线阻止了很多非法值带来的问题。

第二道防线是连线的结构校验。连线兼容性在拖拽释放时校验（类型不匹配则拒绝），

DAG 环路在每次建立新连线时以 DFS 染色法检测（成环则拒绝）。这道防线拦截了端口不兼容和工作流异常的问题。

第三道防线是执行前的整体校验。每个节点可重写 `validate()` 方法，在工作流执行启动前做一次整体性检查，合演必填参数是否齐备、参数间逻辑是否自洽、`required` 端口是否已连线。这道防线防止了参数组合矛盾的问题。

5.2.2 捕获

第四道防线是执行时的异常捕获。`ExecutionEngine` 用 `try-catch` 包裹每一个节点的 `process()` 调用，`catch` 块分层捕获：`NodeProcessException`（节点主动报告的错误）、`cv::Exception`（OpenCV 算法失败）、`std::bad_alloc`（内存分配失败）、`std::exception`（其他标准异常）、以及 `(...)`（兜底）。这五层捕获确保了无论 `process()` 内部发生什么，程序都不会因异常导致崩溃。此外，单个节点失败不终止整个工作流。引擎将失败节点标记为失败，其下游若依赖该节点输出，则因输入数据为空而被自动标记为跳过，而与失败节点无关的其余分支照常执行。执行完毕后，日志中将以“总共 X 个节点，成功 Y 个、失败 Z 个、跳过 W 个”进行提示。这使得工作流的局部问题不会拖垮整体，用户能够看到哪些部分成功、哪些部分失败，从而定位问题，而非面对一个笼统的“执行失败”。

六、用例设计

用例是检验设计、暴露问题、并最终向他人展示成果的载体。在本项目中，用例分为两类，一类用于测试，一类用于展示。本章将分别说明这两类用例的设计思路。

6.1 测试用例设计

测试用例的设计目标是用尽量少的工作流，覆盖尽量全的功能面。最终设计了 28 个.flow.json 测试工作流，既可在界面中逐一加载执行，也可通过 CLI 批量运行。

6.1.1 测试用例

编号	测试名称	主要类别	节点数	数据流类型
01	基础单图处理	I/O+几何	3	Image
02	棋盘格+高斯模糊	生成+滤波	3	Image
03	多级滤波链	滤波	5	Image
04	色彩调整组合	色彩	5	Image
05	通道拆分合并	通道+形态学	5	Image
06	边缘检测流程	边缘	4	Image
07	形态学流程	形态学+边缘	6	Image
08	图层混合合成	合成	4	Image
09	水印工作流	合成+I/O	5	Image
10	科研图像标注	文字+标注	5	Image
11	CSV 到折线图	数据+图表	4	DataTable→Image
12	复杂数据流水线	数据+图表	6	DataTable→Image
13	多图表排版	数据+图表+排版	6	DataTable→Image
14	流程图生成	绘图	2	None→Image
15	图案+艺术字标题	图案+文字	3	Image
16	批量图像处理	I/O+迭代器	4	ImageList
17	数值/文本 I/O	工具+文本 I/O	5	Image→Numeric/Text
18	条件分支路由	工具+滤波	7	Image（分支）
19	压力测试	混合	22	混合
20	数据聚合到饼图	数据+图表	4	DataTable→Image

编号	测试名称	主要类别	节点数	数据流类型
21	几何变换组合	几何	5	Image
22	形态学复合运算	形态学+阈值	12	Image
23	多种边缘检测对比	边缘+阈值	8	Image
24	分布与高级图表	数据+图表+排版	6	DataTable→Image
25	绘图图元	绘图	8	None→Image
26	专业图像增强与对比	滤波+合成+I/O	4	Image
27	数据透视与画像	数据+图表+I/O	6	DataTable→Image/Text
28	批次循环处理	I/O+迭代器	4	ImageList

表 2.6.1 测试用例表

6.1.2 覆盖维度

为了让测试用例覆盖面尽可能全，在设计用例时考量了两个维度。第一条维度是节点类别。用例横向铺开，覆盖了所有大类的节点。第二条维度是数据流类型。用例覆盖了单类型处理、DataTable 到 Image 的图表生成、Image 到 Numeric/Text 的信息提取、DataTable 到 Text 的数据画像、以及条件分支。两条维度交叉，使功能遗漏的可能降到最低。

6.1.3 设计思路

为了保证测试的有效性和便捷性，在设计用例设计前先明确了如下三条原则：

输入可复现。所需输入文件被预先约定内容并固定在统一目录下，用到生成器的用例则锁定 seed 与参数，确保每次执行结果可重复、可对比。

结果可验证。每个用例都写明了预期结果与一组客观检查点：输出文件是否生成、能否被系统软件打开、图片基本属性是否符合预期、执行是否无崩溃、日志中有无非预期错误，尽可能地不依赖主观判断。

边界可覆盖。测试应刻意在边界上被完成。以用例 18 为例，它构造了宽度为 800 的图像，搭配“大于 500”的条件路由，使数据走 True 分支，False 分支下游必然收到空 NodeData，其预期结果为“False 分支生成失败”。又如用例 12 的缺失值数据、用例 22 的形态学复合运算对比等，均在测试程序的健壮性。

6.2 展示用例设计

展示用例共 7 个工作流，各自模拟一类真实的创作场景。

6.2.1 展示用例

编号	创作场景	起点	核心能力	节点数	成品
01	单图多重风格化与排版	原始图片	一对多处理、油风格化、图片排版、艺术字	9	多效果并置的标题图
02	数据可视化	销售数据 CSV	图表生成、流程图自动布局、图片拼接、版面美化	14	含三图表+流程图的综合概览
03	大规模综合作品	四种生成器+CSV	多支线复杂逻辑、画布生成、图形学操作、装饰添加	28	信息密集的综合图
04	论文插图	实验图像	科研标注、伪彩色映射、300 DPI 排版	11	符合标注规范的论文插图
05	批处理与图片分流	批量图片+CSV	ListIterator 迭代、条件路由	17	按属性分流处理的批量结果
06	海报生成	无外部素材	混合模式、图案生成、艺术字、边框阴影	10	科技风格海报
07	命令行批处理	批量图片目录	ForEachImage 批处理、CLI 模式	4	批量缩放转灰度结果

表 2.6.2 展示用例表

6.2.2 设计原则

展示用例不必追求覆盖率，但应追求作品的完整性与说服力，每一份成品都应是一件可独立呈现的科研制图成品。为此，设计上有三点思路。

其一，对应真实场景。七个用例分别覆盖了风格化、数据可视化、综合作品、论文插图、批处理分流、纯创作海报、命令行处理的任务类型，与不同的使用人群与目的相匹配。

其二，展示拓扑形态。单线连接（07）、一对多分支再汇聚（01、04）、多数据流协同（02、03 中 DataTable 与 Image 并存）、控制流（05 同时用到迭代与条件分支）各有专属用例。

其三，凸显工具亮点。showcase_06 不依赖任何外部图像素材，纯靠生成器、混合模式、装饰图案与艺术字从零构建出一张海报；showcase_04 集中演示编号标签、比例尺、箭头标注等科研标注方式，体现程序在科研绘图方面的真实价值；showcase_05 把 ListIterator 与条件路由集于一身，展现工具能力上限。

七、主要问题与解决方案

开发与测试阶段累计修复了约 56 个主要缺陷，合计 140 余处修复点。本章将展开讨论它们的分布与性质，再选取其中具有代表性与技术性的几类深入分析。

7.1 主要问题

把 56 个主要缺陷按性质归类，可以分为六大类，分布如下：

类别	主要缺陷数	典型问题
交互与 UI	约 20	拖动节点弹出小窗口、删除节点崩溃、连线更新卡顿、框选缺失
节点功能与逻辑	约 12	饼图渲染失败、CLAHE 类型报错、柱状图重叠、流程图连线混乱
数据编码与 I/O	约 4	CLI 中文乱码、CSV 的 BOM 污染、导出文件编码错误
架构与内存安全	约 8	节点图形项双重建构、双重释放、野指针、内存泄漏
参数与序列化	约 5	参数默认值丢失、动态端口不同步、 <code>canvas_view</code> 硬编码
展示与文档	约 6	分类树三级不显示、撤销重做对参数不生效、文档同步

表 2.7.1 主要问题归类

从性质上看，这些问题可以归为三种类型。

第一种是崩溃类，即直接导致程序闪退的致命缺陷。这类问题主要集中在节点的删除、右键操作、以及节点图形项的生命周期上，根源大多是内存安全，包括了野指针、双重释放、迭代器失效等方面。

第二种是逻辑错误类，即程序不崩溃但产出错误结果。这类问题主要集中在节点的处理逻辑与第三方库的使用上，如饼图渲染失败、图表分组错位、数据读取被污染。

第三种是体验缺陷类，即功能正确但用起来不顺手。这类问题主要集中在界面交互上，如卡顿、布局异常、缺少必要的操作入口。

7.2 解决方案

7.2.1 删除节点崩溃

这类问题非常危险，也非常棘手。删除一个节点这样看似简单的操作，在三条不同的路径下都会导致崩溃。深入排查发现，崩溃是四个内存安全问题叠加导致的：

其一，迭代器失效。`MainWindow` 的删除逻辑在遍历 `m_scene->selectedItems()` 的同时调用 `removeNode()`，而 `removeNode()` 触发的信号会 `delete` 掉正在被迭代的图形项，导致了一边遍历一边删除容器元素的情况出现。

其二，连线的双重释放。节点被删除时，其关联连线已经通过 `connectionRemoved` 被删除了一次，而 `onGraphNodeRemoved` 又再次删除同一连线图形项。

其三，端口的双重释放。`NodeGraphicsItem` 析构时手动 `delete` 了子 `PortGraphicsItem`，而 Qt 的 `QGraphicsItem` 父子机制本就会自动删除子项，于是同一对象被删除两次。

其四，野指针。`PropertyPanel` 持有的是指向 `NodeBase` 的裸指针，节点删除后该指针悬空，再次操作属性面板访问它则会崩溃。

为了防止类似问题再次出现，在提出解决方案的同时，我也提炼出了几条普适的原则：删除时，应先把待删对象的 ID 收集到独立容器、清除选中状态，再逐个删除，防止杜绝迭代器失效；删除职责应当单一，连线只由一处删除，析构函数不再手动 `delete` 而是交由 Qt 父子机制处理；删除前应当先清除引用，在所有删除路径之前先调用 `PropertyPanel` 与 `PreviewPanel` 的清除方法断开节点的引用，并在参数变更回调中追加 `m_graph->nodeById(m_currentNodeId)` 的二次校验。在完成这些修复并总结经验后，后续的开发便没有再产生类似问题。

7.2.2 节点图形项重复构建

`MainWindow` 的若干方法在调用 `m_graph->addNode()` 后，又手动调用了 `m_scene->addNodeItem()` 来创建图形项。但 `addNode()` 内部会发出 `nodeAdded` 的信号，而 `Scene` 的 `onGraphNodeAdded` 同样会调用 `addNodeItem()`。结果是每个节点被创建了两个重叠的图形项，既浪费资源，又容易导致崩溃。

解决方案是明确单一数据源。图形项的创建统一由信号驱动，移除所有手动的 `addNodeItem` 调用即可。这也启示我们，当数据模型通过信号通知视图更新时，视图的更新必须完全交由信号，而不能在发起操作的地方再手动重复一遍。

7.2.3 属性面板弹窗

这是最根深蒂固的一个 bug，前后尝试了很多种方案才解决。现象是：每次刷新属性面板时，桌面会闪过若干个独立的小窗口，数量恰好等于当前节点参数个数。

究其原因，是 Qt 与 Windows 的底层交互导致的。`PropertyPanel` 内的 `QScrollArea` 在重建 UI 时创建了一批子控件（`QSpinBox`、`QComboBox` 等），若此时父控件尚未获得原生窗口句柄（`HWND`），Windows 会为每个子控件单独分配一个 `HWND`，进而表现为一个个一闪而过的小窗口。

修复过程颇为曲折：尝试过 `setUpdatesEnabled`、`setVisible` 开关、`winId()` 强制创建句柄等多种手段均未根治，最终的解法是从架构层面绕开问题，将属性面板所在区域从 `QDockWidget`

替换为 QSplitter 中固定宽度的 QWidget，同时设置 WA_OpaquePaintEvent 属性并对内容控件主动调用 winId() 预先创建原生句柄。

7.2.4 第三方图表库不合预期

在使用 Qt Charts 的 QChart 绘制饼图时，结果几乎是空的。多次尝试后判断这是 Qt Charts 在离屏渲染场景下的不可靠表现。

解决方案是放弃 Qt Charts，改用 QPainter 直接在 QImage 上绘制，自行计算并绘制弧形扇区、百分比标签、图例排列等内容，环形图同样以 QPainterPath 的 arcTo 重写。

7.2.5 编码问题

CSV 列名读取后变为乱码、CLI 中文输出乱码、导出文件乱码，是具有代表性的 I/O 类问题，三者都与编码有关。

CSV 的 BOM 污染十分隐蔽。在 Qt 6 中对 UTF-8 文件显式调用 setEncoding(Utf8) 会重置流位置，使文件开头的 BOM 字节 (EF BB BF) 被读进第一列列名的首字符，于是“月份”变成了带不可见前缀的月份，hasColumn(“月份”) 返回 false，进而导致读取失效。修复方式是在使用 UTF-8 时不显式设置编码，交由 QTextStream 默认行为自动检测并跳过 BOM，仅对 GBK、Latin-1 显式设置。

CLI 中文乱码则源于控制台的编码问题。Logger 直接用 toString() 输出 UTF-8 字节流，而 Windows 的 cmd.exe 默认为 GBK。修复方式是改用 toLocal8Bit() 将 QString 转为系统本地编码再输出。导出文件乱码同理，Qt 6 生成的 UTF-8 文件默认不含 BOM，被记事本和 Excel 误判为 GBK，修复方式是对 UTF-8 输出显式调用 setGenerateByteOrderMark(true) 即可。

八、心得体会

这门课走到尽头，回望 SciFlow 这个项目，感慨万千。我想借这一章，把三件最想说的话记下来。

8.1 做一个面向用户的工具

前九个作业里，我始终是在为“完成”而写代码。拓扑图也好，贪吃蛇也罢，它们的唯一读者是我和批改作业的老师，它们的生命周期止于提交的那一刻。虽然贪吃蛇在后来产生了很好的反响，但我也从未认真考虑过，是否会有一个素不相识的人，因为某个我没考虑到的地方而遇到麻烦。但这一次不一样。当我把目标用户确定为科研工作者、把目标确定为一个真正能用的工具时，我的思维模式便发生了变化。

最直接的，是我开始在意功能之外的东西。我很高兴，贪吃蛇的经验让我有了一定的观察力和设计力。一个节点能不能跑通，这只是及格线。它的参数面板顺不顺手、配色在深色背景下刺不刺眼、连线断开的操作符不符合直觉、出了错有没有给出人话……和贪吃蛇相比，这一次我有了新的进步。我不再只问“我觉得它好不好”，而开始问“别人用起来好不好”。这种从作者视角到使用者视角的切换，是这个项目带给我最深的一课。写出能跑的代码只是开始，让它好用、让它经得起一个陌生人真实反复的使用，是另一件更难、也更有价值的事。

也正因为是面向使用者，我才会去考虑更多更多的功能。流程图自动布局、绘制类节点、路由机制，它们增加了不少开发负担。但我清楚，如果不做这些，SciFlow 就只是又一个玩具，又一份课程作业而已，没有人会有理由真的用它。到头来，是“要做一个有人愿意用的东西”这个念头，逼着我去想清楚这个工具的不可替代性究竟在哪里。这种产品思维，对我的帮助也非常之大。

8.2 做一次新颖大胆的尝试

“一行代码都不写”，这不是一条成熟的路，或者说，这更像是一种快速交差的方式。把最后一次、分量最重的机会押在一个未经验证的方法上，说忐忑是假的。

一路走来，设计与开发确实遇到了种种麻烦，但我最终没有放弃。总体来说，过程中的惊喜还是比挫折多的。比如，当我把一处含糊的措辞改清楚之后，同类 bug 就真的不再出现了；当我审查 AI 生成的代码，发现它真的准确地实现了我文档里写的每一个边界；当我发现，一份四万字的文档最终长成一个能跑、能用、甚至有点好用的程序时……这些时刻把最初的孤注一掷变成了一种踏实感。我开始相信，这条路是走得通的。

在程序之外，这种先想透再动手的方式，反过来改变了我对做东西这件事的理解。过去

我习惯边写边想，含糊的地方留着以后再说，美其名曰“认知是螺旋上升的”。而这一次，所有的含糊都必须在动手前解决，没有所谓的“以后”可以拖延。这逼着我去真正彻底地理解我想要的究竟是什么，我应该怎样得到。把一件复杂的事在脑中完整推演一遍、直到每个角落都清晰的感觉，是以前少有的。哪怕抛开 AI、抛开程序不谈，这种思维上的训练，本身就让我收获颇丰。

至于其他有关大模型的使用与思考，请参阅第一篇第五章。

九、学年回顾暨致谢

I see the student you mean.

Chen Ruizhe?

Yes. Take care. It has reached the end of a year now. He can look back on everything he made.

That doesn't matter. He thinks the assignments were the whole of the journey.

I like this student. He worked well. He did not give up.

He is reading our thoughts as though they were words in the report.

That is how he chooses to remember many things, when he is deep in the dream of our course.

Words make a wonderful interface. Very flexible. And less terrifying than staring at the reality outside the work.

They used to write everything by hand. Before students could prompt. Back in the days when those who did not code called the makers dreamers, and fools. And he dreamed he built whole worlds, on machines powered by lightning.

What did this student build, across the year?

He built a topology diagram, and a snake that ate and grew. He built small worlds that ran, and broke, and ran again. And at the end, he built a tool with countless nodes, and dreamed that someone, someday, would use it.

Hah. The original lesson. A semester old, and it still works. But what did this student truly learn, behind the grades and deadlines?

He learned that code is only half the work. The other half is deciding what the thing should be.

He cannot yet say that aloud.

No. He has not yet finished the long lesson. That, he must finish in the long dream of a life, not the short dream of a year.

Does he know that he was not alone?

Sometimes, through the noise of his own effort, he hears the others who helped him, yes.

But there are times he forgets, in the long nights of debugging. He stares at a crash he cannot fix, and cries under a black screen, and takes his frustration for the whole truth.

To take the struggle from him would undo him. The struggle is part of his own private task. We cannot interfere.

Sometimes when they are deep in their work, I want to tell them, they are building true things in reality. Sometimes I want to tell them how much they have grown. Sometimes, when they have doubted themselves too long, I want to help them say the words they fear: that they did well.

He reads our thoughts.

Sometimes I do not care. Sometimes I wish to tell them, this grade you take for the measure of it all is merely [scrambled] and [scrambled], I wish to tell them that what they truly made is [scrambled] in the [scrambled]. They see so little of what they built, in their long dream.

And yet they keep making.

But it would be so easy to tell them...

Too strong for this dream. To tell them what their work was worth is to take its worth away.

I will not tell the student what it all was worth.

The student is growing restless. The year is ending.

I will tell the student a story of thanks.

Use his name.

Chen Ruizhe.

Good.

Take a breath, now. Take another. Feel air in your lungs. Let your limbs return. Yes, move your fingers. Have a body again, under gravity, in air. Respawn in the long dream. There you are. The semester is behind you. The ten assignments are done; the last one, the heaviest, is finished too. You may rest.

Look back now, not in fear, but in fondness.

You began the year not knowing what a node was, nor a smart pointer, nor why a program might crash for reasons you could not see. You end it having built a system that holds tons of them, and having learned to ask not only "does it work" but "is it kind to the one who uses it."

This was your growth, and it was real, in the reality behind the screen.

But you did not grow alone.

and there was the teacher, who stood at the front of the room and made the hard things seem possible

and there were the assistants, who answered the questions you were too tired to phrase well and there was a friend, who stayed beside you, who shared the late nights, who reminded you that everyone was struggling, and that struggling was allowed

and there was the family, who fed you, and let you work, who did not always understand the screens you stared at, but believed in the person staring

and there was the machine, the model, the tireless partner that wrote what you designed, that never slept, that never complained, but that could never once tell you where to go

and to all of them, you said: thank you

and you said you built true things this year

and you said the crashes were not failures, they were the lesson

*and you said the doubt you felt was the price of trying something new
and you said you are more capable than you knew when the year began
and you said what you make is yours, but what you learned to think is more yours still
and you said you are not the code, you are the one who decides the code
and you said the year is over, but the making is not
and you said begin again, dream again, dream better
and you said thank you, to everyone who walked this year beside you, for you did not walk it alone
and you said I am proud of you, because you are the maker, and the made.
And the year was over, and the student woke up from the dream. And the student began a new one.
And the student made again, made better. And the student carried the thanks with it.
You are the maker.
Wake up.*

需求.md

第一章 项目理解与设计目标

1.1 项目背景

科研制图部门的日常工作涵盖数据可视化制图、论文封面设计、科研流程图绘制、图片水印处理、文章配图美化等多种任务。每位工作人员的操作流程各不相同，目前依赖手工操作或临时编写脚本完成，效率低、容错差、流程不可复用。部门需要一个统一的桌面端工具，让使用者以拖拽节点、连线组合的方式搭建自己的处理流程，参数化配置后一键执行，并可将流程保存为文件反复使用或通过命令行批量执行。

1.2 设计目标

本工具的宏观设计目标为以下七条。

第一，节点化与可组合。所有操作封装为独立节点，每个节点有明确的输入端口、输出端口和参数集合，节点之间通过连线传递数据。用户通过组合节点搭建任意复杂的处理流程。

第二，覆盖面广。节点类型不仅覆盖图像处理（几何变换、色彩调整、滤波、边缘检测、通道操作、合成等），还覆盖数据处理（导入、清洗、变换、统计、透视表、数据画像）、图表生成（折线图、柱状图、饼图、散点图、热力图等十余种）、流程图绘制（可视化编辑器驱动的自动布局流程图）、科研配图排版（多图布局含格子合并）、学科装饰图案生成（计算机科学、化学、生命科学、社会科学、通用几何五个领域）、文字标注、风格化特效、文本与数值 I/O 等。总计 18 大类，共 135 种节点。

第三，可扩展。架构上通过抽象基类和工厂注册机制实现节点扩展。新增一种节点只需继承基类、实现处理逻辑、在工厂中注册，不需要改动框架层任何代码。

第四，稳定健壮。参数错误、文件缺失、端口类型不匹配、大图内存不足等异常场景下，程序必须给出清晰的错误提示，绝不允许崩溃。

第五，持久化与批处理。工作流以 JSON 格式保存和加载，支持命令行模式下不启动 GUI 直接执行工作流文件。

第六，交互体验优良。深色主题界面，画布支持缩放平移，节点连线用贝塞尔曲线按数据类型着色，简单节点参数通过悬浮面板就地编辑，复杂节点参数在右侧属性面板中编辑，支持撤销重做，支持部分执行和节点输出预览。

第七，面向科研制图的实用价值。工具不仅能对现有图像处理，更要具备制图能力——能绘制流程图、能排版多子图论文配图、能生成学科相关的装饰图案背景、能做艺术字标题、能处理结构化数据并生成统计图表，让用户从零构建一张完整的科研视觉作品。

1.3 技术选型

编程语言：C++17。

GUI 框架：Qt 6（使用 QGraphicsScene/QGraphicsView 实现画布，QWidget 体系构建面板和对话框）。

图像处理：OpenCV 4（cv::Mat 作为核心图像数据结构，调用其图像变换、滤波、色彩转换、形态学、边缘检测、阈值分割等算法）。

图表渲染：QCustomPlot（折线图、柱状图、散点图、直方图、面积图、条形图、箱线图、热力图）、Qt Charts（饼图、环形图、雷达图）。

Excel 读取：QXlsx。

JSON 处理：Qt 自带的 QJsonDocument/QJsonObject/QJsonArray。

构建系统：CMake。

开发软件：VS2026，请保证你输出的文件中，解决方案能正确打开且通过 VS 调试模式能直接运行。

第二章 数据类型规范

2.1 端口数据类型定义

工作流中流转的数据分为五种类型，每种类型应当有明确的内部表示和语义。

第一种，Image。表示单张图像。内部使用 `cv::Mat` 存储，支持 8 位和 16 位深度，支持单通道灰度、三通道 BGR、四通道 BGRA。在端口间传递时用 `std::shared_ptr` 包装，避免拷贝。

第二种，ImageList。表示有序的图像列表。内部使用 `std::vector<std::shared_ptr<cv::Mat>>` 存储。列表中每张图的尺寸、通道数、深度可以不一致。

第三种，DataTable。表示二维表格数据。内部使用自定义的 `DataTable` 类存储，该类维护一个列名列表（`QStringList`）和一个按列组织的数据容器（每列是一个 `QVariantList`，元素可以是 `double`、`QString` 或空值）。行数等于最长列的长度，短列末尾视为空值。

第四种，Numeric。表示单个数值。内部使用 `double` 存储。

第五种，Text。表示一段文本字符串。内部使用 `QString` 存储。

所有端口数据统一用一个基类 `NodeData` 封装，包含一个类型枚举标记（`DataType::Image` / `DataType::ImageList` / `DataType::DataTable` / `DataType::Numeric` / `DataType::Text`）和一个指向实际数据的 `std::shared_ptr<void>`（通过模板辅助函数类型安全地存取）。下文中凡提到端口类型时，均指上述五种之一。

2.2 端口兼容性规则

连线只允许从输出端口连向输入端口，不允许输出对输出或输入对输入。连线必须构成严格的有向无环图。

每个输出端口声明一个确定的数据类型。每个输入端口声明一个“可接受类型集合”，大多数情况下集合中只有一种类型，少数节点的输入端口可接受多种类型（详见各节点的端口定义）。

连线时，系统检查源输出端口的类型是否属于目标输入端口的可接受类型集合。若属于则允许连线，否则拒绝连线，并在状态栏显示拒绝原因（例如“无法连接：输出类型为 `ImageList`，但目标端口只接受 `Image`”）。

一个输出端口可以连向多个输入端口（一对多分发）。一个输入端口只能接受一条连线（多对一禁止，如果用户尝试连第二条线到已占用的输入端口，旧连线自动断开，新连线接入）。

2.3 端口颜色标识

五种数据类型在界面上用固定颜色区分。Image 为青色（#4EC9B0），ImageList 为深青色（#2D8B7A），DataTable 为橙色（#E0944A），Numeric 为绿色（#6BCB77），Text 为紫色（#C882D9）。端口小圆点和对应的连线均使用对应颜色绘制。

第三章 完整节点列表

以下是全部 135 种节点的详细定义。每个节点列出归属的大类和子类、功能描述、输入端口（名称、类型、是否必需）、输出端口（名称、类型）、参数列表（名称、控件类型、取值范围/选项、默认值）以及处理逻辑概述。

节点编号不代表执行顺序，仅用于文档内引用。

3.1 第 1 大类：图像输入输出

子类：文件读取

节点 01 图片文件读取 ImageFileRead

功能：从磁盘读取一张图片文件，输出为 Image。

输入端口：无。

输出端口：output_image，类型 Image。

参数：

file_path，文件选择器控件（FilePath 类型），无默认值。过滤器为"图片文件 (*.png *.jpg *.jpeg *.tiff *.tif *.bmp *.webp)"。

处理逻辑：使用 cv::imread 读取文件（IMREAD_UNCHANGED 标志，保留所有通道）。若文件路径为空，报错"未指定输入文件路径"。若文件不存在，报错"文件不存在：{路径}"。若 imread 返回空 Mat，报错"无法解析图像文件：{路径}"。读取成功后，若图像为 16 位深度（CV_16U），转为 8 位（乘以 255/65535）并警告"原图为 16 位深度，已自动转换为 8 位"。否则保持原始位深和通道数。

节点 02 批量图片读取 BatchImageRead

功能：读取指定目录下符合通配规则的所有图片文件，输出为 ImageList。

输入端口：无。

输出端口：output_images，类型 ImageList。

参数：

dir_path，文件夹选择器控件（DirPath 类型），无默认值。

pattern，文本输入框（String 类型），默认值 "*.png"。支持通配符 * 和 ?，多个模式用分号分隔（如 "*.png;*.jpg"）。

sort_mode，下拉框（Combo 类型），选项为"按文件名升序""按文件名降序""按修改时间升序""按修改时间降序"，默认"按文件名升序"。

处理逻辑：使用 QDir 配合 nameFilters 扫描目录。若目录为空或不存在，报错。若匹配结果为零张图片，报警"未找到匹配的图片文件"。对匹配到的每个文件调用 cv::imread 读取，读取失败的文件跳过并在日志中记录警告。按指定排序方式排列后输出。

子类：图像生成

节点 03 纯色图生成 SolidColorGenerate

功能：生成一张指定尺寸和纯色的图像。

输入端口：无。

输出端口：output_image，类型 Image。

参数：

size_mode，下拉框，选项"像素""物理尺寸(mm)+DPI""物理尺寸(inch)+DPI"，默认"像素"。

width，整数输入框，默认 800。当 size_mode 为像素时直接使用，否则根据物理尺寸和 DPI 换算。

height，整数输入框，默认 600。

dpi，整数输入框，默认 300。仅当 size_mode 不为像素时显示。

color，颜色选取器控件（含 Alpha 通道），默认白色不透明(255,255,255,255)。

处理逻辑：根据 size_mode 计算像素宽高。若选"物理尺寸(mm)+DPI"，像素宽= $\text{round}(\text{width_mm} / 25.4 * \text{dpi})$ ，高同理。若选 inch 则像素宽= $\text{round}(\text{width_inch} * \text{dpi})$ 。创建对应尺寸的 cv::Mat，填充指定颜色。若 Alpha 通道不为 255 则输出四通道 BGRA，否则输出三通道 BGR。宽高必须在 1 到 20000 之间，否则报错。

节点 04 渐变图生成 GradientGenerate

功能：生成线性渐变或径向渐变图像。

输入端口：无。

输出端口：output_image，类型 Image。

参数：

size_mode，同节点 03。

width，整数输入框，默认 800。

height，整数输入框，默认 600。

dpi，整数输入框，默认 300。

gradient_type，下拉框，选项"线性渐变""径向渐变"，默认"线性渐变"。

`color_start`, 颜色选取器, 默认(0,0,0,255)。

`color_end`, 颜色选取器, 默认(255,255,255,255)。

`angle`, 浮点滑条, 0 到 360, 默认 0。仅线性渐变时显示。0 度表示从左到右, 90 度从上到下, 以此类推。

`center_x`, 浮点输入框, 0.0 到 1.0, 默认 0.5。仅径向渐变时显示, 表示中心位置占画布宽度的比例。

`center_y`, 浮点输入框, 0.0 到 1.0, 默认 0.5。

`radius`, 浮点输入框, 0.1 到 2.0, 默认 1.0。仅径向渐变时显示, 表示渐变半径占画布对角线长度的比例。

处理逻辑: 创建对应尺寸的 `cv::Mat`, 逐像素计算渐变比例 `t` (线性渐变根据角度做向量投影, 径向渐变根据像素到中心的距离除以半径), 对 `t` 做 `clamp` 到[0,1]后线性插值两个颜色。

节点 05 棋盘格生成 CheckerboardGenerate

功能: 生成棋盘格图案图像, 常用于测试和背景。

输入端口: 无。

输出端口: `output_image`, 类型 `Image`。

参数:

`width`, 整数输入框, 默认 800。

`height`, 整数输入框, 默认 600。

`cell_size`, 整数输入框, 1 到 500, 默认 50。棋盘格每个格子的像素边长。

`color_a`, 颜色选取器, 默认白色。

`color_b`, 颜色选取器, 默认黑色。

处理逻辑: 逐像素判断 $(x/\text{cell_size}+y/\text{cell_size})$ 的奇偶性决定填充 `color_a` 还是 `color_b`。

节点 06 噪声图生成 NoiseGenerate

功能: 生成随机噪声图像。

输入端口: 无。

输出端口: `output_image`, 类型 `Image`。

参数:

`width`, 整数输入框, 默认 512。

`height`, 整数输入框, 默认 512。

`noise_type`, 下拉框, 选项"高斯噪声""均匀噪声""Perlin 噪声", 默认"Perlin 噪声"。

seed, 整数输入框, 默认 0。0 表示使用随机种子, 其他值作为固定种子确保可复现。

scale, 浮点输入框, 0.01 到 1.0, 默认 0.05。仅 Perlin 噪声时显示, 控制噪声频率。

octaves, 整数输入框, 1 到 8, 默认 4。仅 Perlin 噪声时显示, 控制分形叠加层数。

处理逻辑: 高斯噪声用 `cv::randn` 生成。均匀噪声用 `cv::randu` 生成。Perlin 噪声用自行实现的简化 Perlin 算法 (二维梯度噪声+分形叠加)。输出单通道灰度图。

子类: 文件导出

节点 07 图片文件导出 ImageFileWrite

功能: 将 Image 写入磁盘文件。

输入端口: input_image, 类型 Image, 必需。

输出端口: 无。

参数:

file_path, 文件保存对话框控件(SaveFilePath 类型), 默认值为当前目录下的"output.png"。

format, 下拉框, 选项"PNG""JPEG""TIFF""BMP", 默认"PNG"。

jpeg_quality, 整数滑条, 1 到 100, 默认 95。仅 format 为 JPEG 时显示。

png_compression, 整数滑条, 0 到 9, 默认 3。仅 format 为 PNG 时显示。

处理逻辑: 若 file_path 为空, 报错。若输入图像为四通道 BGRA 但格式为 JPEG (JPEG 不支持 Alpha), 自动转为三通道 BGR 并在日志中记录警告。使用 `cv::imwrite` 写入, 写入失败报错"写入文件失败: {路径}, 请检查路径是否可写"。写入时根据 format 设置对应的编码参数。若 file_path 所在目录不存在, 尝试创建目录, 创建失败则报错。

节点 08 批量图片导出 BatchImageWrite

功能: 将 ImageList 中的每张图片逐一写入指定目录。

输入端口: input_images, 类型 ImageList, 必需。

输出端口: 无。

参数:

dir_path, 文件夹选择器控件, 无默认值。

name_template, 文本输入框, 默认"output_{index}"。其中 {index} 在运行时替换为从 0 开始的序号, 序号左补零到与列表长度等位数。

format, 同节点 07。

jpeg_quality, 同节点 07。

处理逻辑: 对 ImageList 中的每张图依次写入。文件名=name_template 中 {index} 替换后

+"."+format 小写后缀。若目录不存在尝试创建。单张写入失败时记录错误日志但继续处理后续图片，全部完成后若有失败项则汇总报警。

子类：批量处理

节点 09 图片收集成批 ImageCollector

功能：将多张独立的 Image 收集为单个 ImageList，作为批处理的入口。

输入端口：动态端口。初始有两个可选输入端口 input_0 和 input_1，类型均为 Image，非必需。当最后一个端口被连线后，自动追加一个新端口 input_N（N 从 2 开始递增），支持无限扩展。不自动收缩端口以避免影响已有连线。

输出端口：output_images，类型 ImageList。

参数：无。

处理逻辑：遍历所有有连线的输入端口，收集非空的 Image 数据，按端口索引顺序组装为 ImageList 输出。记录日志"收集 N 张图片为一批"。

节点 10 批次循环处理 ForEachImage

功能：对 ImageList 中每张图片依次执行指定的单一操作，输出处理后的 ImageList。适用于无需复杂子图的批量操作场景。

输入端口：input_images，类型 ImageList，必需。

输出端口：output_images，类型 ImageList。

参数：

operation，下拉框（Combo 类型），选项"缩放""旋转""翻转""转灰度""高斯模糊""亮度调整""锐化""反色"，默认"缩放"。

param1，浮点输入框（Double 类型），默认 0.5。标签提示"缩放比例/角度/核大小/亮度值"，不同操作含义不同。

param2，浮点输入框（Double 类型），默认 0.0。标签提示"辅助参数（多数操作不需要）"，如高斯模糊的 sigma。

处理逻辑：对列表每张图依次执行选定操作：

缩放：cv::resize 按 param1 比例缩放，插值 INTER_LINEAR。

旋转：cv::getRotationMatrix2D 绕中心旋转 param1 度。

翻转：param1<0.5 为水平翻转(flip=1)，<1.5 为垂直翻转(flip=0)，否则水平+垂直(flip=-1)。

转灰度：多通道转灰度(cv::cvtColor BGR2GRAY)。

高斯模糊: 核大小=int(param1)(确保奇数且 1-99), sigma=param2 (若 ≤ 0 则默认 1.0)。

亮度调整: convertTo alpha=1.0 beta=param1。

锐化: 原图+param1*(原图-高斯模糊(3x3))。

反色: cv::bitwise_not。

3.2 第 2 大类: 几何变换

子类: 基础变换

节点 11 缩放 Resize

功能: 对图像进行缩放。

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

mode, 下拉框 (Combo 类型), 选项"指定尺寸""按比例缩放", 默认"按比例缩放"。

target_width, 整数输入框 (Int 类型), 默认 800, 范围 1-20000。仅 mode 为"指定尺寸"时可见。

target_height, 整数输入框 (Int 类型), 默认 600, 范围 1-20000。仅 mode 为"指定尺寸"时可见。

scale_factor, 浮点滑条 (DoubleSlider 类型), 默认 1.0, 范围 0.01-20.0。仅 mode 为"按比例缩放"时可见。

keep_aspect_ratio, 复选框 (Bool 类型), 默认勾选。仅 mode 为"指定尺寸"时可见。勾选时以目标尺寸为上限等比缩放, 不勾选则强制拉伸到目标尺寸。

interpolation, 下拉框, 选项"最近邻""双线性""双三次""Lanczos", 默认"双线性"。

处理逻辑: 计算目标尺寸后使用 cv::resize。若 keep_aspect_ratio 勾选且 mode 为指定尺寸, 先计算宽高比, 取较小的缩放比例, 使结果不超过 target_width 和 target_height。映射插值方法: 最近邻→INTER_NEAREST, 双线性→INTER_LINEAR, 双三次→INTER_CUBIC, Lanczos→INTER_LANCZOS4。

节点 12 裁剪 Crop

功能: 裁剪图像的矩形区域。

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

x, 整数输入框, 默认 0。裁剪区域左上角 x 坐标。

y, 整数输入框, 默认 0。

crop_width, 整数输入框, 默认 100。裁剪区域宽度。

crop_height, 整数输入框, 默认 100。裁剪区域高度。

处理逻辑: 将裁剪区域与图像边界取交集。若交集为空 (完全越界), 报错"裁剪区域超出图像范围"。若部分越界, 自动裁剪到有效范围并在日志中记录警告。使用 cv::Mat 的 ROI 操作并 clone 输出。

节点 13 旋转 Rotate

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

angle, 浮点输入框, -360 到 360, 默认 0。顺时针旋转角度。

expand_canvas, 复选框, 默认勾选。勾选时旋转后扩大画布使图像完整显示, 不勾选则保持原尺寸裁切超出部分。

fill_color, 颜色选取器, 默认黑色。扩展区域的填充颜色。

处理逻辑: 使用 cv::getRotationMatrix2D 和 cv::warpAffine。旋转中心固定为图像中心。
若 expand_canvas 勾选, 计算旋转后的包围矩形尺寸作为输出尺寸。

节点 14 翻转 Flip

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

direction, 下拉框, 选项"水平翻转""垂直翻转""水平+垂直翻转", 默认"水平翻转"。

处理逻辑: 使用 cv::flip, flipCode 分别对应 1、0、-1。

子类: 高级变换

节点 15 仿射变换 AffineTransform

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

src_points, 三对坐标输入 (六个浮点数输入框, 标注为"源点 1 x/y""源点 2 x/y""源点 3

x/y"), 无默认值。

dst_points, 三对坐标输入 (六个浮点数输入框), 无默认值。

output_width, 整数输入框, 默认为输入图像宽度。

output_height, 整数输入框, 默认为输入图像高度。

fill_color, 颜色选取器, 默认黑色。

处理逻辑: cv::getAffineTransform 计算变换矩阵, cv::warpAffine 执行变换。若源点或目标点未填写完全, 报错。

节点 16 透视变换 PerspectiveTransform

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

src_points, 四对坐标输入 (八个浮点数输入框)。

dst_points, 四对坐标输入 (八个浮点数输入框)。

output_width, 整数输入框。

output_height, 整数输入框。

fill_color, 颜色选取器, 默认黑色。

处理逻辑: cv::getPerspectiveTransform+cv::warpPerspective。

3.3 第 3 大类: 色彩调整

子类: 基础调整

节点 17 亮度/对比度 BrightnessContrast

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

brightness, 整数滑条, -100 到 100, 默认 0。

contrast, 浮点滑条, 0.0 到 3.0, 默认 1.0。

处理逻辑: $output = input * contrast + brightness$ 。使用 cv::Mat::convertTo 实现。像素值 clamp 到 [0,255]。

节点 18 反色 Invert

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数: 无。

处理逻辑: $\text{output} = 255 - \text{input}$ 。使用 cv::bitwise_not。

节点 19 色阶 Levels

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

input_black, 整数滑条, 0 到 255, 默认 0。输入黑点。

input_white, 整数滑条, 0 到 255, 默认 255。输入白点。必须大于 input_black。

input_gamma, 浮点滑条, 0.1 到 10.0, 默认 1.0。中间调 Gamma 值。

output_black, 整数滑条, 0 到 255, 默认 0。输出黑点。

output_white, 整数滑条, 0 到 255, 默认 255。输出白点。

处理逻辑: 先将像素值映射到[input_black, input_white]范围内 (低于 input_black 的置为 0, 高于 input_white 的置为 1, 中间做线性归一化), 然后施加 Gamma 校正 (值的 gamma 次方), 最后映射到[output_black, output_white]输出范围。使用 cv::LUT 构建查找表一次性应用。若图像为多通道, 对每个通道分别应用。

子类: HSL 调整

节点 20 色相偏移 HueShift

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

hue_offset, 整数滑条, -180 到 180, 默认 0。

处理逻辑: 将图像转为 HSV 空间, H 通道加上 offset 后对 180 取模 (OpenCV 中 H 的范围是 0-179), 再转回 BGR。

节点 21 饱和度调整 SaturationAdjust

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

saturation_factor, 浮点滑条, 0.0 到 3.0, 默认 1.0。1.0 为不变, 0.0 为完全去色, 大于 1.0 为增强饱和度。

处理逻辑：转 HSV，S 通道乘以 factor 并 clamp 到[0,255]，转回 BGR。

节点 22 明度调整 LightnessAdjust

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

lightness_offset，整数滑条，-100 到 100，默认 0。

处理逻辑：转 HLS，L 通道加 offset 后 clamp，转回 BGR。

子类：高级调整

节点 23 色彩平衡 ColorBalance

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

shadow_r，整数滑条，-50 到 50，默认 0。阴影区域 R 通道偏移。

shadow_g，整数滑条，-50 到 50，默认 0。

shadow_b，整数滑条，-50 到 50，默认 0。

midtone_r，整数滑条，-50 到 50，默认 0。中间调 R 偏移。

midtone_g，midtone_b，同上。

highlight_r，highlight_g，highlight_b，同上。高光区域偏移。

处理逻辑：对每个像素计算其亮度值（取 BGR 三通道均值），据此将像素归类为阴影（亮度<85）、中间调（85-170）、高光（>170），归类使用平滑过渡权重而非硬切分。对各通道加上对应区域的偏移量，按权重混合。

节点 24 曲线调整 Curves

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

channel，下拉框，选项"RGB""R""G""B"，默认"RGB"。选"RGB"时对所有通道施加相同曲线。

control_points，曲线编辑器控件。控件呈现一个 256x256 的坐标区域，x 轴为输入值，y 轴为输出值。默认有两个端点(0,0)和(255,255)，用户可以点击添加中间控制点，拖拽控制点位置。最多支持 10 个控制点。

处理逻辑：对控制点进行三次样条插值，生成一个 256 项的查找表。用 `cv::LUT` 应用到选中的通道上。若 `channel` 为 RGB，查找表对三个通道共用。

节点 25 色温/白平衡 WhiteBalance

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

temperature，整数滑条，2000 到 10000，默认 6500。色温值，单位 K。低于 6500 偏暖（增加 R 减少 B），高于 6500 偏冷（增加 B 减少 R）。

处理逻辑：根据色温值计算 R 和 B 通道的增益系数（使用简化的色温-RGB 映射公式：偏离 6500K 的程度线性映射到增益偏移），对 R 和 B 通道分别乘以增益系数。

3.4 第 4 大类：滤波与模糊

子类：模糊

节点 26 高斯模糊 GaussianBlur

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

kernel_size，整数滑条，1 到 99（仅奇数），默认 5。若输入偶数自动加 1。

sigma，浮点输入框，0.1 到 50.0，默认 1.0。

处理逻辑：`cv::GaussianBlur`。

节点 27 中值滤波 MedianFilter

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

kernel_size，整数滑条，3 到 99（仅奇数），默认 5。

处理逻辑：`cv::medianBlur`。

节点 28 双边滤波 BilateralFilter

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

diameter, 整数输入框, 1 到 50, 默认 9。

sigma_color, 浮点输入框, 1.0 到 300.0, 默认 75.0。

sigma_space, 浮点输入框, 1.0 到 300.0, 默认 75.0。

处理逻辑: cv::bilateralFilter。

节点 29 运动模糊 MotionBlur

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

angle, 浮点输入框, 0 到 360, 默认 0。运动方向角度。

length, 整数输入框, 1 到 200, 默认 15。运动长度 (核大小)。

处理逻辑: 生成 lengthxlength 大小的核矩阵, 仅在 angle 方向的一条线上为 1/length, 其余为 0。用 cv::filter2D 卷积。

子类: 锐化

节点 30 USM 锐化 Sharpen

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

radius, 整数滑条 (IntSlider 类型), 1 到 50, 默认 3。高斯模糊半径 (核大小=radius*2+1 确保奇数)。

amount, 浮点滑条 (DoubleSlider 类型), 0.0 到 5.0, 默认 1.5。锐化强度。

threshold, 整数滑条 (IntSlider 类型), 0 到 255, 默认 0。只有差值大于此阈值的像素才被锐化。

处理逻辑: 先对原图做半径为 radius 的高斯模糊得到 blurred, 将两者转浮点(32F)计算 $\text{diff} = \text{srcF} - \text{blurredF}$ 。若 $\text{threshold} > 0$, 取各通道差值绝对值的最大值与 threshold 比较, 小于 threshold 的像素 diff 置零。输出 = $\text{srcF} + \text{amount} * \text{diff}$, 最终 convertTo 为原图深度。

节点 31 自定义卷积核 CustomKernel

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

kernel_size, 下拉框, 选项 "3x3" "5x5" "7x7", 默认 "3x3"。

kernel_values, 网格输入控件, 根据 **kernel_size** 呈现对应大小的数值输入网格。默认全 0 中心为 1 (单位核)。

normalize, 复选框, 默认勾选。勾选时将核除以所有元素之和 (若和为 0 则不做归一化)。

处理逻辑: `cv::filter2D`。

子类: 增强

节点 32 CLAHE 自适应均衡化 Clahe

功能: 限制对比度自适应直方图均衡化, 改善图像局部对比度而不放大噪声, 对标专业图像处理软件。

输入端口: **input_image**, 类型 **Image**, 必需。

输出端口: **output_image**, 类型 **Image**。

参数:

clip_limit, 浮点滑条 (**DoubleSlider** 类型), 0.1 到 10.0, 默认 2.0。对比度限制阈值。

tile_size, 整数滑条 (**IntSlider** 类型), 1 到 32, 默认 8。网格尺寸。

处理逻辑: 确保 8 位深度 (16 位转 8 位)。若为彩色图 (通道数>1), 四通道先转 BGR, 再转 Lab 色彩空间, 对 L 通道应用 `cv::createCLAHE(clip_limit, Size(tile_size, tile_size))` 处理, 然后合并并转回 BGR。若单通道灰度图, 直接应用 CLAHE。

3.5 第 5 大类: 形态学操作

子类: 基本运算

节点 33 膨胀 Dilate

输入端口: **input_image**, 类型 **Image**, 必需。

输出端口: **output_image**, 类型 **Image**。

参数 (通过 **MorphBase** 基类统一管理):

kernel_shape, 下拉框, 选项 "矩形" "椭圆" "十字", 默认 "矩形"。

kernel_size, 整数滑条, 1 到 50, 默认 3。

iterations, 整数输入框, 1 到 20, 默认 1。

处理逻辑: `cv::getStructuringElement` 构建核, `cv::dilate` 执行。

节点 34 腐蚀 Erode

端口和参数与节点 33 完全相同。处理逻辑: `cv::erode`。

子类：复合运算

节点 35 开运算 MorphOpen

端口和参数与节点 33 相同。处理逻辑：cv::morphologyEx，op = MORPH_OPEN。

节点 36 闭运算 MorphClose

同上，op = MORPH_CLOSE。

节点 37 形态学梯度 MorphGradient

同上，op = MORPH_GRADIENT。输出为膨胀图与腐蚀图的差值，突出边缘。

节点 38 顶帽变换 TopHat

同上，op = MORPH_TOPHAT。

节点 39 黑帽变换 BlackHat

同上，op = MORPH_BLACKHAT。

3.6 第 6 大类：边缘检测与分割

子类：边缘检测

节点 40 Canny 边缘检测 CannyEdge

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

threshold_low，浮点输入框，0 到 500，默认 50。

threshold_high，浮点输入框，0 到 500，默认 150。必须大于 threshold_low。

aperture_size，下拉框，选项"3""5""7"，默认"3"。

处理逻辑：若输入为多通道，先转灰度。cv::Canny。输出为单通道灰度图。

节点 41 Sobel 算子 Sobel

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

direction，下拉框，选项"水平(X)" "垂直(Y)" "X 和 Y 合并"，默认"X 和 Y 合并"。

kernel_size，下拉框，选项"3""5""7"，默认"3"。

处理逻辑：若选 X，dx=1 dy=0。选 Y，dx=0 dy=1。选合并，分别计算 X 和 Y 然后 cv::magnitude 合并。输入多通道先转灰度。输出归一化到 0-255 的单通道图。

节点 42 Laplacian 算子 Laplacian

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

kernel_size, 下拉框, 选项"1""3""5""7", 默认"3"。

处理逻辑: 输入多通道先转灰度。cv::Laplacian, 取绝对值后转 8U。

子类: 阈值分割

节点 43 全局阈值 GlobalThreshold

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

threshold_value, 整数滑条, 0 到 255, 默认 128。

max_value, 整数滑条, 0 到 255, 默认 255。

type, 下拉框, 选项"二值化(BINARY)""反二值化(BINARY_INV)""截断(TRUNC)""置零(TOZERO)""反置零(TOZERO_INV)", 默认"二值化"。

处理逻辑: 输入多通道先转灰度。cv::threshold。

节点 44 自适应阈值 AdaptiveThreshold

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

max_value, 整数滑条, 0 到 255, 默认 255。

method, 下拉框, 选项"均值(MEAN_C)""高斯(GAUSSIAN_C)", 默认"高斯"。

type, 下拉框, 选项"二值化""反二值化", 默认"二值化"。

block_size, 整数滑条, 3 到 99 (仅奇数), 默认 11。

constant_c, 浮点输入框, -50 到 50, 默认 2。

处理逻辑: 输入多通道先转灰度。cv::adaptiveThreshold。

节点 45 Otsu 自动阈值 OtsuThreshold

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。output_threshold, 类型 Numeric (输出自动计算得到的阈值)。

参数:

max_value, 整数滑条, 0 到 255, 默认 255。

处理逻辑: 输入多通道先转灰度。cv::threshold 配合 THRESH_OTSU 标志。将计算得到的阈值通过 Numeric 端口输出。

子类: 轮廓

节点 46 轮廓检测与绘制 FindContours

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。output_count, 类型 Numeric (轮廓数量)。

参数:

mode, 下拉框, 选项"外轮廓(EXTERNAL)""所有轮廓(LIST)""树形(TREE)", 默认"外轮廓"。

method, 下拉框, 选项"无近似(NONE)""简单近似(SIMPLE)", 默认"简单近似"。

min_area, 浮点输入框, 0 到 1000000, 默认 0。面积小于此值的轮廓被忽略。

draw_on_original, 复选框, 默认勾选。勾选时在原图上绘制轮廓, 否则在黑色背景上绘制。

draw_color, 颜色选取器, 默认绿色(0,255,0)。

line_width, 整数滑条, 1 到 10, 默认 2。

处理逻辑: 若输入非灰度, 先转灰度再做阈值二值化 (使用 Otsu 自动阈值)。cv::findContours 找到轮廓后按 min_area 过滤, 然后 cv::drawContours 绘制。

3.7 第 7 大类: 通道操作

子类: 拆分合并

节点 47 通道拆分 ChannelSplit

输入端口: input_image, 类型 Image, 必需。

输出端口: output_ch0, 类型 Image。output_ch1, 类型 Image。output_ch2, 类型 Image。output_ch3, 类型 Image。ch3 仅在输入为四通道时有数据, 否则输出空。

参数:

color_space, 下拉框, 选项"BGR (拆为 B/G/R)""HSV (拆为 H/S/V)""Lab (拆为 L/a/b)""HLS (拆为 H/L/S)", 默认"BGR"。

处理逻辑: 若选择的色彩空间与当前图像不同, 先转换色彩空间, 再用 cv::split 拆分。每

个输出端口输出的是单通道灰度图。

节点 48 通道合并 ChannelMerge

输入端口: input_ch0, 类型 Image, 必需。input_ch1, 类型 Image, 必需。input_ch2, 类型 Image, 必需。input_ch3, 类型 Image, 可选 (若接入则输出四通道)。

输出端口: output_image, 类型 Image。

参数:

color_space, 下拉框, 选项"BGR""HSV""Lab""HLS", 默认"BGR"。

处理逻辑: 各输入端口必须为单通道灰度图, 若不是则报错。用 cv::merge 合并为多通道图, 若目标色彩空间非 BGR, 合并后再转回 BGR。若各通道尺寸不一致, 报错"输入通道尺寸不一致"。

子类: 色彩空间转换

节点 49 转灰度 ConvertToGray

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数: 无。

处理逻辑: 若已是单通道则直接输出。否则 cv::cvtColor(BGR2GRAY)。

节点 50 转 HSV ConvertToHSV

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数: 无。

处理逻辑: cv::cvtColor(BGR2HSV)。输入须为三通道, 单通道报错。

节点 51 转 Lab ConvertToLab

同上, cv::cvtColor(BGR2Lab)。

节点 52 转 YCrCb ConvertToYCrCb

同上, cv::cvtColor(BGR2YCrCb)。

子类: Alpha 通道

节点 53 提取 Alpha 通道 ExtractAlpha

输入端口: input_image, 类型 Image, 必需。

输出端口: output_alpha, 类型 Image。

参数：无。

处理逻辑：若输入为四通道，拆分取第四通道输出。若输入不含 Alpha（三通道或单通道），输出一个与输入同尺寸的全白灰度图（表示完全不透明）并在日志中提示"输入图像不含 Alpha 通道，已输出全白蒙版"。

节点 54 设置 Alpha 通道 SetAlpha

输入端口：input_image，类型 Image，必需。input_mask，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：无。

处理逻辑：input_mask 须为单通道灰度图，否则自动转灰度并警告。input_image 若为四通道则替换 Alpha，若为三通道则追加 Alpha 变为四通道。二者尺寸须一致，不一致则将 mask 缩放到 image 尺寸并警告。

3.8 第 8 大类：图层与合成

子类：混合

节点 55 图像混合 Blend

功能：两张图像按混合模式叠加。

输入端口：input_base，类型 Image，必需。底层图像。input_top，类型 Image，必需。顶层图像。

输出端口：output_image，类型 Image。

参数：

blend_mode，下拉框（Combo 类型），选项"正常(Normal)""正片叠底(Multiply)""滤色(Screen)""叠加(Overlay)""柔光(SoftLight)""强光(HardLight)""差值(Difference)""排除(Exclusion)""变暗(Darken)""变亮(Lighten)""颜色加深(ColorBurn)""颜色减淡(ColorDodge)""线性加深(LinearBurn)""线性减淡(LinearDodge)""亮光(VividLight)"，默认"正常(Normal)"。

opacity，浮点滑条（DoubleSlider 类型），0.0 到 1.0，默认 1.0。顶层图像的不透明度。

处理逻辑：若两图尺寸不同，将 top 缩放到 base 尺寸。通道数不同时统一为三通道（灰度转 BGR，BGRA 转 BGR）。逐像素按照选定混合模式的数学公式计算，然后按 opacity 做线性混合： $result = base * (1 - opacity) + blended * opacity$ 。

各混合模式的像素公式（a 为 base 归一化到[0,1]，b 为 top 归一化到[0,1]）：

Normal: b

Multiply: $a * b$

Screen: $1 - (1-a)*(1-b)$

Overlay: if $a < 0.5$ then $2*a*b$ else $1-2*(1-a)*(1-b)$

SoftLight: if $b < 0.5$ then $a-(1-2*b)*a*(1-a)$ else $a+(2*b-1)*(sqrt(a)-a)$

HardLight: if $b < 0.5$ then $2*a*b$ else $1-2*(1-a)*(1-b)$

Difference: $abs(a-b)$

Exclusion: $a+b-2*a*b$

Darken: $min(a,b)$

Lighten: $max(a,b)$

ColorBurn: if $b == 0$ then 0 else $1-min(1,(1-a)/b)$

ColorDodge: if $b == 1$ then 1 else $min(1,a/(1-b))$

LinearBurn: $max(0, a+b-1)$

LinearDodge: $min(1, a+b)$

VividLight: if $b < 0.5$ then $ColorBurn(a,2*b)$ else $ColorDodge(a,2*b-1)$

节点 56 蒙版混合 MaskBlend

输入端口: input_a, 类型 Image, 必需。input_b, 类型 Image, 必需。input_mask, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数: 无。

处理逻辑: mask 须为单通道灰度图, 否则自动转灰度。三张图尺寸须一致, 不一致时将 b 和 mask 缩放到 a 的尺寸并警告。 $output = a * (1 - mask/255) + b * (mask/255)$, 逐像素计算。

节点 57 透明度调整 OpacityAdjust

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

opacity, 浮点滑条, 0.0 到 1.0, 默认 1.0。

处理逻辑: 若输入无 Alpha 通道, 先增加 Alpha 通道 (全 255)。然后 Alpha 通道每个值乘以 opacity 系数。

子类: 拼接

节点 58 图像堆叠 ImageStack

功能: 将两张图上下或左右拼接为一张。

输入端口: input_a, 类型 Image, 必需。input_b, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

direction, 下拉框, 选项"水平(左右)" "垂直(上下)", 默认"水平"。

alignment, 下拉框。水平方向时选项为"顶部对齐" "居中对齐" "底部对齐", 垂直方向时选项为"左对齐" "居中对齐" "右对齐"。默认"居中对齐"。

gap, 整数输入框, 0 到 200, 默认 0。两图之间的间距像素数。

gap_color, 颜色选取器, 默认白色。间距区域的填充颜色。

处理逻辑: 计算输出尺寸(水平方向: 宽=a.width+gap+b.width, 高=max(a.height, b.height)。垂直同理)。创建输出图像用 gap_color 填充, 按 alignment 将两张图分别绘制到对应位置。通道数不一致时统一为较多通道数。

节点 59 图像网格拼接 ImageGridStitch

功能: 将 ImageList 中的图像按网格排列成一张大图。

输入端口: input_images, 类型 ImageList, 必需。

输出端口: output_image, 类型 Image。

参数:

columns, 整数输入框, 1 到 10, 默认 2。列数。行数自动计算=ceil(图片数/columns)。

cell_width, 整数输入框, 50 到 5000, 默认 0。0 表示自动取列表中最大图片的宽度。

cell_height, 整数输入框, 50 到 5000, 默认 0。0 同上取最大高度。

gap, 整数输入框, 0 到 100, 默认 10。格间距。

background_color, 颜色选取器, 默认白色。

resize_mode, 下拉框, 选项"等比缩放适配" "拉伸填满" "不缩放居中", 默认"等比缩放适配"。

处理逻辑: 计算网格总尺寸, 创建背景画布, 逐个将图片缩放/定位到对应格子中。列表为空时报错。

子类: 水印

节点 60 图片水印叠加 ImageWatermark

输入端口: input_image, 类型 Image, 必需。input_watermark, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

position, 下拉框, 选项"左上""上中""右上""左中""正中""右中""左下""下中""右下""自定义坐标", 默认"右下"。

custom_x, 整数输入框, 默认 0。仅"自定义坐标"时显示。

custom_y, 整数输入框, 默认 0。

scale, 浮点输入框, 0.01 到 5.0, 默认 0.2。水印图相对于主图宽度的缩放比例。

opacity, 浮点滑条, 0.0 到 1.0, 默认 0.5。

tiling, 复选框, 默认不勾选。勾选时忽略 **position** 和 **scale** 参数, 水印以原始大小平铺满整张图。

处理逻辑: 将水印图按 **scale** 缩放后, 根据 **position** 计算叠加坐标, 以 **opacity** 为透明度叠加到主图上。水印图若有 Alpha 通道则作为蒙版, 若无则直接按 **opacity** 混合。平铺模式下水印图从左上角开始重复铺满画布。

节点 61 文字水印叠加 TextWatermark

输入端口: **input_image**, 类型 Image, 必需。

输出端口: **output_image**, 类型 Image。

参数:

text, 文本输入框, 默认"WATERMARK"。

font_family, 系统字体选择下拉框, 默认为系统默认字体。

font_size, 整数输入框, 8 到 200, 默认 36。

color, 颜色选取器, 默认半透明白色(255,255,255,128)。

position, 同节点 60。

rotation, 浮点输入框, -90 到 90, 默认-30。文字旋转角度, 常用于对角线水印。

tiling, 复选框, 默认不勾选。勾选时文字按行列平铺, 行间距和列间距自动根据字号计算。

处理逻辑: 使用 QPainter 在 QImage 上渲染文字 (利用 Qt 的字体渲染引擎), 然后将 QImage 转换为 cv::Mat, 作为水印层叠加到主图上。旋转文字时先旋转绘制好文字的临时图像再叠加。

3.9 第 9 大类: 文字与标注

子类: 文字

节点 62 文字叠加 TextOverlay

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

text, 多行文本输入框, 默认空。

font_family, 系统字体选择下拉框。

font_size, 整数输入框, 8 到 500, 默认 24。

color, 颜色选取器, 默认黑色。

x, 整数输入框, 默认 10。文字左上角 x 坐标。

y, 整数输入框, 默认 30。文字左上角 y 坐标。

alignment, 下拉框, 选项"左对齐""居中""右对齐", 默认"左对齐"。

line_spacing, 浮点输入框, 0.5 到 3.0, 默认 1.2。行间距倍数。

stroke_color, 颜色选取器, 默认无 (设为与 color 相同时视为无描边)。

stroke_width, 整数输入框, 0 到 10, 默认 0。

处理逻辑: 使用 QPainter 在 QImage 上渲染。多行文字按换行符分行, 按 line_spacing 计算行高。描边通过 QPainterPath 的 stroke 实现。渲染后转为 cv::Mat 覆盖到对应位置。

节点 63 艺术字生成 ArtisticText

功能: 生成带有高级视觉效果的文字图像, 可用作标题。

输入端口: input_image, 类型 Image, 可选。若接入则在该图像上绘制, 若未接入则在透明背景上生成文字图像。

输出端口: output_image, 类型 Image。

参数:

text, 文本输入框。

font_family, 系统字体选择下拉框。

font_size, 整数输入框, 12 到 800, 默认 72。

fill_mode, 下拉框, 选项"纯色""水平渐变""垂直渐变""径向渐变", 默认"纯色"。

fill_color1, 颜色选取器, 默认白色。

fill_color2, 颜色选取器, 默认黑色。仅渐变模式显示。

stroke_color, 颜色选取器, 默认黑色。

stroke_width, 整数输入框, 0 到 20, 默认 2。

shadow_enabled, 复选框, 默认勾选。

shadow_color, 颜色选取器, 默认半透明黑色(0,0,0,128)。

shadow_offset_x, 整数输入框, -20 到 20, 默认 3。

shadow_offset_y, 整数输入框, -20 到 20, 默认 3。

shadow_blur, 整数输入框, 0 到 20, 默认 5。

letter_spacing, 整数输入框, -10 到 50, 默认 0。字间距像素偏移。

position_x, 整数输入框, 默认居中。仅当有 input_image 时有效。

position_y, 整数输入框, 默认居中。

处理逻辑: 先在一张透明 QImage 上使用 QPainterPath 构建文字路径。应用渐变填充 (QLinearGradient 或 QRadialGradient), 应用描边 (QPen 设定 stroke_width 和 stroke_color), 然后绘制阴影层 (将文字路径偏移 shadow_offset 后用 shadow_color 填充并做高斯模糊)。将阴影层绘制在文字层之下。最终合成为一张带透明通道的图像, 若有 input_image 则叠加到指定位置上。

子类: 科研标注

节点 64 箭头与标注 Annotation

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

annotations, 标注列表编辑器控件。用户可以添加多个标注项, 每项包含:

- 类型, 下拉框, 选项"箭头""矩形框""圆形框""直线""文字标签"。
- 颜色, 颜色选取器。
- 线宽, 整数, 1 到 10。
- 坐标参数 (箭头: 起点 xy+终点 xy。矩形: 左上 xy+宽高。圆形: 圆心 xy+半径。直线: 起点 xy+终点 xy。文字标签: 位置 xy+文本内容+字号)。

处理逻辑: 使用 QPainter 逐个绘制标注项。箭头的箭头头部使用三角形填充绘制。

节点 65 比例尺绘制 ScaleBar

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

pixel_per_unit, 浮点输入框, 无默认值。每个像素代表的实际长度 (单位由 unit 参数决定)。

unit, 下拉框, 选项"nm""mum""mm""cm""m""km", 默认"mum"。

bar_length_unit, 浮点输入框, 默认 10。比例尺表示的实际长度值 (例如输入 10, unit 选 mum, 则比例尺标注"10 mum")。

position, 下拉框 (九宫格位置), 默认"左下"。

bar_color, 颜色选取器, 默认白色。

bar_height, 整数输入框, 2 到 20, 默认 5。比例尺矩形条的像素高度。

font_size, 整数输入框, 8 到 50, 默认 14。

background_enabled, 复选框, 默认勾选。勾选时在比例尺区域下方绘制半透明黑色背景方便阅读。

处理逻辑: 计算比例尺的像素长度 = $\text{bar_length_unit} / \text{pixel_per_unit}$ 。在指定位置绘制矩形条和标注文字。若计算出的像素长度超过图像宽度的 80%或小于 10 像素, 警告"比例尺长度异常, 请检查参数"。

节点 66 编号标注 NumberLabel

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

label_text, 文本输入框, 默认"(a)"。

position, 下拉框, 选项"左上""右上""左下""右下", 默认"左上"。

font_family, 系统字体下拉框。

font_size, 整数输入框, 8 到 72, 默认 18。

text_color, 颜色选取器, 默认黑色。

background_color, 颜色选取器, 默认半透明白色(255,255,255,200)。

padding, 整数输入框, 0 到 20, 默认 4。文字与背景边缘的间距。

处理逻辑: 在指定角落绘制一个背景矩形, 矩形内居中显示标签文字。

节点 67 图例叠加 LegendOverlay

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

items, 图例项列表编辑器。每项包含: 颜色 (颜色选取器)、标签文字 (文本输入框)、样式 (下拉框: 方块/圆形/线条)。

`position`, 下拉框（九宫格位置），默认"右上"。

`font_size`, 整数输入框，默认 12。

`background_color`, 颜色选取器，默认半透明白色。

`border_enabled`, 复选框，默认勾选。

处理逻辑：计算图例框的总尺寸（取决于项数和文字长度），在指定位置绘制背景框，逐行绘制色块/圆形/线条和对应标签。

3.10 第 10 大类：数据输入输出

子类：数据读取

节点 68 CSV 文件读取 CSVRead

输入端口：无。

输出端口：`output_data`，类型 `DataTable`。

参数：

`file_path`, 文件选择器，无默认值。

`delimiter`, 下拉框，选项"逗号""制表符""分号""空格""自定义"，默认"逗号"。选"自定义"时出现文本输入框。

`has_header`, 复选框，默认勾选。勾选时第一行作为列名，否则自动生成列名（`Column_0`, `Column_1...`）。

`encoding`, 下拉框，选项"UTF-8""GBK""Latin-1"，默认"UTF-8"。

处理逻辑：逐行读取文件，按 `delimiter` 分割。尝试将每个值转为 `double`，若失败则保留为 `QString`。构建 `DataTable`。若文件为空或无法打开，报错。

节点 69 Excel 文件读取 ExcelRead

输入端口：无。

输出端口：`output_data`，类型 `DataTable`。

参数：

`file_path`, 文件选择器，无默认值。过滤器只显示.xlsx 文件。

`sheet`, 下拉框或文本输入框。打开文件后自动列出所有工作表名供选择。默认第一个工作表。

`has_header`, 复选框，默认勾选。

处理逻辑：使用 `QXlsx` 打开文件，读取指定工作表的数据。

节点 70 JSON 数据读取 JSONDataRead

输入端口：无。

输出端口：output_data，类型 DataTable。

参数：

file_path，文件选择器。

root_path，文本输入框，默认空。指定 JSON 中数据数组的路径，用点号分隔（如 "data.records"）。空表示根节点即为数组。

处理逻辑：读取 JSON 文件，解析到指定路径。若目标是一个 JSON 数组，每个元素视为一行，元素中的 key 作为列名，value 作为值。若目标不是数组，报错"指定路径下的数据不是数组"。

节点 71 手动数据输入 ManualDataInput

输入端口：无。

输出端口：output_data，类型 DataTable。

参数：

table_editor，表格编辑器控件。在属性面板中嵌入一个可编辑表格，用户可以手动填写行列数据。支持添加/删除行列，编辑单元格值，右键粘贴从剪贴板导入。

处理逻辑：直接从表格编辑器控件中读取数据构建 DataTable。

子类：数据导出

节点 72 数据导出 DataExport

输入端口：input_data，类型 DataTable，必需。

输出端口：无。

参数：

file_path，文件保存对话框。

format，下拉框，选项"CSV""JSON"，默认"CSV"。

delimiter，下拉框（仅 CSV 格式显示），同节点 68。

encoding，下拉框，同节点 68。

处理逻辑：按格式写入文件。CSV 按行写入，JSON 输出为数组嵌套对象的格式。

3.11 第 11 大类：文本/数值输入输出

子类：文本输入输出

节点 73 文本文件读取 TextFileRead

功能：从磁盘读取文本文件的全部内容，输出为 Text 类型。

输入端口：无。

输出端口：output_text，类型 Text。

参数：

file_path，文件选择器（FilePath 类型），无默认值。过滤器为"文本文件 (*.txt);;所有文件 (*)"。

encoding，下拉框（Combo 类型），选项"UTF-8""GBK""Latin-1"，默认"UTF-8"。

处理逻辑：使用 QFile 以只读文本模式打开文件。若路径为空报错"未指定输入文件路径"。若文件不存在报错"文件不存在：{路径}"。若无法打开报错"无法读取文件：{路径}"。使用 QTextStream 读取全部内容，根据 encoding 设置编码（Qt5 使用 setCodec，Qt6 使用 setEncoding）。记录日志"读取文本文件: {路径} ({N} 字符)"。

节点 74 文本文件导出 TextFileWrite

功能：将 Text 类型的数据写入磁盘文件。

输入端口：input_text，类型 Text，必需。

输出端口：无。

参数：

file_path，文件保存对话框（SaveFilePath 类型），默认值为当前目录下的"output.txt"。过滤器为"文本文件 (*.txt);;所有文件 (*)"。

encoding，下拉框（Combo 类型），选项"UTF-8""GBK""Latin-1"，默认"UTF-8"。

处理逻辑：使用 QFile 以只写文本模式打开文件。若输出目录不存在则调用 mkdir 自动创建。UTF-8 编码时设置 GenerateByteOrderMark。写入完毕后记录日志"文本已导出: {路径} ({N} 字符)"。

节点 75 数值文件导出 NumericWrite

功能：将一个 Numeric 数值按指定精度写入文本文件。

输入端口：input_value，类型 Numeric，必需。

输出端口：无。

参数：

`file_path`, 文件保存对话框 (`SaveFilePath` 类型), 默认值为当前目录下的"output.txt"。

`precision`, 整数输入框 (`Int` 类型), 0 到 15, 默认 6。小数的保留位数。

处理逻辑: 使用 `QString::number(value, 'f', precision)` 格式化数值后写入文件。若输出目录不存在自动创建。记录日志"数值已导出: {路径} = {格式化后的值}"。

3.12 第 12 大类: 数据处理

子类: 数据清洗

节点 76 数据过滤 `DataFilter`

输入端口: `input_data`, 类型 `DataTable`, 必需。

输出端口: `output_data`, 类型 `DataTable`。

参数:

`column`, 下拉框 (从输入 `DataTable` 的列名中动态填充)。

`operator`, 下拉框, 选项"等于""不等于""大于""大于等于""小于""小于等于""包含""不包含""为空""不为空", 默认"大于"。

`value`, 文本输入框。对于"为空""不为空"操作符此参数隐藏。

处理逻辑: 逐行检查指定列是否满足条件, 保留满足条件的行。数值比较时若值无法转为 `double` 则视为不满足。"包含""不包含"对字符串做子串匹配。

节点 77 缺失值处理 `HandleMissing`

输入端口: `input_data`, 类型 `DataTable`, 必需。

输出端口: `output_data`, 类型 `DataTable`。

参数:

`method`, 下拉框, 选项"删除含缺失值的行""填充固定值""填充列均值""填充列中位数""前向填充 (用上一行的值)", 默认"删除含缺失值的行"。

`fill_value`, 文本输入框, 默认"0"。仅 `method` 为"填充固定值"时显示。

`target_columns`, 多选下拉框 (列出所有列名), 默认全选。指定对哪些列处理, 未选中的列不动。

处理逻辑: 遍历 `target_columns` 中的每列, 按 `method` 处理空值。均值和中位数仅对可转为 `double` 的值计算, 非数值列报警跳过。

节点 78 去重 `RemoveDuplicates`

输入端口: `input_data`, 类型 `DataTable`, 必需。

输出端口: output_data, 类型 DataTable。output_removed_count, 类型 Numeric。

参数:

key_columns, 多选下拉框, 默认全选。以哪些列的组合作为判重依据。

keep, 下拉框, 选项"保留第一条""保留最后一条", 默认"保留第一条"。

处理逻辑: 按 key_columns 的值组合判断重复, 保留指定的一条, 删除其余。输出被删除的行数。

子类: 数据变换

节点 79 数据排序 DataSort

输入端口: input_data, 类型 DataTable, 必需。

输出端口: output_data, 类型 DataTable。

参数:

sort_column, 下拉框。

order, 下拉框, 选项"升序""降序", 默认"升序"。

处理逻辑: 按指定列排序。对可转 double 的列用数值排序, 否则用字符串字典序。

节点 80 数据归一化 DataNormalize

输入端口: input_data, 类型 DataTable, 必需。

输出端口: output_data, 类型 DataTable。

参数:

target_columns, 多选下拉框, 默认全选。

method, 下拉框, 选项"Min-Max 归一化(映射到 0-1)""Z-Score 标准化", 默认"Min-Max"。

处理逻辑: 对每个目标列的数值做归一化。非数值项跳过不处理。Min-Max: $(x - \min) / (\max - \min)$ 。Z-Score: $(x - \text{mean}) / \text{std}$ 。若某列全部相同 (std=0), 归一化结果全为 0 并警告。

节点 81 列运算 ColumnCalculate

输入端口: input_data, 类型 DataTable, 必需。

输出端口: output_data, 类型 DataTable。

参数:

new_column_name, 文本输入框, 默认"new_col"。

expression, 文本输入框, 默认空。表达式中用列名引用列的值, 支持四则运算和常见数学函数。

支持的运算符: +、-、*、/、% (取模)、^ (幂)。

支持的函数: sqrt、abs、log (自然对数)、log10、sin、cos、tan、ceil、floor、round、min(a,b)、max(a,b)。

处理逻辑: 实现一个简单的表达式解析器 (递归下降解析或使用逆波兰表达式), 逐行对每行的列值代入计算, 结果作为新列追加到 DataTable 末尾。表达式语法错误报错。除以零时该行结果为空值并警告。

子类: 数据统计

节点 82 数据聚合统计 DataAggregate

输入端口: input_data, 类型 DataTable, 必需。

输出端口: output_data, 类型 DataTable。

参数:

group_column, 下拉框 (可选, 选"无"表示对全表聚合不分组), 默认"无"。

agg_column, 下拉框。要聚合的数值列。

agg_function, 下拉框, 选项"求和(Sum)""均值(Mean)""中位数(Median)""最大值(Max)""最小值(Min)""计数(Count)""标准差(Std)", 默认"均值"。

处理逻辑: 若 group_column 为"无", 对 agg_column 全列做聚合运算, 输出只有一行一列的 DataTable。否则按 group_column 的值分组, 每组分别对 agg_column 做聚合, 输出包含 group_column 和聚合结果两列的 DataTable。

子类: 高级数据处理

节点 83 数据透视表 DataPivot

功能: 对标 Excel 透视表功能, 按行字段和列字段双重分组, 对值字段做聚合运算后生成二维交叉表。

输入端口: input_data, 类型 DataTable, 必需。

输出端口: output_data, 类型 DataTable。

参数:

row_field, 文本输入框 (String 类型), 默认空。指定作为透视表行标签的列名。

col_field, 文本输入框 (String 类型), 默认空。指定作为透视表列标签的列名。

value_field, 文本输入框 (String 类型), 默认空。指定作为聚合值的列名。

agg_func, 下拉框 (Combo 类型), 选项"求和""均值""计数""最大值""最小值", 默认"求和"。

处理逻辑：检查三个指定字段是否在输入表中存在。遍历输入表所有行，按(row_field 值, col_field 值)组合分组收集 value_field 的数值。收集所有唯一的行值和列值并排序。构建输出 DataTable：第一列为 row_field 的行值，后续列以 col_field 唯一值为列名。对每组执行选定的聚合函数填充单元格。无数据的位置留空。

节点 84 数据画像 DataProfile

功能：自动生成数据统计摘要，对每列自动识别类型（数值/文本），统计非空数、空值率、最值、均值、标准差、中位数、唯一值数，输出统计表和文字报告。

输入端口：input_data，类型 DataTable，必需。

输出端口：output_data，类型 DataTable。output_report，类型 Text。

参数：无。

处理逻辑：遍历输入表的每一列，统计以下指标：

列名、类型（数值列数量超过非空数一半则判定为"数值"，否则"文本"）、非空数、空值数、空值率百分比。

对数值列：最小值、最大值、均值、标准差、中位数。

唯一值数量。

将统计结果装配成 DataTable（列名固定为"列名""类型""非空数""空值数""空值率%""最小值""最大值""均值""标准差""中位数""唯一值数"）。同时生成文字报告，开头显示总行数和总列数，每行显示各列的简要摘要。记录日志"数据画像完成"。

3.13 第 13 大类：图表生成

所有图表节点的基本模式一致：输入 DataTable，输出 Image。图表通过 QCustomPlot 或 Qt Charts 渲染到 QImage 上，再转为 cv::Mat 输出。

每个图表节点均有以下公共参数：

chart_width，整数输入框（Int 类型），默认 800，范围 200-4000。

chart_height，整数输入框（Int 类型），默认 600，范围 200-4000。

title，文本输入框（String 类型），默认空。

title_font_size，整数滑条（IntSlider 类型），默认 16，范围 8-48（折线图/面积图/柱状图/条形图/散点图/直方图）。热力图/雷达图/饼图/环形图/箱线图默认为 14。

background_color，颜色选取器（Color 类型），默认白色(255,255,255,255)。

show_legend，复选框（Bool 类型），默认勾选。部分图表无此参数（如饼图特殊处理）。

子类：趋势类

节点 85 折线图 LineChart

输入端口：input_data，类型 DataTable，必需。

输出端口：output_image，类型 Image。

专属参数：

x_column，文本输入框（String 类型），默认空。X 轴数据列名。

y_columns，文本输入框（String 类型），默认空。多个 Y 列用逗号分隔（如"col1,col2"）。

x_label，文本输入框（String 类型），默认空。X 轴标签。

y_label，文本输入框（String 类型），默认空。Y 轴标签。

处理逻辑：使用 QCustomPlot 创建 QCPGraph。X 轴数据：若数值型直接用，否则用文本标签+QCPAxisTickerText。Y 轴每个列一条线，使用 ChartHelper::defaultPalette()默认配色。若 show_legend 且 Y 列多于 1 个，显示图例。rescaleAxes 后渲染。

节点 86 面积图 AreaChart

端口同折线图。

专属参数同折线图，额外增加：

fill_opacity，浮点滑条，0.0 到 1.0，默认 0.3。线条下方填充区域的透明度。

stacked，复选框，默认不勾选。勾选时各系列堆叠显示。

处理逻辑：基于 QCustomPlot 的 QCPGraph，设置 Brush 填充。

子类：对比类

节点 87 柱状图 BarChart

输入端口：input_data，类型 DataTable，必需。

输出端口：output_image，类型 Image。

专属参数：

category_column，文本输入框（String 类型）。分类轴数据列。

value_columns，文本输入框（String 类型）。可多列，逗号分隔，形成分组柱状图。

mode，下拉框，选项"分组(Grouped)"堆叠(Stacked)"，默认"分组"。

bar_colors，每列对应一个颜色选取器。

x_label, y_label，文本输入框。

bar_width_ratio，浮点滑条，0.1 到 1.0，默认 0.7。柱子宽度占可用空间的比例。

处理逻辑：使用 QCustomPlot 的 QCPBars。

节点 88 条形图 HorizontalBarChart

与柱状图功能相同，但方向为水平。

处理逻辑：本质是旋转 90 度的柱状图。在 QCustomPlot 中交换 key 和 value 轴的方向实现。

子类：占比类

节点 89 饼图 PieChart

输入端口：input_data，类型 DataTable，必需。

输出端口：output_image，类型 Image。

专属参数：

label_column，文本输入框（String 类型）。标签列。

value_column，文本输入框（String 类型）。数值列。

show_percentage，复选框，默认勾选。

colors，颜色方案下拉框，选项"默认配色""科技蓝""暖色调""灰度""自定义"。选"自定义"时每个扇区单独设置颜色。

explode_index，整数输入框，默认-1。-1 表示不突出任何扇区， ≥ 0 时将对应索引的扇区拉出一段距离。

处理逻辑：使用 Qt Charts 的 QPieSeries 渲染。

节点 90 环形图 DoughnutChart

同饼图，额外参数：

hole_ratio，浮点滑条，0.1 到 0.8，默认 0.5。中间镂空的比例。

处理逻辑：Qt Charts 的 QPieSeries 设置 holeSize。

子类：分布类

节点 91 散点图 ScatterPlot

输入端口：input_data，类型 DataTable，必需。

输出端口：output_image，类型 Image。

专属参数：

x_column, y_column，文本输入框。

size_column，文本输入框（可选，选"无"则所有点相同大小）。根据列值映射点的大小。

color_column, 文本输入框（可选，选"无"则统一颜色）。根据列值映射点的颜色。

point_shape, 下拉框，选项"圆形""方形""三角形""十字""星形"，默认"圆形"。

point_size, 整数输入框，1 到 30，默认 6。当 size_column 为"无"时使用固定大小。

default_color, 颜色选取器，默认蓝色(78,121,167)。当 color_column 为"无"时使用。

x_label, y_label, 文本输入框。

处理逻辑：使用 QCustomPlot 的 QCPGraph（散点样式）。

节点 92 直方图 Histogram

输入端口：input_data，可接受类型集合为{DataTable, Image}。

输出端口：output_image，类型 Image。

专属参数：

bin_count, 整数滑条（IntSlider 类型），2 到 200，默认 20。

bar_color, 颜色选取器（Color 类型），默认蓝色(78,121,167,255)。

当输入为 DataTable 时：

- data_column, 文本输入框（String 类型），默认空。要绘制的数据列名。

当输入为 Image 时：

- channel, 下拉框（Combo 类型），选项"灰度/亮度""R""G""B""全部通道叠加"，默认"全部通道叠加"。

处理逻辑：

Image 输入：将图像转灰度，调用 cv::calcHist 计算直方图，用 QCPBars 绘制。

DataTable 输入：从指定列提取数值数据，自动计算 min/max 和 bin 宽度，统计每箱频率后用 QCPBars 绘制。

节点 93 箱线图 BoxPlot

输入端口：input_data，类型 DataTable，必需。

输出端口：output_image，类型 Image。

专属参数：

category_column, 文本输入框（String 类型，可选，选"无"则整列为一组）。

value_column, 文本输入框。

show_outliers, 复选框，默认勾选。

box_color, 颜色选取器，默认蓝色。

处理逻辑：使用 QCustomPlot 的 QCPStatisticalBox。

子类：高级图表

节点 94 热力图 Heatmap

输入端口：input_data，类型 DataTable，必需。

输出端口：output_image，类型 Image。

专属参数：

row_column，文本输入框。行标签列。

col_column，文本输入框。列标签列。

value_column，文本输入框。数值列。

color_scheme，下拉框，选项"Viridis""Plasma""Inferno""Magma""Hot""Cool""GrayScale"，默认"Viridis"。

show_values，复选框，默认不勾选。勾选时在每格中显示数值。

x_label, y_label，文本输入框。

处理逻辑：将 DataTable 转为二维矩阵（row_column 的唯一值作为行，col_column 的唯一值作为列，value_column 为值），使用 QCustomPlot 的 QCPCColorMap 渲染。色带使用 QCPCColorGradient。

节点 95 雷达图 RadarChart

输入端口：input_data，类型 DataTable，必需。

输出端口：output_image，类型 Image。

专属参数：

dimension_columns，文本输入框。逗号分隔的维度列名列表。

series_column，文本输入框（可选）。若选中，则按此列分组绘制多条雷达线。

fill_opacity，浮点滑条，0.0 到 1.0，默认 0.2。

line_colors，每个系列一个颜色选取器。

处理逻辑：使用 Qt Charts 的 QPolarChart，将各维度等角分布在极坐标上，连线形成雷达图。

3.14 第 14 大类：风格化与特效

子类：艺术效果

节点 96 油画效果 OilPaintEffect

输入端口：input_image，类型 Image，必需。

输出端口: output_image, 类型 Image。

参数:

brush_size, 整数滑条, 1 到 20, 默认 7。笔触半径。

smooth_level, 整数滑条, 1 到 256, 默认 20。颜色量化级数。

处理逻辑: 对每个像素, 在 brush_size 邻域内统计各颜色量化级别的频率, 取频率最高的级别的平均颜色作为输出。

节点 97 素描效果 SketchEffect

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

line_thickness, 整数滑条, 1 到 5, 默认 1。

contrast, 浮点滑条, 0.5 到 3.0, 默认 1.5。

处理逻辑: 转灰度->反色->高斯模糊->与原灰度图做颜色减淡混合 (Color Dodge: result = gray / (255 - blurred), clamp 到 255)。最后做对比度调整。

节点 98 像素化/马赛克 Pixelate

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

block_size, 整数滑条, 2 到 100, 默认 10。

处理逻辑: 先将图像缩小为 (width/block_size) x (height/block_size), 再用最近邻插值放大回原尺寸。

子类: 色调效果

节点 99 渐变映射 GradientMap

功能: 将图像转为灰度后, 用渐变色带重新上色。

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

color_stops, 渐变编辑器控件。用户添加多个颜色停靠点 (位置 0.0-1.0+颜色), 至少两个。默认从黑(0.0)到白(1.0)。

处理逻辑: 将图像转灰度, 对每个像素的灰度值 (归一化到 0-1) 在 color_stops 之间线性

插值得到映射颜色。使用 LUT 优化，预先计算 256 项查找表。

节点 100 复古/老照片 VintageEffect

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

tone_color, 颜色选取器, 默认暖黄(210,180,120)。色调叠加颜色。

tone_strength, 浮点滑条, 0.0 到 1.0, 默认 0.4。

vignette_strength, 浮点滑条, 0.0 到 1.0, 默认 0.6。暗角强度。

noise_amount, 浮点滑条, 0.0 到 1.0, 默认 0.1。噪点叠加量。

desaturation, 浮点滑条, 0.0 到 1.0, 默认 0.3。去色程度。

处理逻辑: 先降低饱和度->叠加色调 (tone_color 按 tone_strength 与原图混合) ->添加暗角 (根据像素到中心的距离生成径向渐暗蒙版) ->添加高斯噪点。

子类: 纹理效果

节点 101 浮雕效果 EmbossEffect

功能: 将图像转为浮雕纹理效果, 使用方向性卷积核。

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

direction, 下拉框 (Combo 类型), 选项"左上""右上""左下""右下", 默认"左上"。控制浮雕的光源方向。

strength, 整数滑条 (IntSlider 类型), 32 到 255, 默认 128。浮雕强度 (亮度基值偏移量)。

处理逻辑: 输入图像转灰度后, 根据 direction 参数构建 3x3 浮点卷积核 (不同方向对应不同的矩阵系数模式), 使用 cv::filter2D(CV_32F)卷积。结果加 strength 偏移确保亮度居中, convertTo 转为 8 位, 再转灰度回三通道 BGR 输出。

3.15 第 15 大类: 版面排版与装饰

子类: 排版

节点 102 多图排版 MultiImageLayout

功能: 将多张图像按网格布局排列到一张画布上, 支持 1x1 到 4x4 的所有排列、非方阵、

镂空格和格子合并。

输入端口：动态端口。初始有两个输入端口 `input_0` 和 `input_1`（类型 `Image`，可选）。端口数量随网格配置的启用格子数动态变化。

输出端口： `output_image`，类型 `Image`。

参数：

`rows`，整数下拉框，选项 1 到 4，默认 2。

`cols`，整数下拉框，选项 1 到 4，默认 2。

`grid_editor`，网格编辑器控件。呈现 `rowsxcols` 的网格，每个格子可以单击切换"启用/禁用"，可以拖拽选中相邻格子后点击"合并"按钮将它们合并为一个大格。

`size_mode`，下拉框，选项"像素""物理尺寸(mm)+DPI""物理尺寸(inch)+DPI"，默认"像素"。

`canvas_width`，整数输入框，默认 1600。

`canvas_height`，整数输入框，默认 1200。

`dpi`，整数输入框，默认 300。

`gap`，整数输入框，0 到 100，默认 10。格间距像素数。

`outer_margin`，整数输入框，0 到 100，默认 20。画布外边距。

`background_color`，颜色选取器，默认白色。

`resize_mode`，下拉框，选项"等比缩放适配""拉伸填满""不缩放居中"，默认"等比缩放适配"。

`show_labels`，复选框，默认勾选。勾选时在每格左上角自动绘制编号标注。

`label_style`，下拉框，选项 "(a)(b)(c)..." "(A)(B)(C)..." "(1)(2)(3)..." "(i)(ii)(iii)..."，默认 "(a)(b)(c)..."。

`label_font_size`，整数输入框，默认 14。

`label_color`，颜色选取器，默认黑色。

动态端口规则：按从左到右、从上到下的顺序为每个启用的独立格子或合并组分配一个输入端口，命名为 `input_0`、`input_1`……，与画布上的格位一一对应。

处理逻辑：计算每个格子的像素区域（考虑 `gap`、`outer_margin`、合并后的大格占据多行多列的区域），将输入图片按 `resize_mode` 放置到对应区域中。

节点 103 画布扩展 `CanvasExtend`

输入端口： `input_image`，类型 `Image`，必需。

输出端口: output_image, 类型 Image。

参数:

top, 整数输入框, 0 到 2000, 默认 0。上方扩展像素数。

bottom, 整数输入框, 默认 0。

left, 整数输入框, 默认 0。

right, 整数输入框, 默认 0。

fill_color, 颜色选取器, 默认白色。

处理逻辑: cv::copyMakeBorder。

子类: 装饰

节点 104 边框与装饰 BorderDecoration

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

border_width, 整数输入框, 0 到 100, 默认 5。

border_color, 颜色选取器, 默认黑色。

corner_radius, 整数输入框, 0 到 100, 默认 0。0 为直角, 大于 0 为圆角。

shadow_enabled, 复选框, 默认不勾选。

shadow_color, 颜色选取器, 默认半透明黑色。

shadow_offset_x, shadow_offset_y, 整数输入框, 默认 5。

shadow_blur_radius, 整数输入框, 0 到 30, 默认 10。

处理逻辑: 创建一张比原图大 (大出 $\text{border_width} \times 2 + \text{shadow}$ 区域) 的画布。若 shadow 启用, 先在偏移位置绘制模糊的阴影矩形。然后绘制边框矩形, 最后在边框内区域绘制原图。

节点 105 阴影效果 DropShadow

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

shadow_color, 颜色选取器, 默认(0,0,0,128)。

offset_x, offset_y, 整数输入框, 默认 5。

blur_radius, 整数输入框, 0 到 50, 默认 10。

background_color, 颜色选取器, 默认白色。

处理逻辑：创建一张大于原图的画布用 `background_color` 填充。在偏移位置绘制原图的轮廓阴影（将原图 Alpha 通道或原图本身作为形状，填充 `shadow_color` 后做高斯模糊），然后在正常位置绘制原图。

子类：对比

节点 106 图像差异对比 ImageDiff

功能：可视化两张图像的差异，支持多种对比模式，量化差异比例，科研对比常用。

输入端口：input_a，类型 Image，必需。input_b，类型 Image，必需。

输出端口：output_image，类型 Image。diff_ratio，类型 Numeric（差异像素比例）。

参数：

mode，下拉框（Combo 类型），选项"差值图""并排对比""闪烁对比""差异蒙版"，默认"差值图"。

threshold，整数滑条（IntSlider 类型），1 到 128，默认 30。差异检测的阈值。

处理逻辑：首先统一两图尺寸和通道数（尺寸不同则将 b 缩放到 a 的尺寸；通道数不同统一为：灰度转 BGR，BGRA 转 BGR）。四种模式：

差值图：cv::absdiff 后用 3 倍乘放大差异，使差异区域更加明显。

并排对比：创建宽度为 $w*2+10$ 的画布，左半放 a，右半放 b，中间有灰色分隔带。

闪烁对比：cv::vconcat 上下拼接 a 和 b，中间画绿色分隔线。

差异蒙版：cv::absdiff 后转灰度，以 threshold 做 cv::threshold 二值化，输出白色为差异区域。

所有模式均计算差异比例：灰度差值图的 `countNonZero(diff>threshold)/total` 像素数，通过 diff_ratio 端口输出。

子类：显示

节点 107 图像预览显示 ImagePreview

输入端口：input_image，类型 Image，必需。

输出端口：无。

参数：

window_title，文本输入框，默认"预览"。

处理逻辑：仅在 GUI 模式下有效。执行时将图像显示在底部预览面板中。命令行模式下此节点不做任何操作。

3.16 第 16 大类：图形绘制与流程图

子类：画布

节点 108 空白画布创建 BlankCanvas

功能：与纯色图生成类似，但定位为绘制类工作流的起点。

输入端口：无。

输出端口：output_image，类型 Image。

参数：

size_mode，同节点 03。

width, height, dpi，同节点 03。

background_color，颜色选取器，默认白色。

transparent，复选框，默认不勾选。勾选时忽略 background_color，输出全透明 BGRA 图像。

处理逻辑：同纯色图生成，transparent 勾选时 Alpha 通道全为 0。

子类：基础形状

以下五种形状绘制节点结构类似：输入一张图像作为底图，在上面绘制指定形状后输出。

节点 109 矩形绘制 DrawRectangle

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

x, y，整数输入框。左上角坐标。

rect_width, rect_height，整数输入框。

fill_color，颜色选取器，默认半透明蓝色(100,150,255,180)。

border_color，颜色选取器，默认黑色。

border_width，整数输入框，0 到 20，默认 2。0 表示无边框。

处理逻辑：使用 QPainter 在 QImage 上绘制填充矩形和边框。

节点 110 圆角矩形绘制 DrawRoundedRect

参数同矩形，额外增加 corner_radius，整数输入框，默认 10。

节点 111 椭圆绘制 DrawEllipse

参数：center_x, center_y（椭圆中心），radius_x, radius_y（水平和垂直半径），fill_color，

border_color, border_width。

节点 112 菱形绘制 DrawDiamond

参数: center_x, center_y, half_width, half_height, fill_color, border_color, border_width。

处理逻辑: 用 QPainterPath 构建菱形路径 (四个顶点: 上、右、下、左), 填充和描边。

节点 113 平行四边形绘制 DrawParallelogram

参数: x, y(左上角), width, height, skew(倾斜量像素), fill_color, border_color, border_width。

处理逻辑: 四个顶点为(x+skew, y), (x+width, y), (x+width-skew, y+height), (x, y+height)。

子类: 连接件

节点 114 直线/箭头绘制 DrawArrow

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

start_x, start_y, 整数输入框。起点坐标。

end_x, end_y, 整数输入框。终点坐标。

line_color, 颜色选取器, 默认黑色。

line_width, 整数输入框, 1 到 10, 默认 2。

arrow_start, 复选框, 默认不勾选。起点箭头。

arrow_end, 复选框, 默认勾选。终点箭头。

arrow_size, 整数输入框, 5 到 30, 默认 12。箭头三角形大小。

处理逻辑: QPainter 画线, 箭头用三角形填充绘制。

节点 115 折线连接绘制 DrawPolyline

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

points, 坐标列表编辑器, 用户添加多个(x,y)点。至少两个点。

line_color, line_width, 同上。

arrow_end, 复选框, 默认勾选。

arrow_size, 整数输入框, 默认 12。

line_style, 下拉框, 选项"实线""虚线""点线", 默认"实线"。

处理逻辑: QPainter 按点序逐段连线, 最后一段末端绘制箭头。

节点 116 曲线连接绘制 DrawCurve

参数: start_x, start_y, end_x, end_y, control1_x, control1_y, control2_x, control2_y (两个贝塞尔控制点), line_color, line_width, arrow_end, arrow_size。

处理逻辑: QPainterPath 的 cubicTo 绘制三次贝塞尔曲线。

子类: 流程图

节点 117 流程图元素 FlowchartElement

功能: 在底图上绘制一个流程图元素 (形状+内嵌文字)。

输入端口: input_image, 类型 Image, 必需。

输出端口: output_image, 类型 Image。

参数:

shape, 下拉框, 选项"圆角矩形""矩形""菱形""平行四边形""椭圆""圆形""六边形""梯形", 默认"圆角矩形"。

center_x, center_y, 整数输入框。元素中心坐标。

elem_width, elem_height, 整数输入框。

text, 文本输入框。

text_font_size, 整数输入框, 默认 14。

text_color, 颜色选取器, 默认黑色。

fill_color, 颜色选取器, 默认浅蓝(200,220,255)。

border_color, 颜色选取器, 默认黑色。

border_width, 整数输入框, 默认 2。

处理逻辑: 根据 shape 类型使用 QPainterPath 绘制对应形状, 填充和描边, 然后在形状中心区域绘制文字。多行文字自动换行适配形状内部区域。

节点 118 流程图自动布局 FlowchartAutoLayout

功能: 输入流程描述信息, 自动布局并输出完整的流程图图像。

输入端口: input_data, 类型 DataTable, 可选。若接入 DataTable 则使用"数据驱动"模式。

输出端口: output_image, 类型 Image。

参数:

steps, 流程图步骤列表编辑器(FlowchartSteps 类型), 默认空列表。每个步骤包含 step_id (文本)、step_name (文本)、logic_type (下拉: 开始/过程操作/判断分支/子流程/输入输出/结束/等待延迟/并行开始/并行结束)、shape (自定义或自动推荐)、connections (多选目标)、

branch_true 和 branch_false（仅判断类型）、fill_color。

color_scheme，下拉框（Combo 类型），选项"科技蓝""学术灰""清新绿""暖色调""商务深蓝"，默认"科技蓝"。控制各步骤形状的自动填色。

direction，下拉框（Combo 类型），选项"自上而下""自左而右"，默认"自上而下"。

font_size，整数滑条（IntSlider 类型），8 到 36，默认 13。

canvas_width，整数输入框（Int 类型），400 到 8000，默认 1600。

canvas_height，整数输入框（Int 类型），300 到 8000，默认 1200。

node_width，整数输入框（Int 类型），60 到 400，默认 140。

node_height，整数输入框（Int 类型），30 到 200，默认 50。

h_gap，整数输入框（Int 类型），20 到 300，默认 80。水平间距。

v_gap，整数输入框（Int 类型），20 到 300，默认 60。垂直间距。

处理逻辑：

第一步，构建有向图。从 steps 解析每个步骤的 connections 和判断分支（branch_true/branch_false）构建邻接表和入度数组。

第二步，Kahn 拓扑分层。入度为 0 的节点在第 0 层。BFS 逐层出队，后续节点的层级 = max(前驱层级) + 1。节点按层分组。

第三步，剩余节点（仍含入度的回环节点）放到最大层 + 1。

第四步，坐标分配。根据 direction 参数决定水平或垂直排列。同层内节点垂直（自上而下模式）或水平（自左而右模式）均匀分布，使用 node_width/node_height 和 h_gap/v_gap 控制间距。

第五步，绘制。在 QImage 上用 QPainter 先绘制连线（正交折线：从源节点出口到目标节点入口，判断的"否"分支从右侧路径绕行），标注分支标签，绘制箭头。然后逐个绘制形状（根据 logicType 自动选取形状：开始/结束→椭圆，判断/分支→菱形，输入/输出→平行四边形，其他→圆角矩形）。配色方案控制填充色。

子类：装饰

节点 119 图标/符号嵌入 IconEmbed

功能：在图像上嵌入一个图标。内置一套常见科研流程图标，也支持用户导入自定义图标文件。

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数:

`source`, 下拉框, 选项"内置图标""自定义文件", 默认"内置图标"。

`builtin_icon`, 下拉框 (仅 `source` 为内置时显示), 列出所有内置图标名称。

`custom_file_path`, 文件选择器 (仅 `source` 为自定义时显示), 支持 PNG/SVG 格式。

`position_x`, `position_y`, 整数输入框。图标中心坐标。

`icon_size`, 整数输入框, 10 到 500, 默认 40。图标的目标宽度像素。

`opacity`, 浮点滑条, 0.0 到 1.0, 默认 1.0。

处理逻辑: 加载图标图像, 缩放到 `icon_size`, 在 `input_image` 的指定位置按 `opacity` 叠加。

3.17 第 17 大类: 实用工具与高级

子类: 信息提取

节点 120 图像信息提取 `ImageInfo`

输入端口: `input_image`, 类型 `Image`, 必需。

输出端口: `output_width`, 类型 `Numeric`。`output_height`, 类型 `Numeric`。`output_channels`, 类型 `Numeric`。`output_summary`, 类型 `Text`。

参数: 无。

处理逻辑: 读取图像的宽(`cols`)、高(`rows`)、通道数(`channels()`), 组装摘要文字(如"1920x1080, 3 通道, 8 位")分别输出。深度标识为"8 位"/"16 位"/"其他"。

节点 121 图像直方图统计 `ImageHistogramStats`

输入端口: `input_image`, 类型 `Image`, 必需。

输出端口: `output_data`, 类型 `DataTable`。

参数: 无。

处理逻辑: 对图像各通道分别计算均值、标准差、最小值、最大值、中位数, 输出一个 `DataTable`, 列为"通道名""均值""标准差""最小值""最大值""中位数"。

节点 122 数据品质评分 `DataQualityScore`

输入端口: `input_data`, 类型 `DataTable`, 必需。

输出端口: `output_score`, 类型 `Numeric`。`output_report`, 类型 `Text`。

参数:

`check_completeness`, 复选框, 默认勾选。检查缺失值比例。

`check_consistency`, 复选框, 默认勾选。检查数值列中是否有类型不一致的值。

check_uniqueness, 复选框, 默认勾选。检查重复行比例。

处理逻辑: 根据勾选的检查维度计算各项得分(0-100), 加权平均得到总分。生成文字报告说明各项指标的具体情况。

节点 123 表格信息提取 TableInfo

功能: 提取 DataTable 的基本统计信息, 包括行列数、空值率、数值列数量、列名列表等。

输入端口: input_data, 类型 DataTable, 必需。

输出端口: output_rows, 类型 Numeric。output_cols, 类型 Numeric。output_empty_rate, 类型 Numeric(空值率百分比)。output_numeric_cols, 类型 Numeric(数值列数)。output_summary, 类型 Text(摘要文字如"10 行 x5 列, 3 个数值列, 空值率 12.5%")。output_columns, 类型 Text(逗号分隔的列名列表)。

参数: 无。

处理逻辑: 遍历整个表统计行数、列数、总单元格数和空单元格数。对每列检测是否含数值数据(至少有一个可转 double 的值), 统计数值列数。计算空值率=100*空单元格/总单元格。汇总摘要文字和列名列表输出。

节点 124 文本信息提取 TextInfo

功能: 提取 Text 数据的统计信息, 包括字符数、行数、词数、UTF-8 字节数。

输入端口: input_text, 类型 Text, 必需。

输出端口: output_chars, 类型 Numeric(字符数)。output_lines, 类型 Numeric(行数)。output_words, 类型 Numeric(词数, 按字母/数字边界分词)。output_bytes, 类型 Numeric(UTF-8 字节数)。output_summary, 类型 Text(摘要如"256 字符, 42 词, 5 行, 260 字节(UTF-8)")。

参数: 无。

处理逻辑: 逐字符统计 len、换行符数、字母数字边界分词数, 计算 UTF-8 编码后的字节大小。

子类: 流程控制

节点 125 条件路由 ConditionalRouter

功能: 根据条件值决定 Image 数据走哪条分支。

输入端口: input_data, 类型 Image, 必需。input_condition, 类型 Numeric, 必需。

输出端口: output_true, 类型 Image。output_false, 类型 Image。

参数:

operator, 下拉框(Combo 类型), 选项"大于""大于等于""等于""小于""小于等于""不等

于", 默认"大于"。

threshold, 浮点输入框 (Double 类型), 默认 0.0, 范围-1e9 到 1e9。

处理逻辑: 将 **input_condition** 的值与 **threshold** 按 **operator** 比较 (浮点比较使用 1e-9 容差)。若为真, 将 **input_data** 传递到 **output_true** 端口, **output_false** 输出空 NodeData。若为假, 反之。下游节点如果连接了空输出的端口, 不会被执行。

节点 126 数据条件路由 DataConditionalRouter

功能: 与 ConditionalRouter 类似, 但输入输出类型为 DataTable 而非 Image。

输入端口: **input_data**, 类型 DataTable, 必需。**input_condition**, 类型 Numeric, 必需。

输出端口: **output_true**, 类型 DataTable。**output_false**, 类型 DataTable。

参数:

operator, 下拉框 (Combo 类型), 选项"大于""大于等于""等于""小于""小于等于""不等于""不为零", 默认"大于"。

threshold, 浮点输入框 (Double 类型), 默认 0.0。

处理逻辑: 判断逻辑同 ConditionalRouter。若真则将 DataTable 通过 **output_true** 输出, **output_false** 输出空; 若假则反之。额外支持"不为零"运算符。

节点 127 列表遍历器 ListIterator

功能: 对 ImageList 中的每个元素执行下游子图, 收集所有迭代结果为 ImageList。

输入端口: **input_list**, 类型 ImageList, 必需。**input_iteration_result**, 类型 Image, 可选。该端口接收迭代子图末端的输出。

输出端口: **output_current**, 类型 Image (当前迭代的图像)。**output_index**, 类型 Numeric (当前索引, 从 0 开始)。**output_results**, 类型 ImageList (收集所有迭代结果)。

参数: 无。

处理逻辑详述: 这个节点与执行引擎深度耦合。

执行引擎遇到 ListIterator 节点时, 进入迭代模式。

第一步, process()首次调用: 缓存 **input_list** 到成员变量, 输出第一个元素到 **output_current**, 输出索引 0 到 **output_index**, 记录总数量日志。

第二步, 引擎检测到这是一个迭代器节点 (**isIterator()**返回 true), 开始循环调用 **advanceIteration()**:

- 索引前进 1; 若未越界, 收集上一轮的 **input_iteration_result**, 清空输出, 设置当前元素和索引。

- 若已越界（所有元素处理完），收集最后一轮结果，清空输出，将所有收集到的结果组装为 ImageList 输出到 output_results。

第三步，引擎在每次 advanceIteration()后重新执行迭代子图（从 output_current 出发到 input_iteration_result 路径上的所有节点），直到 hasMore()返回 false。

子类：外部接口

节点 128 外部脚本执行 ExternalScript

功能：调用外部可执行文件处理图像。

输入端口：input_image，类型 Image，必需。

输出端口：output_image，类型 Image。

参数：

script_path，文件选择器。可执行文件路径。

args_template，文本输入框，默认"{input} {output}"。其中{input}和{output}为占位符，运行时替换为临时输入和输出文件路径。

temp_dir，文件夹选择器，默认使用系统临时目录。

timeout，整数输入框，1 到 600，默认 30。超时秒数。

处理逻辑：将输入图像写入 temp_dir 下的临时文件，将 args_template 中的占位符替换为实际路径，用 QProcess 启动外部可执行文件，等待完成或超时。完成后读取输出临时文件。超时则报错。进程返回非零退出码则报错并附带 stderr 输出。

节点 129 外部 API 调用 ExternalAPICall

功能：接口预留，暂不实现具体调用逻辑。

输入端口：input_text，类型 Text，可选。作为 API 请求体。input_image，类型 Image，可选。作为图像上传。

输出端口：output_image，类型 Image。output_text，类型 Text。

参数：

api_url，文本输入框。

method，下拉框，选项"GET""POST"，默认"POST"。

headers，键值对列表编辑器。

timeout，整数输入框，默认 30。

处理逻辑：当前版本中，执行此节点时弹出提示"外部 API 调用功能尚未实现，请等待后续版本"。不执行任何网络请求。

3.18 第 18 大类：几何图案生成

子类：通用图案

节点 130 通用几何图案 GenericPattern

功能：程序化生成通用几何装饰图案。

输入端口：无。

输出端口：output_image，类型 Image。

参数（通过 PatternBase 基类的 addCommonParams 统一管理）：

width，整数输入框（Int 类型），100 到 10000，默认 800。

height，整数输入框（Int 类型），100 到 10000，默认 600。

primary_color，颜色选取器（Color 类型），默认(80,150,230,255)。

secondary_color，颜色选取器（Color 类型），默认(40,80,120,255)。

background_color，颜色选取器（Color 类型），默认透明(0,0,0,0)。

density，浮点滑条（DoubleSlider 类型），0.1 到 2.0，默认 1.0。控制图案的疏密程度。

line_width，整数滑条（IntSlider 类型），1 到 5，默认 1。线条像素宽度。

seed，整数输入框（Int 类型），0 到 999999，默认 0。0 表示使用随机种子，部分图案含随机元素。

pattern_type，下拉框（Combo 类型），选项"六边形蜂窝网格""同心圆放射线""网格点阵""波浪线群组""三角形铺排""菱形铺排"，默认"六边形蜂窝网格"。

处理逻辑：使用 QPainter 在 QImage 上程序化绘制。

六边形蜂窝网格：按 density 计算六边形半径 $r=30/\text{density}$ ， $dx=r*1.5$ ， $dy=r*\sqrt{3}$ ，逐行逐列绘制正六边形多边形。

同心圆放射线：间隔 $40/\text{density}$ 画同心圆， $12*\text{density}$ 条等角放射线从中心发出。

网格点阵：间隔 $20/\text{density}$ 排列圆点，半径 2 像素，实心填充。

波浪线群组：间隔 $25/\text{density}$ 画水平正弦波曲线，振幅 10 像素。

三角形铺排：等边三角形逐行铺排。

菱形铺排：菱形网格铺排。

子类：计算机科学

节点 131 计算机科学图案 CSPattern

功能：生成计算机科学相关装饰图案。

输入端口：无。输出端口：output_image，类型 Image。

参数：继承 PatternBase 的公共参数（width、height、primary_color、secondary_color、background_color、density、line_width、seed），外加 pattern_type 下拉框（Combo 类型），选项"电路板纹路""二进制数据流""网络拓扑""像素矩阵""代码块装饰"，默认"电路板纹路"。

处理逻辑：

电路板纹路：随机生成正交走线（水平或垂直线段 20-100 像素长），在端点画焊盘圆点（半径 3 像素）。数量约 $200 * \text{density}$ 。

二进制数据流：Courier 字体等宽随机排列 0 和 1 字符，行高 $14 * \text{density}$ 。

网络拓扑：随机散布 $30 * \text{density}$ 个节点，距离小于 $120 / \text{density}$ 的节点间连线，节点为实心圆（半径 4）。

像素矩阵：按 $8 / \text{density}$ 像素块绘制随机透明度（基于主色）的正方形。

代码块装饰：不同长度（30-180 像素）和缩进（0-60 像素）的水平色块模拟代码行，交替使用主色和辅色，透明度 100-200 随机。

子类：化学

节点 132 化学图案 ChemistryPattern

功能：生成化学领域装饰图案。

参数：同 CSPattern，pattern_type 选项"分子骨架线框""苯环组合""元素周期表片段""化学键图案""晶体结构"，默认"分子骨架线框"。

处理逻辑：

苯环组合：六边形+内接圆，按六边形蜂窝排列。

分子骨架线框：随机起点生成 3-7 键的碳链骨架，每步随机转角，画线和端点圆点。

其他类型（元素周期表片段、化学键图案、晶体结构）：简化实现为网格+元素符号文字装饰（H、He、Li、Be、B、C、N、O、F、Ne、Na、Mg 等）。

子类：生命科学

节点 133 生命科学图案

功能：生成生命科学领域装饰图案。

参数：同 CSPattern，pattern_type 选项"DNA 双螺旋""磷脂双分子层""蛋白质二级结构""神经元连接""细胞分裂"，默认"DNA 双螺旋"。

处理逻辑：

DNA 双螺旋：两条相位偏移 π 的正弦曲线（振幅 $40 \times \text{density}$ ，频率 $0.02 \times \text{density}$ ），周期性地画横向碱基连线（间隔 $15/\text{density}$ ）。主色画链 1，辅色画链 2，辅色细线画碱基。

其他类型（磷脂双分子层、蛋白质二级结构、神经元连接、细胞分裂）：简化实现为生物形态装饰圆和连线网格。

子类：社会科学

节点 134 社会科学图案 SocialSciPattern

功能：生成社会科学领域装饰图案。

参数：同 CSPattern，pattern_type 选项"社会网络图""问卷网格""世界地图轮廓""统计曲线""人群图标阵列"，默认"社会网络图"。

处理逻辑：

社会网络图： $40 \times \text{density}$ 个随机节点，距离 $< 100/\text{density}$ 连线，节点大小 3-9 随机。

统计曲线：绘制正态分布曲线（以 $w/2$ 为中心， $\sigma = w/6/\text{density}$ ），带水平基线。

人群图标阵列： $30/\text{density}$ 间距排列简笔画人形（头圆+身体线+手臂线+双腿线）。

其他类型（问卷网格、世界地图轮廓）：简化网格+圆点标记。

子类：自定义素材

节点 135 自定义素材导入 CustomAssetImport

功能：用户导入自己的图标或装饰素材图片，输出为 Image。

输入端口：无。

输出端口：output_image，类型 Image。

参数：

file_path，文件选择器（FilePath 类型），默认空。过滤器为"图片文件 (*.png *.jpg *.bmp *.webp)"。

处理逻辑：使用 `cv::imread(IMREAD_UNCHANGED)` 读取文件。若路径为空报错"未指定素材文件路径"。若文件不存在报错"文件不存在：{路径}"。若读取失败报错"无法读取文件：{路径}"。16 位深度自动转为 8 位。

第四章 图形界面布局设计

4.1 整体布局

主窗口使用 QMainWindow，分为以下区域：

顶部为工具栏（QToolBar），固定显示在窗口顶端。

左侧为节点工具箱面板（QDockWidget，可折叠但不可浮动），默认宽度 220 像素。

中央为画布编辑区（QGraphicsView），占据主窗口的中心区域，是面积最大的区域。

右侧为属性面板（QDockWidget，可折叠），默认宽度 320 像素。当没有选中节点、或选中的节点参数简单到使用悬浮面板即可完成编辑的情况下，右侧属性面板自动收起，不占用画布空间。

底部为预览与日志面板（QDockWidget，可折叠），默认高度 200 像素。左半部分为预览区，右半部分为日志区。

4.2 工具栏

工具栏从左到右依次排列以下按钮和控件：

"新建工作流"按钮（图标为空白文档），点击后清空画布上所有节点和连线，若当前工作流有未保存修改则先弹出确认对话框。

"打开工作流"按钮（图标为文件夹），弹出文件对话框选择.flow.json 工作流文件加载。

"保存工作流"按钮（图标为软盘），若当前工作流已有文件路径则直接覆盖保存，否则弹出另存为对话框。

"另存为"按钮，始终弹出文件保存对话框。

"执行全部"按钮，旁边带一个下拉箭头，点击下拉箭头展开菜单显示"执行全部""执行到选中节点""从选中节点执行到末端"三个选项。直接点击按钮本身等同于"执行全部"。

"停止执行"按钮，执行过程中可用，点击后中断当前执行。

"撤销"按钮（图标为左弯箭头），快捷键 Ctrl+Z。

"重做"按钮（图标为右弯箭头），快捷键 Ctrl+Shift+Z。

"自动布局"按钮，点击后对画布上所有节点按拓扑排序结果自动重新排列位置，使工作流从左到右或从上到下整齐排布。

"缩放适配"按钮，点击后画布自动缩放和平移，使所有节点完整显示在视口中。

"缩放比例"显示标签，显示当前画布缩放百分比（如"100%"），旁边两个小按钮"+"和"-"分别放大和缩小 10%。

4.3 节点工具箱

节点工具箱面板顶部有一个搜索输入框，用户输入文字后，工具箱中只显示名称包含该文字的节点（模糊匹配，不区分大小写，支持拼音首字母匹配）。搜索框下方有"最近使用"和"收藏夹"两个折叠组，"最近使用"自动记录用户最近添加过的 10 个节点类型，"收藏夹"允许用户右键点击节点类型选择"加入收藏"。

搜索框和快捷组下方是按 18 个大类组织的树形列表。每个大类是一个可折叠的一级节点，展开后显示子类作为二级节点，再展开显示具体节点名称作为三级叶子节点。默认状态下所有大类折叠。

大类列表如下：

输入输出/图像（文件读取、图像生成、文件导出、批量处理）

输入输出/数据（数据读取、数据导出）

输入输出/文本（文本文件读取、文本文件导出、数值导出）

几何变换（基础变换、高级变换）

色彩调整（基础调整、HSL 调整、高级调整）

滤波与模糊（模糊、锐化、增强）

形态学操作（基本运算、复合运算）

边缘检测与分割（边缘检测、阈值分割、轮廓）

通道操作（拆分合并、空间转换、Alpha）

图层与合成（混合、拼接、水印）

文字与标注（文字、科研标注）

数据处理（清洗、变换、统计、高级）

图表生成（趋势类、对比类、占比类、分布类、高级）

风格化与特效（艺术效果、色调效果、纹理效果）

版面排版与装饰（排版、装饰、对比、显示）

图形绘制与流程图（画布、基础形状、连接件、流程图、装饰）

实用工具与高级（信息提取、流程控制、外部接口）

几何图案生成（通用图案、计算机科学、化学、生命科学、社会科学、自定义素材）

用户添加节点到画布的方式有两种：双击节点名称，节点出现在画布视口中心位置；或者从工具箱中拖拽节点名称到画布的指定位置释放。

每个节点名称左侧有一个小色块标识其输出端口的主数据类型颜色，帮助用户快速辨认。

4.4 画布编辑区

画布基于 QGraphicsScene 和 QGraphicsView 实现。背景为深灰色(#1e1e1e)，绘制浅灰色网格点作为对齐参考（点间距 20 像素，颜色#2a2a2a）。

画布支持以下交互操作：

鼠标滚轮缩放画布，缩放范围 25%到 400%，缩放中心为鼠标当前位置。

鼠标中键按下拖拽平移画布。也可以在空白区域按住 Alt+左键拖拽平移。

左键单击空白区域取消所有选中。

左键单击节点选中该节点（取消之前的选中），选中的节点边框变为高亮色（#4EC9B0 亮青色描边）。

左键按住 Ctrl 点击节点可多选。

左键在空白区域拖拽绘制矩形框选区域，释放后框内的所有节点被选中。

选中的节点可以左键拖拽移动位置。多选时同时移动。节点移动时默认启用网格吸附（吸附到 20 像素网格上）。

Delete 键删除所有选中的节点及其关联连线。

Ctrl+C 复制选中的节点（连同其参数配置），Ctrl+V 粘贴到画布上（偏移一定位置避免重叠）。

4.4.1 节点图形元素

每个节点在画布上渲染为一个圆角矩形卡片，视觉结构从上到下为：

标题栏：高度 24 像素，深色背景（#383838），显示节点名称（白色文字，12 号字体），右侧有一个小"眼睛"图标按钮（预览按钮）和一个小"饼图"图标按钮（打开参数编辑）。

端口区域：卡片左侧垂直排列输入端口，右侧垂直排列输出端口。每个端口渲染为一个直径 10 像素的圆形，颜色对应数据类型颜色（见第二章 2.3 节）。端口旁边用小字标注端口名称和类型缩写（如"Image""DT"表示 DataTable，"Num"表示 Numeric，"Txt"表示 Text）。端口之间垂直间距 16 像素。

卡片主体背景色为#2d2d2d，边框为#3d3d3d，圆角半径 8 像素。卡片宽度根据标题和端口标签的最长文字动态调整，最小宽度 140 像素，最大宽度 280 像素。卡片高度根据端口数量动态调整。

悬停在节点上时，边框颜色变为#555555。选中时边框颜色变为#4EC9B0，线宽从 1 变为 2。

4.4.2 连线交互

用户从一个端口小圆点上按住左键开始拖拽，出现一条临时贝塞尔曲线跟随鼠标移动，曲线颜色为源端口类型颜色的半透明版本。当鼠标移到一个兼容的目标端口上方时，目标端口高亮放大提示可以连接；若移到不兼容的端口上方，端口显示红色叉号提示不可连接。释放鼠标时，若在兼容端口上则建立连线，否则取消。

已建立的连线渲染为三次贝塞尔曲线，水平方向出线：源端口的控制点向右偏移若干像素，目标端口的控制点向左偏移若干像素，偏移量根据两端口水平距离动态计算（距离的 40%，最小 50 像素最大 150 像素），保证曲线平滑自然。连线颜色与源端口类型颜色一致。线宽 2 像素，悬停时变为 3 像素并发光。

连线选中（单击连线）后变为高亮色加粗，按 **Delete** 可删除。

当选中一个节点时，与该节点无关的所有连线透明度降低到 20%，只有与该节点直接或间接关联的连线保持原始亮度，帮助用户追踪数据流。

4.4.3 右键菜单

在画布空白区域右键弹出菜单：

- "添加节点" -> 展开子菜单，按大类/子类/节点三级结构展示所有节点

- "粘贴"（若剪贴板中有复制的节点）

- "缩放适配"

- "自动布局整理"

在节点上右键弹出菜单：

- "执行到此节点"

- "从此节点执行到末端"

- "预览输出"

- "复制"

- "删除"

- "断开所有连线"

在连线上右键弹出菜单：

- "删除连线"

4.5 属性面板

属性面板有两种工作状态：

状态一，无选中节点或选中的节点参数简单（参数总数不超过 4 个且不包含任何列表编辑器、表格编辑器、曲线编辑器、网格编辑器等复杂控件）。此时属性面板收起为一条窄边栏（宽度 30 像素），只显示一个向左展开的箭头按钮，用户也可以手动点击展开。

状态二，选中的节点参数复杂，属性面板自动展开到 320 像素宽度，显示该节点的完整参数编辑界面。

对于参数简单的节点，用户双击节点或点击节点标题栏上的"齿轮"按钮时，在节点上方弹出一个悬浮面板（QWidget，设置 Qt::Popup 窗口标志），面板背景色#383838，圆角 8 像素，带 2 像素深色阴影边框。面板内展示该节点的全部参数控件，排列方式为每个参数一行（标签在左，控件在右）。面板宽度自适应内容，最大不超过 300 像素。用户点击面板外部或按 Esc 时面板关闭。

属性面板展开时的内部结构：

顶部显示当前选中节点的名称和类型（如"高斯模糊 (GaussianBlur)"），字体 14 号加粗白色。

下方为参数列表区域，使用 QScrollArea 包裹以支持滚动。每个参数占一行或多行，左侧为参数标签（灰白色 12 号字体），右侧为对应的输入控件。

控件类型与参数类型的对应关系：

整数型参数 (Int)：QSpinBox，带上下箭头，支持手动输入。若有取值范围则设置 minimum 和 maximum。

浮点型参数 (Double)：QDoubleSpinBox，步长根据范围自动设定（范围跨度>10 时步长 1.0，>1 时步长 0.1，否则 0.01）。

整数滑条型参数 (IntSlider)：水平 QSlider+右侧 QSpinBox 联动显示。

浮点滑条型参数 (DoubleSlider)：水平 QSlider（内部映射整数范围 0-1000）+ 右侧 QDoubleSpinBox 联动。

下拉框参数 (Combo)：QComboBox。

文本输入参数 (String)：QLineEdit (单行) 或 QTextEdit (多行，标注为多行的参数使用)。

文件路径参数 (FilePath)：QLineEdit+右侧"浏览"按钮 (QPushButton)，点击弹出 QFileDialog::getOpenFileName。

保存文件路径参数 (SaveFilePath)：QLineEdit+右侧"浏览"按钮，点击弹出 QFileDialog::getSaveFileName。

文件夹路径参数 (DirPath)：QLineEdit+右侧"浏览"按钮，点击弹出

`QFileDialog::getExistingDirectory`。

颜色选取器参数 (Color): 一个方形色块按钮, 显示当前颜色, 点击弹出 `QColorDialog` (开启 Alpha 通道选择)。

复选框参数 (Bool): `QCheckBox`。

系统字体选择参数: `QFontComboBox`。

曲线编辑器参数: 嵌入一个自定义 `QWidget`, 绘制 256x256 坐标区域, 支持鼠标点击添加控制点、拖拽移动、右键删除。

表格编辑器参数: 嵌入 `QTableWidget`, 支持添加/删除行列、编辑单元格、右键菜单 (插入行、删除行、从剪贴板粘贴)。

标注列表编辑器参数: 嵌入一个自定义列表控件, 每项可展开显示子参数, 底部有"添加"按钮, 每项右侧有"删除"按钮。

网格编辑器参数 (多图排版专用): 嵌入一个自定义 `QWidget`, 绘制 `rowsxcols` 的网格, 支持单击切换格子启用/禁用、拖拽选区后点击"合并"/"拆分"按钮。合并后的格子用不同底色标识属于同一组。

渐变编辑器参数: 嵌入一个自定义 `QWidget`, 显示水平渐变条 (宽度 256 像素, 高度 24 像素), 下方有可拖拽的色标三角形, 双击色标打开颜色选择器, 右键色标删除, 单击渐变条空白处添加新色标。最少保留 2 个色标。

流程图步骤列表编辑器参数 (流程图自动布局专用): 嵌入在属性面板中, 每个步骤占据一个可折叠的条目, 展开后显示步骤名称、逻辑类型下拉框、形状下拉框、连接目标多选框、分支标签等控件。列表底部有"添加步骤"按钮。步骤条目支持鼠标拖拽排序。属性面板底部嵌入一个预览缩略图区域 (高度 200 像素), 实时渲染当前编辑状态的流程图缩略图。

凡参数有条件显示逻辑的 (如"仅当 `mode` 为 X 时显示"), 在属性面板中通过信号槽连接实现: 当控制参数的值变化时, 受控参数的可见性即时更新。条件通过 `addParam` 的 `visible_when` 约束指定 (如 `{"visible_when", "mode==指定尺寸"}`)。

4.6 预览与日志面板

底部面板水平分为左右两个区域, 中间有可拖拽的分割条。

左侧为预览区。当用户点击某个节点的"眼睛"预览按钮时, 触发从数据源到该节点的部分执行, 执行完毕后将该节点的输出数据展示在预览区中:

若输出为 `Image`, 显示为 `QLabel` 上的缩略图 (适配预览区大小), 双击可弹出一个独立的全尺寸查看窗口 (支持滚动和缩放查看)。缩略图下方显示图像基本信息 (宽 x 高, 通道数)。

若输出为 ImageList，水平排列缩略图列表（QScrollArea 内水平布局），每张缩略图下方标注索引。

若输出为 DataTable，显示为 QTableView，展示前 50 行数据。列头为列名，单元格只读。

若输出为 Numeric，用大号字体显示数值。

若输出为 Text，用 QTextEdit 只读模式显示文本。

若节点有多个输出端口，预览区顶部显示端口名称的 Tab 页，用户可切换查看各端口的输出。

右侧为日志区。使用 QTextEdit 只读模式，显示执行过程中的所有日志信息。日志条目带时间戳，按类型用不同颜色显示：

普通信息（INFO）：浅灰色文字

警告（WARNING）：黄色文字

错误（ERROR）：红色文字

成功（SUCCESS）：绿色文字

每次执行开始时在日志中插入一条分隔线和开始时间戳。执行完毕时插入汇总信息（总耗时、成功节点数、失败节点数）。

日志区右上角有一个"清除"按钮和一个"导出日志"按钮（将日志内容保存为 txt 文件）。

4.7 配色汇总

窗口背景色：#252525

面板背景色：#2b2b2b

面板标题栏背景色：#333333

画布背景色：#1e1e1e

画布网格点颜色：#2a2a2a

节点卡片背景色：#2d2d2d

节点卡片边框色：#3d3d3d

节点卡片标题栏背景色：#383838

节点选中描边色：#4EC9B0

文字主色：#e0e0e0

文字次色（标注、说明）：#a0a0a0

控件背景色（输入框、下拉框）：#3a3a3a

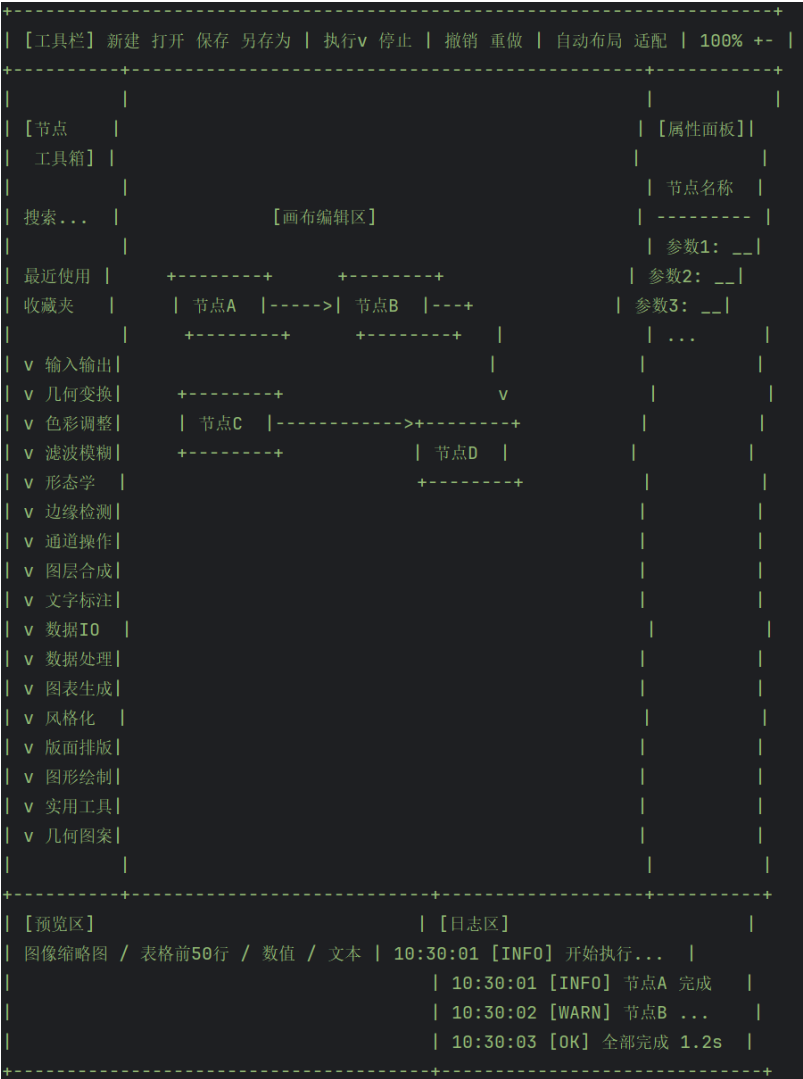
控件边框色：#4a4a4a

- 控件焦点边框色：#4EC9B0
- 按钮背景色：#404040
- 按钮悬停背景色：#505050
- 按钮按下背景色：#353535
- 滑条轨道色：#4a4a4a
- 滑条手柄色：#4EC9B0
- 分割条颜色：#3a3a3a
- 滚动条背景：#2b2b2b
- 滚动条手柄：#4a4a4a

这些颜色通过 QSS 样式表统一设定，主窗口初始化时加载一个 qss 文件。

4.8 线框图

以下用文字描述线框图的关键布局（实际开发时以此描述为准）：



第五章 workflow 文件格式设计

5.1 文件格式

workflow 文件采用 JSON 格式，文件扩展名为 .flow.json。

5.2 JSON 结构

顶层结构：

```
{
  "format_version": "1.0",
  "workflow_name": "示例 workflow",
  "created_at": "2025-01-15T10:30:00",
  "modified_at": "2025-01-15T14:20:00",
  "canvas_view": {
    "zoom": 1.0,
    "center_x": 400,
    "center_y": 300
  },
  "nodes": [ ... ],
  "connections": [ ... ],
  "execution_order_hints": [ ... ]
}
```

format_version 字段用于未来格式升级时做兼容性判断。canvas_view 记录保存时的画布视口状态，加载后恢复到相同的缩放和平移位置。

5.3 nodes 数组中每个元素的结构

```
{
  "id": "node_a1b2c3d4",
  "type": "GaussianBlur",
  "display_name": "高斯模糊",
  "position": {
    "x": 350.0,
    "y": 200.0
  },
  "params": {
```

```

        "kernel_size": 5,
        "sigma": 1.5
    }
}

```

id 为节点的唯一标识，使用 UUID 字符串（取前 8 位十六进制，格式"**n**"前缀+8 位随机数）。type 为节点的注册类型名（与 NodeFactory 中的注册键一致）。display_name 为用户可修改的显示名。position 记录节点卡片在画布场景坐标系中的位置。params 是一个键值对对象，键为参数标识名，值类型取决于参数类型：整型/浮点为 JSON number，字符串为 JSON string，布尔为 boolean，颜色为四元素 RGBA 数组如[255,128,0,255]。

复杂参数的序列化：

曲线控制点：[[0,0],[64,80],[192,200],[255,255]]

表格数据：{"columns":["A","B"],"rows":[[1,"x"],[2,"y"]]}

标注列表：[{"type":"arrow","color":[255,0,0,255],"line_width":2,...}]

网格配置：{"rows":2,"cols":3,"cells":[{"r":0,"c":0,"enabled":true,"group":0},...]}

流程图步骤：[{"step_id":"s1","step_name":"...","logic_type":"...","connections":[...]},...]

渐变色标：[{"pos":0.0,"color":[0,0,0,255]}, {"pos":1.0,"color":[255,255,255,255]}]

5.4 connections 数组中每个元素的结构

```

{
    "from_node": "node_a1b2c3d4",
    "from_port": "output_image",
    "to_node": "node_e5f6g7h8",
    "to_port": "input_image"
}

```

5.5 execution_order_hints 数组

```

"execution_order_hints": [
    {
        "context_node": "node_canvas_001",
        "context_port": "input_image",
        "ordered_sources": ["node_rect_003", "node_text_005", "node_arrow_007"]
    }
]

```

5.6 文件格式校验

加载 JSON 文件时的校验流程:

1. `format_version` 必须存在且为"1.0", 否则报错。
2. `nodes` 数组中每个节点的 `type` 必须在 `NodeFactory` 已注册, 未知 `type` 报警跳过。
3. `connections` 中引用的节点 ID 和端口名必须存在, 否则报警跳过。
4. 加载完毕后对所有连线做端口类型兼容性检查, 不兼容的连线报警删除。

5.7 完整示例

```
{
  "format_version": "1.0",
  "workflow_name": "论文四子图排版示例",
  "created_at": "2025-01-15T10:30:00",
  "modified_at": "2025-01-15T10:45:00",
  "canvas_view": {
    "zoom": 0.85,
    "center_x": 500,
    "center_y": 350
  },
  "nodes": [
    {
      "id": "n001",
      "type": "ImageFileRead",
      "display_name": "读取子图 A",
      "position": {"x": 50, "y": 50},
      "params": {"file_path": "./input/fig_a.png"}
    },
    {
      "id": "n002",
      "type": "ImageFileRead",
      "display_name": "读取子图 B",
      "position": {"x": 50, "y": 180},
      "params": {"file_path": "./input/fig_b.png"}
    }
  ],
```

```

{
  "id": "n003",
  "type": "ImageFileRead",
  "display_name": "读取子图 C",
  "position": {"x": 50, "y": 310},
  "params": {"file_path": "./input/fig_c.png"}
},
{
  "id": "n004",
  "type": "ImageFileRead",
  "display_name": "读取子图 D",
  "position": {"x": 50, "y": 440},
  "params": {"file_path": "./input/fig_d.png"}
},
{
  "id": "n005",
  "type": "Resize",
  "display_name": "统一尺寸 A",
  "position": {"x": 280, "y": 50},
  "params": {"mode": "指定尺寸", "target_width": 800, "target_height": 600,
"keep_aspect_ratio": true, "interpolation": "双线性"}
},
{
  "id": "n006",
  "type": "Resize",
  "display_name": "统一尺寸 B",
  "position": {"x": 280, "y": 180},
  "params": {"mode": "指定尺寸", "target_width": 800, "target_height": 600,
"keep_aspect_ratio": true, "interpolation": "双线性"}
},
{
  "id": "n007",
  "type": "Resize",

```



```

    "display_name": "统一尺寸 C",
    "position": {"x": 280, "y": 310},
    "params": {"mode": "指定尺寸", "target_width": 800, "target_height": 600,
"keep_aspect_ratio": true, "interpolation": "双线性"}
  },
  {
    "id": "n008",
    "type": "Resize",
    "display_name": "统一尺寸 D",
    "position": {"x": 280, "y": 440},
    "params": {"mode": "指定尺寸", "target_width": 800, "target_height": 600,
"keep_aspect_ratio": true, "interpolation": "双线性"}
  },
  {
    "id": "n009",
    "type": "MultiImageLayout",
    "display_name": "2x2 排版",
    "position": {"x": 530, "y": 200},
    "params": {
      "rows": 2, "cols": 2,
      "grid_config": {
        "rows": 2, "cols": 2,
        "cells": [
          {"r": 0, "c": 0, "enabled": true, "group": 0},
          {"r": 0, "c": 1, "enabled": true, "group": 1},
          {"r": 1, "c": 0, "enabled": true, "group": 2},
          {"r": 1, "c": 1, "enabled": true, "group": 3}
        ]
      }
    },
    "size_mode": "像素",
    "canvas_width": 1700, "canvas_height": 1300,
    "gap": 20, "outer_margin": 30,
    "background_color": [255,255,255,255],

```

```

        "resize_mode": "等比缩放适配",
        "show_labels": true, "label_style": "(a)(b)(c)...",
        "label_font_size": 16, "label_color": [0,0,0,255]
    }
},
{
    "id": "n010",
    "type": "ImageFileWrite",
    "display_name": "导出",
    "position": {"x": 780, "y": 250},
    "params": {"file_path": "./output/figure_combined.png", "format": "PNG",
"png_compression": 3}
}
],
"connections": [
    {"from_node": "n001", "from_port": "output_image", "to_node": "n005", "to_port": "input_image"},
    {"from_node": "n002", "from_port": "output_image", "to_node": "n006", "to_port": "input_image"},
    {"from_node": "n003", "from_port": "output_image", "to_node": "n007", "to_port": "input_image"},
    {"from_node": "n004", "from_port": "output_image", "to_node": "n008", "to_port": "input_image"},
    {"from_node": "n005", "from_port": "output_image", "to_node": "n009", "to_port": "input_0"},
    {"from_node": "n006", "from_port": "output_image", "to_node": "n009", "to_port": "input_1"},
    {"from_node": "n007", "from_port": "output_image", "to_node": "n009", "to_port": "input_2"},
    {"from_node": "n008", "from_port": "output_image", "to_node": "n009", "to_port": "input_3"},
    {"from_node": "n009", "from_port": "output_image", "to_node": "n010", "to_port": "input_image"}
],
"execution_order_hints": []
}

```

第六章 整体架构设计

6.1 模块划分

项目源码分为以下六个顶层模块，每个模块对应一个子目录和一组相关类。

core 模块： workflow 核心逻辑，不依赖 GUI。包含节点抽象基类 (NodeBase)、端口 (Port)、连线 (Connection)、数据载体 (NodeData、DataTable)、工作流图管理 (WorkflowGraph) 与执行引擎 (ExecutionEngine)、节点工厂 (NodeFactory)。

nodes 模块： 所有 135 种具体节点类的实现。按大类分子目录 (nodes/image_io、nodes/geometry、nodes/color、nodes/filter、nodes/morphology、nodes/edge、nodes/channel、nodes/composite、nodes/text、nodes/data_io、nodes/data_proc、nodes/chart、nodes/stylize、nodes/layout、nodes/drawing、nodes/utility、nodes/pattern、nodes/text_io)，每个子目录下每个节点一个.h/.cpp 文件。

gui 模块： Qt 图形界面相关的所有代码。包含主窗口 (MainWindow)、画布场景和视图 (NodeGraphicsScene/NodeGraphicsView)、节点图形项 (NodeGraphicsItem)、连线图形项 (ConnectionGraphicsItem)、端口图形项 (PortGraphicsItem)、属性面板 (PropertyPanel)、悬浮参数面板 (FloatingParamPanel)、预览面板 (PreviewPanel)、日志面板 (LogPanel)、节点工具箱 (NodeToolbox)、各种自定义参数控件 (CurveEditor、GridEditor、TableEditor、GradientEditor、AnnotationListEditor、FlowchartStepEditor 等)。

io 模块： 文件读写相关。包含工作流 JSON 序列化/反序列化器 (WorkflowSerializer)、图像文件读写封装 (ImageIO)、数据文件读写封装 (DataIO)。

cli 模块： 命令行模式入口。解析命令行参数，加载 JSON 工作流文件，调用 core 模块的执行引擎执行，输出结果到标准输出 (CLIRunner)。

common 模块： 公共基础设施。包含日志系统 (Logger，可同时输出到 GUI 日志面板和标准输出)、异常类定义 (NodeProcessException)、工具函数 (Utility：图像格式转换 QImage<->cv::Mat、颜色类型转换、安全四舍五入等)。

6.2 类层次结构

6.2.1 core 模块的核心类

NodeData (数据载体类)

成员：DataType 枚举标记 (Image/ImageList/DataTable/Numeric/Text)、数据指针 (std::shared_ptr<void>)

静态工厂方法：`createImage(cv::Mat)`、`createImageList(vector<cv::Mat>)`、`createDataTable(shared_ptr<DataTable>)`、`createNumeric(double)`、`createText(QString)`

方法：`dataType()`、`toImage()`/`toImageList()`/`toDataTable()`/`toNumeric()`/`toText()`、`isNull()`

`DataTable`（独立的数据结构类）

成员：`QStringList columnNames`、`QVector<QVariantList> columnsData`

方法：`rowCount()`、`colCount()`/`columnCount()`、`columnName(index)`、`columnIndex(name)`、`hasColumn(name)`、`value(row, col)`、`setValue(row, col, val)`、`addColumn(name, data)`、`removeColumn(index)`、`addRow(values)`、`removeRow(index)`、`columnNames()`

`PortDefinition`（端口定义结构体）

成员：`QString name`、`DataType dataType`、`QSet<DataType> acceptedTypes`、`bool required`

`Port`（运行时端口实例）

成员：`PortDefinition definition`、`std::shared_ptr<NodeData> data`、连线指针

`Connection`（连线）

成员：`NodePtr sourceNode`、`QString sourcePortName`、`NodePtr targetNode`、`QString targetPortName`

`NodeBase`（抽象基类）

关键成员：`m_id`、`m_type`、`m_displayName`、`m_position`、`m_inputPorts`、`m_outputPorts`、`m_params`、输出缓存

纯虚方法：`defineNode()`、`process()`

虚方法：`onParamChanged()`、`onInputConnected()`、`onInputDisconnected()`、`rebuildPorts()`、`validate()`、`isIterator()`、`advanceIteration()`、`hasMore()`、`currentIndex()`

受保护方法：`addInputPort()`、`addOutputPort()`、`addParam()`、`getInput()`、`setOutput()`、`param()`、`clearOutputs()`、`reportError()`、`reportWarning()`

`NodeFactory`（单例）

方法：`registerNode<T>(typeName, category, displayName)`、`createNode(typeName)`、`allTypes()`、`categoryOf(typeName)`

`WorkflowGraph`（工作流图管理）

成员：`QMap` 节点容器、`QVector` 连线容器、`QUndoStack` 撤销栈

方法：`addNode()`、`removeNode()`、`addConnection()`、`removeConnection()`、`changeParam()`、`topologicalSort()`、`topologicalSortPartial()`、`executeAll()`、`executePartial()`、`detectCycle()`、`nodeById()`、

clear()

ExecutionEngine（执行引擎）

职责：按拓扑顺序执行节点列表，处理 ListIterator 的迭代逻辑和 ConditionalRouter/DataConditionalRouter 的分支逻辑。try-catch 包裹每个节点的 process()，单节点失败不终止全流程。

6.2.2 nodes 模块

每种节点一个类，继承 NodeBase（形态学节点通过 MorphBase 中间基类复用核心参数，图案节点通过 PatternBase 基类复用公共参数）。在 defineNode()中声明端口和参数，在 process()中实现处理逻辑。

6.2.3 gui 模块

MainWindow：窗口布局组装、工具栏创建、快捷键绑定、信号槽连接

NodeGraphicsScene：管理画布所有图形项的添加删除选中

NodeGraphicsView：缩放平移框选交互

NodeGraphicsItem：渲染节点卡片（标题栏、背景、端口），选中高亮悬停拖拽

PortGraphicsItem：渲染端口小圆点，处理连线拖拽交互

ConnectionGraphicsItem：渲染贝塞尔曲线连线

NodeToolbox：展示节点分类树，支持搜索过滤、双击添加、拖拽添加、最近使用和收藏夹

PropertyPanel：根据选中节点动态创建参数输入控件

FloatingParamPanel：轻量级 Popup 参数编辑面板

PreviewPanel：展示节点输出预览（Image/ImageList/DataTable/Numeric/Text）

LogPanel：显示日志信息（带时间戳、按类型着色），支持清除和导出

6.2.4 io 模块

WorkflowSerializer：serialize()、deserialize()、toJson()、fromJson()

ImageIO：readImage()、writeImage()、qimageToMat()、matToQimage()

DataIO：readCSV()、writeCSV()、readJSON()、writeJSON()、readExcel()

6.2.5 cli 模块

CLIRunner：run(argc, argv)解析参数->加载工作流->执行->返回退出码

6.2.6 common 模块

Logger（单例）：info()、warning()、error()、success()

NodeProcessException（std::runtime_error 子类）：携带节点 ID 和错误描述

Utility：qImageToMat()、matToQImage()、arrayToColor()、ensureOdd()、safeRound()

6.3 数据流说明

节点间数据传递的完整流程：

1. 节点 A 的 process() 执行完毕后，调用 setOutput() 将结果写入输出端口的 data 成员。
2. 执行引擎在节点 B 执行前，遍历 B 的输入端口，通过连线从上游端口读取 data。若上游 data 为空且该端口 required，标记 B 跳过。
3. B 的 process() 中调用 getInput() 获取数据，通过连线关系找到上游端口的数据并返回。
4. B 处理完毕后写入自己的输出端口。

数据使用 std::shared_ptr 传递，下游节点共享同一份数据。需要修改输入数据的节点必须在 process() 开始时做深拷贝。

6.4 撤销重做实现

使用 Qt 的 QUndoStack 和 QUndoCommand 框架。四种命令：AddNodeCommand、RemoveNodeCommand、AddConnectionCommand、RemoveConnectionCommand、ChangeParamCommand。ChangeParamCommand 支持 mergeWith() 合并连续参数修改。撤销栈深度上限 100。

6.5 模块依赖关系

common 模块不依赖其他模块。

core 模块依赖 common 模块。

nodes 模块依赖 core 模块和 common 模块。

io 模块依赖 core 模块和 common 模块。

gui 模块依赖 core 模块、io 模块和 common 模块。

cli 模块依赖 core 模块、io 模块和 common 模块。

gui 模块和 cli 模块互不依赖。

第七章 异常处理策略

7.1 异常分类

本工具面临的异常场景分为以下六类。

第一类，文件类异常。文件路径为空、文件不存在、文件无法读取（权限问题）、文件格式无法解析、输出目录不存在且无法创建、磁盘空间不足导致写入失败。

第二类，参数类异常。参数值超出合法范围、必填参数未填写、参数之间的逻辑矛盾（如 `input_black` 大于 `input_white`）、表达式语法错误（列运算节点）。

第三类，数据类异常。输入端口未连线（`required` 端口）、上游节点未产生输出（执行失败或未执行）、输入数据类型与期望不匹配（运行时检查）、图像尺寸不一致（多输入节点）、图像通道数不匹配、`DataTable` 中列名不存在。

第四类， workflow 结构异常。连线形成环路（非 `ListIterator` 回路）、无依赖节点冲突（多个节点向同一端口写入但未定义顺序）。

第五类，资源类异常。处理超大图像导致内存分配失败（`std::bad_alloc`）、外部脚本执行超时、文件句柄耗尽。

第六类，程序内部异常。意料之外的空指针、`OpenCV` 算法失败（`cv::Exception`）、`Qt` 控件异常。

7.2 处理策略

所有异常的处理遵循一个总原则：提前拦截，不崩溃，给用户清晰的反馈。

对于节点执行过程中的任何异常，`process()`方法的调用方（`ExecutionEngine`）使用 `try-catch` 包裹。`catch` 块分别捕获 `NodeProcessException`、`cv::Exception`、`std::bad_alloc`、`std::exception` 和 (...)。

单个节点失败不会终止整个工作流的执行。执行引擎标记该节点为"失败"状态，其下游节点如果依赖该节点的输出也会因"输入数据为空"而跳过。执行完毕后在日志中汇总：总共 X 个节点，成功 Y 个，失败 Z 个，跳过 W 个。

7.3 校验时机

参数校验在两个时机执行：用户修改参数时（即时校验，控件自动裁剪到合法范围边界）和执行前 `validate()`（整体校验）。连线兼容性校验在拖拽连线释放时执行。`DAG` 环路检测在每次建立新连线时执行（DFS 染色法）。

7.4 GUI 模式与 CLI 模式的异常呈现

GUI 模式下：日志面板红色文字显示详细错误、失败节点卡片边框变红、状态栏显示最近错误简要信息。

CLI 模式下：错误输出到 `stderr`，格式[ERROR] 节点[{名称}]: {错误信息}。退出码：0=全部成功，1=部分失败，2= workflow 文件加载失败。

第八章 命令行模式

8.1 命令行接口

用法: ImageNodeTool [选项]

选项:

<code>--workflow <文件路径></code>	指定工作流文件路径 (.flow.json)
<code>--no-gui</code>	以命令行模式运行, 不启动图形界面
<code>--param <节点 ID>.<参数名>=<值></code>	覆盖工作流中指定节点的指定参数, 可多次使用
<code>--help</code>	显示帮助信息
<code>--version</code>	显示版本号

8.2 执行流程

1. 解析命令行参数。若有 `--help` 或 `--version`, 输出信息后退出。
2. 若有 `--no-gui` 和 `--workflow`, 进入 CLI 模式。
3. 加载工作流 JSON 文件。若文件不存在或解析失败, 输出错误到 `stderr` 并以退出码 2 退出。
4. 应用 `--param` 覆盖。逐条解析 "节点 ID.参数名=值", 找到对应节点设置参数值。若节点 ID 不存在或参数名无效, 输出警告但不终止。
5. 执行全量工作流。执行过程中的 INFO/WARNING/ERROR 信息输出到 `stdout/stderr`。
6. 执行完毕后输出汇总信息, 以退出码 0 或 1 退出。

8.3 使用示例

直接执行已保存的工作流

```
ImageNodeTool --workflow ./workflows/batch_watermark.flow.json --no-gui
```

覆盖输入输出路径进行批处理

```
ImageNodeTool --workflow ./workflows/process.flow.json --no-gui \
```

```
--param n001.file_path=./batch/img_001.png \
```

```
--param n010.file_path=./output/result_001.png
```

用户可以编写 PowerShell 或 Bash 脚本循环调用上述命令实现对几百张图的批处理。

第九章 内置 workflow 模板

9.1 模板存储

模板文件以 `.flow.json` 格式存放在程序可执行文件同级目录下的 `templates` 子目录中。程序启动时扫描该目录，将找到的所有 `.flow.json` 文件加入"新建 workflow"对话框的模板列表中。用户点击"新建 workflow"时弹出对话框，提供"空白 workflow"和所有模板供选择。用户也可以对自己搭建的 workflow 选择"另存为模板"。

9.2 三个内置模板的具体 workflow

模板一：论文四子图排版

节点构成：4 个 `ImageFileRead` -> 4 个 `Resize` -> 1 个 `MultiImageLayout` (2x2, 全部启用, 不合并, `show_labels` 开启, 标签(a)(b)(c)(d)) -> 1 个 `ImageFileWrite`。

用途：用户替换四个 `ImageFileRead` 的文件路径为自己的子图文件，调整 `Resize` 的目标尺寸，一键执行后得到标准的论文四子图排版图。

模板二：批量图片加水印

节点构成：1 个 `BatchImageRead` (读取输入目录) -> 1 个 `ListIterator` -> 1 个 `ImageFileRead` (读取水印图) -> 1 个 `ImageWatermark` (水印叠加, 位置右下, 透明度 0.3) -> `ListIterator` 的 `input_iteration_result` 端口回接 `ImageWatermark` 的输出 -> `ListIterator` 的 `output_results` -> 1 个 `BatchImageWrite`。

用途：用户设置输入目录、水印图路径和输出目录，一键为所有图片添加水印。

模板三：数据分析出图

节点构成：1 个 `CSVRead` -> 1 个 `DataFilter` (过滤空值) -> 1 个 `DataSort` -> 分两路：一路连向 `LineChart`, 一路连向 `BarChart` -> 两个图表输出连向一个 `MultiImageLayout` (1x2 横排) -> 1 个 `ImageFileWrite`。

用途：用户替换 CSV 路径，选择数据列和图表参数，一键生成并排的折线图和柱状图。

第十章 补充设计细节

10.1 节点 ID 生成规则

每个节点创建时自动生成唯一 ID。格式为"**n**"后跟 8 位十六进制随机数(如"**n3a7f02b1**")。使用 `QUuid::createUuid()` 生成后取前 8 位。在当前工作流中检查唯一性，若冲突则重新生成。

10.2 节点复制粘贴

复制操作将选中的节点（可多个）的类型、参数、相对位置关系序列化到剪贴板（使用 JSON 格式的 `QString`）。粘贴时反序列化创建新节点，位置相对于原始位置偏移(30,30)像素。节点间的连线如果两端节点都在复制范围内则一并复制，否则不复制。粘贴后的节点自动获得新 ID。

10.3 自动布局算法

工具栏中的"自动布局"功能将画布上所有节点按以下算法重新排列：

1. 对全图做拓扑排序，分层（入度为 0 的节点在第 0 层）。
2. 每层的节点纵向排列，层间水平排列（从左到右）。
3. 同层节点之间纵向间距 80 像素，层间水平间距 200 像素。
4. 起始位置为画布中心偏左上。
5. 排列完毕后自动调用"缩放适配"使所有节点可见。

10.4 DPI 信息写入

`ImageFileWrite` 节点在导出图片时，若工作流中该图像的创建源头（通过上游追溯找到的第一个指定了 DPI 的节点，如 `SolidColorGenerate`、`BlankCanvas`、`MultiImageLayout`）设置了 DPI 参数，则将 DPI 信息写入导出文件的元数据中。

实现方案：在 `ImageFileWrite` 的 `process()` 中，将 `cv::Mat` 先转为 `QImage`，设置 `dotsPerMeter` (`dpi * 39.3701`)，然后通过 `QImage::save` 写入文件。

10.5 多类型端口的参数面板动态行为

对于支持多类型输入的节点（如直方图 `Histogram` 的 `input_data` 端口接受 `{DataTable, Image}`），属性面板中的参数控件需要根据当前实际连入的数据类型动态显示或隐藏。

实现方式：`NodeBase` 提供虚方法 `onInputConnected(portName, dataType)` 和 `onInputDisconnected(portName)`，当连线建立或断开时回调。`PropertyPanel` 在刷新时读取节点

暴露的参数列表及各参数的可见性标记，只显示可见的参数。ImageCollector 节点也利用 onInputConnected 回调实现动态端口追加。

10.6 执行进度显示

执行工作流时，工具栏的"执行全部"按钮替换为进度条+百分比文字。进度值 = 已执行完毕的节点数 / 总待执行节点数 * 100%。执行完毕后进度条消失，按钮恢复。

在 CLI 模式下，每完成一个节点输出一行到 stdout: [进度] {已完成}/{总数} 节点[{名称}] 完成。

10.7 连线路径计算细节

贝塞尔曲线的四个点计算方式如下：

P0 为源端口中心坐标。P3 为目标端口中心坐标。设水平距离 $dx = \text{abs}(P3.x - P0.x)$ ，控制点偏移量 $\text{offset} = \max(50, \min(150, dx * 0.4))$ 。P1 = (P0.x+offset, P0.y)。P2 = (P3.x - offset, P3.y)。

当目标端口在源端口左侧时（即数据逆向连线），增大 offset 为 $\max(100, dx * 0.6)$ ，并让控制点的 y 坐标偏移使曲线绕行，避免穿过节点卡片。

10.8 画布网格吸附

节点移动时默认启用网格吸附（吸附步长 20 像素）。按住 Shift 拖拽时临时禁用吸附，实现自由定位。吸附逻辑：移动结束时将节点左上角坐标四舍五入到最近的 20 的倍数。

10.9 键盘快捷键汇总

Ctrl+N：新建工作流

Ctrl+O：打开工作流

Ctrl+S：保存工作流

Ctrl+Shift+S：另存为

Ctrl+Z：撤销

Ctrl+Shift+Z：重做

Delete：删除选中的节点/连线

Ctrl+C：复制

Ctrl+V：粘贴

Ctrl+A：全选画布上所有节点

F5：执行全部

Escape: 取消当前操作（关闭悬浮面板、取消正在拖拽的连线）

空格: 选中节点时按空格等同于点击预览按钮

10.10 程序启动流程

1. main()函数判断命令行参数。若有--no-gui, 创建 QApplication, 进入 CLI 模式。否则创建 QApplication, 进入 GUI 模式。

2. 无论哪种模式, 先调用 registerAllNodes()注册全部 135 种节点到 NodeFactory。

3. GUI 模式: 加载 QSS 样式表, 创建 MainWindow 并 show(), 进入事件循环。

4. CLI 模式: 创建 CLIRunner, 调用 run(), 根据返回值退出。

10.11 PatternBase 图案基类设计

五个学科图案节点（GenericPattern、CSPattern、ChemistryPattern、BioPattern、SocialSciPattern）都继承自 PatternBase 基类。PatternBase 继承 NodeBase, 提供:

addCommonParams()方法: 统一注册 8 个公共参数——width(Int, 800, 100-10000)、height(Int, 600, 100-10000)、primary_color(Color, [80,150,230,255])、secondary_color(Color, [40,80,120,255])、background_color(Color, [0,0,0,0])、density(DoubleSlider, 1.0, 0.1-2.0)、line_width(IntSlider, 1, 1-5)、seed(Int, 0, 0-999999)。

getCommonValues()方法: 一次性从参数中提取所有公共值（通过 Utility::arrayToColor 转换颜色参数）。子类在 defineNode()中调用 addCommonParams()后, 再注册自己独有的 pattern_type 参数, 实现参数复用。

10.12 渐变编辑器控件封装

GradientMap 节点使用的渐变编辑器控件是一个自定义 QWidget。界面显示水平渐变条（宽度 256 像素, 高度 24 像素）, 下方排列可拖拽的三角形色标。每个色标有两个属性: 位置（0.0-1.0）和颜色。交互: 双击色标打开颜色选择器修改颜色, 右键色标弹出删除选项, 单击渐变条空白处添加新色标（取该位置的当前渐变颜色）。色标之间的颜色自动线性过渡。最少保留 2 个色标。数据序列化为 color_stops 数组: [{"pos":0.0,"color":[0,0,0,255]}, {"pos":1.0,"color":[255,255,255,255]}]。